

Reinforcement learning for versatile, dynamic, and robust bipedal locomotion control

The International Journal of
Robotics Research
2025, Vol. 44(5) 840–888
© The Author(s) 2024
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/02783649241285161
journals.sagepub.com/home/ijr



Zhongyu Li¹ , Xue Bin Peng², Pieter Abbeel³, Sergey Levine³,
Glen Berseth^{4,5} and Koushil Sreenath¹

Abstract

This paper presents a comprehensive study on using deep reinforcement learning (RL) to create dynamic locomotion controllers for bipedal robots. Going beyond focusing on a single locomotion skill, we develop a general control solution that can be used for a range of dynamic bipedal skills, from periodic walking and running to aperiodic jumping and standing. Our RL-based controller incorporates a novel dual-history architecture, utilizing both a long-term and short-term input/output (I/O) history of the robot. This control architecture, when trained through the proposed end-to-end RL approach, consistently outperforms other methods across a diverse range of skills in both simulation and the real world. The study also delves into the adaptivity and robustness introduced by the proposed RL system in developing locomotion controllers. We demonstrate that the proposed architecture can adapt to both time-invariant dynamics shifts and time-variant changes, such as contact events, by effectively using the robot's I/O history. Additionally, we identify task randomization as another key source of robustness, fostering better task generalization and compliance to disturbances. The resulting control policies can be successfully deployed on Cassie, a torque-controlled human-sized bipedal robot. This work pushes the limits of agility for bipedal robots through extensive real-world experiments. We demonstrate a diverse range of locomotion skills, including: robust standing, versatile walking, fast running with a demonstration of a 400-meter dash, and a diverse set of jumping skills, such as standing long jumps and high jumps.

Keywords

Humanoid and bipedal locomotion, humanoids and animaloids, reinforcement learning, robot learning, legged robots, whole-body motion planning and control

Received 30 January 2024; Revised 5 July 2024; Accepted 6 August 2024

Senior Editor: Christian Ott

Associate Editor: Majid Khadiv

1. Introduction

Human environments are diverse and predominantly tailored for bipedal locomotion, and therefore the overarching goal in the bipedal robot community has been to develop robots capable of reliably operating within these environments. This paper aims to address one of the bottlenecks in achieving this objective: developing a solution for the control of diverse, agile, and robust legged locomotion skills, such as walking, running, and jumping for high-dimensional human-sized bipedal robots.

Although research on bipedal robot locomotion has been ongoing for decades (e.g., Raibert et al., 1984), developing a general framework capable of achieving robust control for diverse locomotion skills remains an open problem. The challenges arise from the complexity of the underactuated dynamics of bipedal robots and the distinct contact plan associated with each locomotion skill. First, given the floating base and resulting underactuated

dynamics, bipedal robots rely on contacts with the environment in order to move. The consistent (and hard-to-model) contacts lead to discontinuities in the trajectories, requiring contact mode planning and stabilization during

¹Department of Mechanical Engineering, University of California Berkeley, Berkeley, CA, USA

²School of Computing Science, Simon Fraser University, Burnaby, BC, Canada

³Department of Electrical Engineering and Computer Sciences, University of California Berkeley, Berkeley, CA, USA

⁴Department of Computer Science and Operations Research, Université de Montréal, Montreal, QC, Canada

⁵Mila – Quebec AI Institute, Canada

Corresponding author:

Zhongyu Li, Department of Mechanical Engineering, University of California Berkeley, Etchevery Hall, 2521 Hearst Ave, Berkeley, CA 94709, USA.

Email: zhongyu_li@berkeley.edu

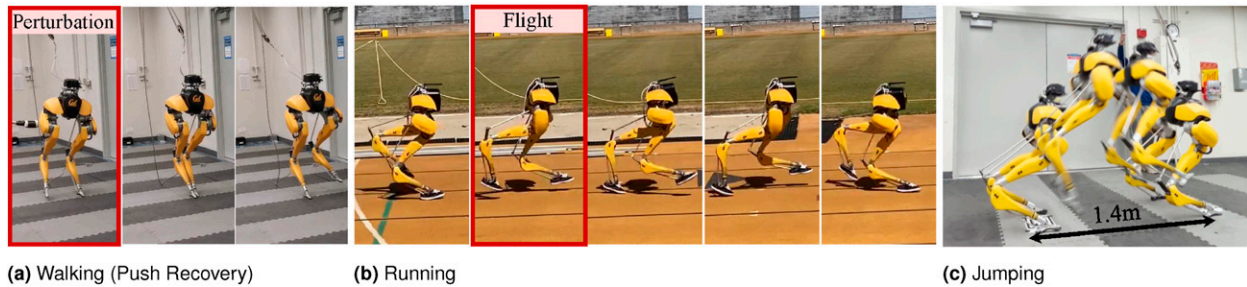


Figure 1. Cassie, a torque-controlled human-sized bipedal robot, performs various locomotion skills by using controllers developed through our framework. We present a unified RL framework that is able to train robust and agile controllers for a diverse range of highly dynamic skills, such as (a) walking, (b) running, and (c) jumping. The control policies developed by the proposed dual-history policy architecture and training system can adapt to changes in the robot dynamics, thereby enabling the direct transfer of learned policies to a real robot after training only in simulation without further real-world tuning. More experimental results can be best seen in the supplementary videos which are summarized in Table 1.

mode transitions. However, due to the high dimensionality and nonlinearity of bipedal robots, leveraging its full-order dynamics model for motion planning and control is computationally expensive and intractable for online applications. Second, the diverse nature of bipedal locomotion skills, whether periodic or aperiodic, presents significant challenges to the development of a simple and general framework. For instance, running, unlike walking, introduces more complexity due to a repeated flight phase where the robot is underactuated. For periodic skills such as walking or running, we can achieve *orbital stability* (Westervelt et al., 2003), by allowing for small corrections over multiple gait cycles. However, aperiodic motions, such as jumping, lack this inherent stability, which again poses additional challenges due to the requirement of finite-time stability (Goswami and Vadakkepat, 2009), which is further compounded by a large impact force upon landing.

In this work, we address the aforementioned challenges by leveraging reinforcement learning (RL) to create controllers for robots that feature high-dimensional, nonlinear dynamics in the real world. These controllers can leverage the robot’s proprioceptive information to adapt to the robot’s uncertain dynamics, which may be potentially time-varying due to wear and tear. These controllers are able to generalize to new environments and settings, exhibiting robust behaviors to unexpected scenarios by utilizing the agility of bipedal robots. Furthermore, our framework provides a general recipe for reproducing a variety of bipedal locomotion skills.

1.1. Objective of this work

The high-dimensionality and nonlinearity of a torque-controlled human-sized bipedal robot may at first seem like a daunting obstacle for developing effective controllers. However, these characteristics can also be advantageous by enabling complex agile maneuvers through the robot’s high-dimensional dynamics. Our objective is

to develop a general control framework for such bipedal robots to unlock their full potential. The goal of this framework is to enable a range of dynamic bipedal locomotion skills in the real world with a limited prescription on the resulting maneuvers. The skills we consider are shown in Figure 1, which include robust standing, walking, running, and jumping. These skills can also be used to perform a diverse repertoire of tasks, including walking at various velocities and heights, running at different speeds and directions, and jumping to various targets, all while maintaining robustness during real-world deployment. To achieve this, we leverage model-free RL, which allows the robot to learn through trial-and-error on the system’s full-order dynamics. In addition to real-world experiments, we also provide an in-depth analysis of the benefits of using RL for legged locomotion control and offer a detailed study into how to effectively structure the learning process to harness these advantages, such as adaptivity and robustness.

1.2. Terminology

First, we will establish the terminology we will use to describe various aspects of *legged locomotion* in this paper. The term *skill* is used to characterize a particular type of locomotion, which includes behaviors such as walking, running, and jumping. The robot can then leverage those skills to perform various *tasks*, which are defined by a given goal. For example, these tasks can include following different target velocities while walking, turning at different angles while running, or leaping towards different target locations while jumping. We use the term *versatile policy* to describe a control policy that can accomplish various tasks using one or more locomotion skills.

1.3. Contributions

This work advances the field of legged locomotion control for bipedal robots, with the following key contributions:

Table 1. A supplementary videos included in this paper.

Vid.	Content	Link
1	Summary video	https://youtu.be/sQEnDbET75g
2	Complete 400-meter dash	https://youtu.be/wzQtRaXjvAk
3	Suppl. walking experiments	https://youtu.be/vUewNUtSG3c
4	Suppl. running experiments	https://youtu.be/ad3ZrvUzbXM
5	Suppl. jumping experiments	https://youtu.be/aAPSZ2QFB-E

Development of a new framework for general bipedal locomotion control: We introduce a general RL framework for bipedal robots that is effective across a wide range of locomotion skills, spanning periodic skills such as walking and running, aperiodic skills such as jumping, and stationary skills such as standing. The resulting controllers can be directly deployed on a real robot without any additional tuning or training on the physical system.

Novel design choices for RL-based control policy: We present a new dual-history architecture for non-recurrent RL policies, which integrates both the long and short input/output (I/O) history with explicit length of the robot, for RL-based control. When combined with the proposed training strategy that trains the base policy with the short history jointly with the long history encoder, this architecture demonstrates state-of-the-art performance in learning dynamic bipedal locomotion control, offering consistent benefits across various locomotion skills, which are validated in both simulation and real world experiments, as detailed in Sec. 7 and discussed in Sec. 11.1.

Empirically investigating adaptivity in RL controllers: In the control theory community, there have been notable efforts in bridging adaptive control and RL, like Annaswamy (2023). In this work, we conduct a detailed empirical study to investigate the adaptivity of the control policy developed through RL. We show that the adaptivity from RL includes not only time-invariant shifts in the dynamics but also time-variant changes like contact events. We validated this not only in simulation, as described in Sec. 8, but also by several real-world (zero-shot-transferred) experiments such as in-place walking with minimal drift and targeted jumping using a bipedal robot as detailed in Sec. 10.

Improving robustness in RL controllers: Our study introduces a new dimension of robustness in RL-based control policies. Beyond the commonly-used dynamics randomization in robotics (Peng et al., 2018), we demonstrate that *task randomization*, which trains the policy on a wide range of tasks, significantly enhances robustness by enabling task generalization. This approach, distinct from dynamics randomization, provides the robot with

disturbance compliance, which is demonstrated in both simulation and real-world experiments in Sec. 9.

Extensive real-world validation and demonstrations of novel bipedal locomotion capabilities: Our system is able to reproduce a wide variety of locomotion skills using Cassie, a human-sized bipedal robot, in the real world as detailed in Sec. 10. Cassie can track varying commands with negligible tracking errors and significant robustness to unexpected disturbances, including walking (Figures 14 and 18(a)), running (Figures 20 and 24), and jumping (Figures 25 and 27). Additionally, we demonstrate novel capabilities for bipedal robots, such as robust standing recovery using different skills (Figure 11), robust walking (Figures 17 and 19) with control performance being consistent over a long time frame (Figure 14), completing a 400-meter dash using a running controller (Figure 20), and performing diverse bipedal jumps (Figures 25 and 26) including standing long and high jumps (Figure 25). The experiments are shown in the supplementary videos provided in Table 1.

This paper builds on our preliminary work presented at Robotics: Science and System (Li et al., 2023b), which focused on bipedal jumping. We expanded our RL framework to encompass a broader range of skills, proving its applicability to both aperiodic skills like jumping and periodic skills such as walking and running. The effectiveness of this framework is consistently demonstrated across these skills through extensive real-world experiments. Furthermore, the added ablation study and benchmarks further explore the design decisions in our framework, shedding light on the crucial elements that enhance adaptivity and robustness in RL-based locomotion control.

We hope this work could serve as a milestone towards creating robust, versatile, and dynamic bipedal locomotion control in the real world, providing insights and guidance for future applications of RL for legged locomotion control and other complex systems.

2. Related work

Locomotion control for a bipedal robot requires solving a problem that tightly couples motion planning and control of the robot's whole body. Previous efforts can be broadly categorized into two main approaches: (1) model-based optimal control (OC), and (2) model-free reinforcement learning (RL). While this work is based on RL, the following review gives equal attention to relevant studies from both model-based OC and model-free RL domains. We hope it can provide a succinct survey of the recent trends in legged locomotion control, which could be informative for readers from both perspectives, with a primary focus on bipedal robots. In the specific case of Cassie, which serves as the experimental platform in this work, we provide an overview of the most related work for its locomotion control in Table 2.

Table 2. Comparison of our work with prior studies on implementing real-world locomotion controllers for the bipedal robot Cassie. Our work provides an introduction to a general control framework to realize diverse periodic and aperiodic bipedal locomotion skills including robust walking and standing, fast running, and versatile jumping in the real world.

Walking skill						
Previous literature	Implementation	Variable velocity	Variable height	Consistency over time	Consistent perturbation	Change of terrain
Gong et al. (2019)	HZD, Model: Full-order	Yes	No	No	No	No
Li et al. (2020); Yang et al. (2022)	HZD, Model: Full-order	Yes	Yes	No	No	No
Reher and Ames (2021)	HZD, Model: Full-order	Yes	No	Not demonstrated	Not demonstrated	No
Xie et al. (2020)	RL, Model-free	Yes	No	Not demonstrated	Not demonstrated	No
Siekmann et al. (2020)	RL, Model-free	Forward walking only	No	Not demonstrated	Not demonstrated	No
Li et al. (2021)	RL, Model-free	Yes	Yes	Not demonstrated	Yes (untrained)	No
Siekmann et al. (2021a)	RL, Model-free	Yes	No	Not demonstrated	Not demonstrated	Yes (small, trained)
Siekmann et al. (2021b)	RL, Model-free	Yes	No	Not demonstrated	Not demonstrated	Yes (trained)
Dao et al. (2022)	RL, Model-free	Forward walking only	No	Not demonstrated	Yes (trained)	No
Yu et al. (2022)	RL, Model-free	Sharp turn only	No	Not demonstrated	Not demonstrated	No
Gong and Grizzle (2022)	OC, Model: ALIP	Yes	No	Not demonstrated	Not demonstrated	Yes (unmodeled)
Xiong and Ames (2022)	OC, Model: H-LIP	Yes	Yes	Not demonstrated	Not demonstrated	Yes (small, unmodeled)
Agrawal (2022)	OC, Model: Centrodial	Yes	No	Not demonstrated	Not demonstrated	Yes (small, unmodeled)
Ours	RL, Model-free	Yes	Yes	Yes	Yes (untrained)	Yes (small, untrained)
Running skill						
Previous literature	Implementation	Controlled velocity	Transition from/to standing	100 m dash finish time	400 m dash finish time	Uneven terrain
Siekmann et al. (2021a)	RL+	No	Not demonstrated	Not demonstrated	Not demonstrated	Yes (small, trained)
Yang and Posa (2023)	OC+	Yes	Not demonstrated	Not demonstrated	Not demonstrated	No
Crowley et al. (2023)	RL with noticeable flight phase	No	Only transit from standing	24.73s	Not capable of turning	No
Ours	RL with noticeable flight phase	Yes, w/ sharp turn (untrained)	Yes	27.06s	2 min 34s	Yes (large, trained)
Jumping skill						Maximum leap distance
Previous literature	Implementation	Targeted landing	Apex foot clearance	Longest flight phase	(Forward, backward, lateral, turning, elevation)	
Xiong and Ames (2018)	Aperiodic Hop by OC [‡]	No	0.18m	0.42s	In-place	
Yang and Posa (2021)	Aperiodic Hop by OC [‡]	No	0.15m*	0.33s*	In-place	
Siekmann et al. (2021a)	Periodic Hop by RL [‡]	No	0.16m*	0.33s*	Tracking a forward speed	
Yang and Posa (2023)	Aperiodic Jump by OC	No	0.42m*	0.33s*	(0, 0, 0, 0, 0.41 m)	
Ours	Aperiodic Jump by RL	Yes	0.47m	0.58s	(1.4m, -0.3m, ±0.3m, ± 55°, 0.44m)	

+ Although being termed as running, the demonstrated flight phase, foot clearance during flight, and speed in the real world are not comparable to the rest of the work listed here.

[‡] The demonstrated flight phase, apex foot clearance, and the resulting impacts upon landing are not comparable with the rest, so we use “hop” to distinguish from a “jump”.

* Not provided in the paper and the listed value is roughly estimated from the accompanying video.

2.1. Model-based optimal control for bipedal robots

Locomotion control for bipedal robots can be formulated as an optimal control (OC) problem (Wensing et al., 2023, equation (1)), with the robot's dynamics model appearing as a motion constraint. To manage the computational complexity of solving constrained optimization problems, this method often employs a cascaded optimization framework, starting with generating long horizon reference trajectories to low-level motion control and immediate reactive control, with progressively higher control rates and different modeling choices at each stage.

2.1.1. Choice of models. The robot's full-order dynamics and contact models can be leveraged to optimize for a bipedal robot's trajectory and corresponding inputs for a specific behavior (Fevre et al., 2020; Hereid et al., 2019; Marcucci et al., 2016). However, given that bipedal robots typically have high-dimensional nonlinear dynamics, the use of detailed models is generally limited to offline optimization. Notably, the Hybrid Zero Dynamics (HZD) method (Westervelt et al., 2003) uses the bipedal robot's full-order model to design attractive periodic gaits offline with online feedback controllers to enforce the virtual constraints (Gong et al., 2019; Hereid et al., 2018; Reher and Ames, 2021). For online trajectory optimization, reduced-order models that simplify robot dynamics are necessary. Various reduced-order models, such as centroidal dynamics (Orin et al., 2013), the linear inverted pendulum (LIP) (Kajita et al., 2001) and its variants like SLIP (Rummel et al., 2010), ALIP (Gong and Grizzle, 2022), H-LIP (Xiong and Ames, 2022), are used for online optimization of reduced-order dynamics, like the Center of Mass (CoM) and/or Center of Pressure (CoP) (Fernbach et al., 2020; Pratt et al., 2012; Vukobratović and Borovac, 2004). These models also enable trajectory tracking control (Daneshmand et al., 2021; Gong and Grizzle, 2022; Kuindersma et al., 2014). Reactive controllers, like whole-body control (WBC, Sentis and Khatib (2006)), translate these reduced-order states to joint-level inputs, operating as a fast-solving QP that considers various constraints *without* unrolling the full-order dynamics over a horizon like Moro and Sentis (2019), Bouyarmane and Kheddar (2011), and Wensing and Orin (2016). However, such a cascaded OC method does not fully utilize the robot's potential agility due to the limitations of reduced-order models and/or the inability of online controllers to re-plan whole-body maneuvers. Our work overcomes these limitations by directly learning on the robot's full-order dynamics and approximately solving the OC problem by model-free RL. The resulting controllers are able to leverage the full agility potential of the underlying system.

2.1.2. Contact planning. Legged robots require making and breaking contact with the environment. The resulting velocity

jumps at contact make the robot's trajectory non-smooth. This makes it challenging to solve an OC problem that decides each leg's contact mode throughout the movement (Posa et al., 2014). To simplify this, many studies, including most of the above-mentioned work, pre-define fixed contact sequences for specific locomotion skills, such as walking (Caron et al., 2019; Hereid et al., 2019; Xiong and Ames, 2022), running (Ma et al., 2017; Sreenath et al., 2013; Takenaka et al., 2009b), and jumping (Chignoli et al., 2021; Goswami and Vadakkepat, 2009; Qi et al., 2023). However, pre-defined handcrafted contact sequences may not be optimal. For instance, different jumping tasks might require varying contact schedules, like extended flight times for longer jumps. Consequently, there are efforts to integrate contact planning with trajectory optimization by computationally expensive mixed-integer programming (Deits and Tedrake, 2014; Ibanez et al., 2014). There are also attempts to avoid using explicit discrete variables, which results in contact-implicit methods, by enforcing complementarity constraints (Dai et al., 2014; Drnach and Zhao, 2021; Posa et al., 2014; Yunt and Glocker, 2006) or utilizing a bilevel optimization (Landry et al., 2022; Le Cleac'h et al., 2024; Zhu et al., 2021). However, given the high-dimensional nonlinear dynamics of bipedal robots, contact explicit or implicit planning is still limited to offline optimization. As we will see, our method enables choosing contact plans in real-time for deployment in the real world.

2.1.3. Scalability to different locomotion skills. The scalability of model-based OC across various bipedal locomotion skills and tasks is a significant challenge, largely due to the task-specific nature of robot models and control frameworks (Meduri et al., 2023). For example, extending the HZD approach used for controlling 2D bipedal walking (Sreenath et al., 2011) to running (Sreenath et al., 2013) or 3D walking (Da et al., 2016) requires considerable efforts in finding appropriate periodic gaits and then designing specific controllers to stabilize them. Furthermore, HZD's reliance on the stability of periodic gaits limits its extension to aperiodic skills like jumping. This limitation is also observed in LIP-based methods, where separate frameworks and models are required for walking (Takenaka et al., 2009a) and running (Takenaka et al., 2009b) while jumping skills are excluded due to LIP's assumption of an approximately constant CoM height as in Boroujeni et al. (2021). Although in-place jumps have been achieved through other carefully engineered cascaded OC frameworks over different jumping phases, such as Xiong and Ames (2018), Kojima et al. (2019), and Qi et al. (2023), for each different jumping task, these require starting from scratch with offline trajectory optimization, often overlooking lateral or turning motions. Furthermore, it is very challenging to develop a single model-based controller capable of managing diverse *targeted* bipedal jumping tasks in the real world. This is because besides accomplishing a jump,

the controller also needs to quickly adapt to the dynamics of the robot hardware and produce an accurate translational velocity at the take-off to land at the targets located at different places, as shown in this work.¹ The model-based OC community, as seen in works like Yang and Posa (2023), is making promising strides toward developing general tracking controllers that are invariant to impact. These could potentially handle different contact sequences like walking, running, and jumping, but still rely on *task-specific* optimized trajectories.

2.2. Model-free reinforcement learning on legged locomotion control

Recent developments in deep RL have brought about exciting progress in creating locomotion controllers for quadrupedal robots in the real world, such as Margolis et al. (2022), Chen et al. (2023), Feng et al. (2023), and Fu et al. (2023). However, due to the inherently less stable nature of bipedal robots, methods successful with quadrupeds might not directly apply to bipedal systems, as an example seen in Kumar et al. (2022). Therefore, within the bipedal robotics community, there are different strategies to employ RL tailored to the challenges of high dimensional nonlinearity.

2.2.1. Control policy structure. In RL-based locomotion control, the structure of the control policy is largely influenced by how observations are formulated, particularly through the use of robot states-only history (with no robot's input history) or I/O history (with both the robot's input and output). For quadrupedal robots, there is no consensus on the history length to use: it ranges from a short I/O history of 1 to 15 timesteps (Escontrela et al., 2022; Huang et al., 2023; Hwangbo et al., 2019) to longer sequences over 50 timesteps involving states-only history (Lee et al., 2020; Miki et al., 2022; Shao et al., 2021) or I/O history (Kumar et al., 2021). The policy (neural network) architecture is chosen depending on the history length, with MLPs suited for shorter histories and recurrent units needed for longer sequences. Real-world deployments on quadrupedal robots have shown comparable performance across these varying history lengths. For bipedal robots, a trend toward longer history lengths is observed, evolving from a single timestep state feedback with a need of residual action (Castillo et al., 2022; Rodriguez and Behnke, 2021; Xie et al., 2018, 2020), to a short I/O history (Li et al., 2021), to a longer sequence of states-only history (Siekmann et al., 2020) or I/O history (Kumar et al., 2022; Radosavovic et al., 2024). While utilizing long history for robotic control as suggested by Peng et al. (2018) is a common strategy, most prior ablation studies focus on contrasting long histories with immediate state or I/O feedback through MLPs, like Peng et al. (2018); Lee et al. (2020); Kumar et al. (2021); Siekmann et al. (2020), with less exploration into shorter I/O histories. A recent study, Singh et al. (2023), reported

that, short state histories yield better learning performance than a longer history in bipedal humanoid robot control, aligning with our finding in this work. This raises a question: while long histories are expected to enhance performance in RL-based control, how can their benefits be fully utilized? Our study presents an effective solution: a dual-history approach.

2.2.2. Sim-to-real transfer. There are some attempts using RL to directly collect data and train on the robot hardware, such as Haarnoja et al. (2019); Wu et al. (2023b); Smith et al. (2023), and there are also alternatives on leveraging a pre-training stage in simulation and fine-tuning on hardware like Peng et al. (2020); Smith et al. (2022); Westenbroek et al. (2022), most of which are deployed on small-sized quadrupedal robots. However, for human-sized bipedal robots, performing hardware rollouts is expensive, making it more appealing to directly transfer diverse dynamic bipedal skills from simulation to hardware. Achieving zero-shot transfer needs extensive dynamics randomization in simulation as suggested by Peng et al. (2018, 2020). There are two primary approaches to training policies under randomized dynamics: (1) end-to-end training with a history of robot measurements or I/O, which has been applied to bipedal robots like Siekmann et al. (2020, 2021a); Li et al. (2021), and (2) policy distillation where, with separated training stages, an expert policy with access to privileged environmental information supervises the training of a student policy with proprioceptive feedback. This method, initially developed for small servo-controlled bipedal robots in Yu et al. (2019), has been adapted to quadrupeds using strategies like Teacher-Student (TS, Lee et al. (2020)) or RMA (Kumar et al., 2021), and is prevalent in quadrupedal robot community (e.g., Fu et al. (2021); Ji et al. (2022); Margolis and Agrawal (2023)). Though policy distillation offers advantages in quadrupedal locomotion controls (Kumar et al., 2021), its extension to torque-controlled bipedal robots benefits from an additional finetuning stage like Kumar et al. (2022). In this work, we demonstrate that end-to-end training is a more effective approach for developing controllers for bipedal robots encompassing a variety of dynamic locomotion skills.

2.2.3. Scalability to different locomotion skills. Leveraging RL to learn diverse locomotion skills or tasks with a single policy poses a challenge that arises from the need to optimize multiple objectives for different tasks, as noted in Kalashnikov et al. (2021). Initially, efforts in this field focused on single skills with fixed tasks, like just walking forward in Peng et al. (2020); Siekmann et al. (2020); Kumar et al. (2021). In developing a single-skill policy capable of multiple tasks, methods that provide varying commands for tracking different walking velocities without specifying reference motions have been considered, like Hwangbo et al. (2019); Fu et al. (2021); Cheng

et al. (2023). While this method is viable for quadruped robots, it demands extensive reward tuning and may not suit bipedal robots due to their higher dimensionality. For bipedal robots, approaches such as providing parameterized reference motions (Li et al., 2021) or using policy distillation from task-specific policies (Xie et al., 2020) have been considered. Some methods also involve commanding periodic contact sequences, resulting in diverse but periodic bipedal gaits like Siekmann et al. (2021a). However, the prescriptive contact sequence restricts the robot’s potential to optimize its contact strategy for improved stability. Additionally, limitations arise when such a policy attempts to accomplish specific tasks, leading to follow-up works that require specialized training for distinct behaviors. This includes requiring hand-tuned strategies for transitioning between fast straight running and standing (Crowley et al., 2023), load carrying (Dao et al., 2022), or sharp turns during walking (Yu et al., 2022). There are other attempts that sought to create a single policy that can perform diverse skills, regardless of periodicity, for bipedal humanoids using adversarial motion priors as developed in Peng et al. (2021) in simulation. While this has seen success in transfer to real quadrupeds like Escontrela et al. (2022); Wu et al. (2023a); Li et al. (2023a), it is still an open question if more dynamic bipedal skills can be transferred to the real world due to larger sim-to-real gaps. In light of these challenges, our work strikes a balance in versatility for bipedal robots. We focus on developing skill-specific control policies that can perform a diverse set of tasks while maintaining a general framework suitable for developing different skills.

3. Overview

In this section, we provide an overview of the entire paper and proposed RL system for general bipedal locomotion control, as illustrated in Figure 2. We first provide an introduction of the importance of utilizing the robot’s I/O history in the locomotion control in Sec. 4. In this section, we showcase that the robot’s long I/O history can enable system identification and state estimations during real-time control, from both control and RL perspectives. This results in the design of the backbone of this work: a new control architecture that utilizes a dual-history of both the bipedal robot’s long-term and short-term I/O history, which is presented in Sec. 5. Specifically, such a control architecture does not only utilize the long history but also exploits explicit short history of the robot. The proposed dual-history structure enables effective use of robot’s history, as the long history brings adaptivity (validated in Sec. 8) and the short history complements the use of long history by enabling better real-time control (substantiated in Sec. 7). This control policy, which is represented by a deep neural network, is optimized by model-free RL in Sec. 6. Since this paper aims to develop a controller that is

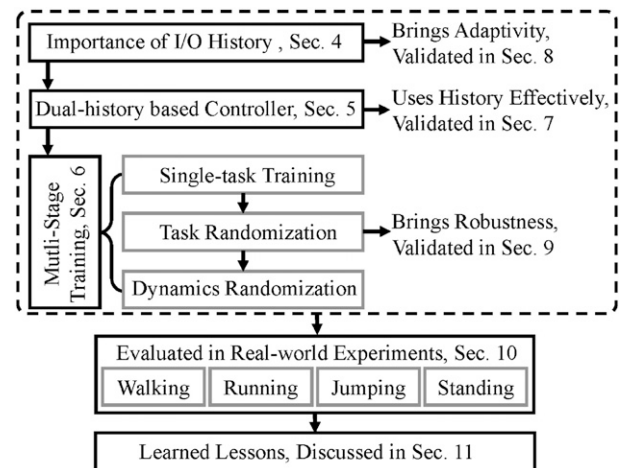


Figure 2. Overview of this paper. First, Sec. 4 introduces the formulation of the locomotion control problem and the importance of utilizing the robot’s I/O history. The details of our dual-history-based control architecture for various bipedal locomotion skills are presented in Sec. 5, followed by the training scheme discussed in Sec. 6. Then, detailed studies are conducted to validate the advantages of the proposed policy structure in Sec. 7, sources of adaptivity in Sec. 8, and we investigate the sources of the robust behaviors observed in the proposed RL-based controller in Sec. 9. Extensive experiments using the proposed RL-based locomotion controllers to enable Cassie to perform robust standing, walking, running, and jumping skills are presented in Sec. 10. Insights and discussions for readers interested in applying RL to train bipedal robots are provided in Sec. 11.

capable of accomplishing diverse tasks using highly dynamic locomotion skills, the training in Sec. 6 is characterized by multi-stage training in simulation. This training strategy provides a structured curriculum, starting with a single-task training where the robot focuses on a fixed task, followed by task randomization that diversifies the tasks the robot is trained on, and concluded by dynamics randomization which alters the dynamics parameters of the robot. Such a training strategy is able to provide a versatile control policy that can perform a large variety of tasks and zero-shot transferred to the robot hardware. Furthermore, task randomization can also enhance the robustness of the resulting policy through the generalization among different learned tasks. We show that such robustness results in compliant behaviors to disturbance, which is “orthogonal” to the one brought by dynamics randomization. This is validated in Sec. 9.

By using this framework, we obtain skill-specific versatile policies for walking, running, and jumping for a bipedal robot, Cassie. We evaluate the effectiveness of these control policies extensively in the real world in Sec. 10. As we will see, by the adaptivity of the proposed control architecture, the resulting policies can accomplish various tasks with minimal degradation from simulation to the real world and maintain consistency over a long timespan (more than 1 year). Because of the robustness brought by the

proposed training strategy, especially the task randomization, the resulting policies showcase complicated recovery maneuvers when facing unexpected disturbances in the real-time and real world. We further summarize the insights and lessons learned during the development of this work in a discussion in Sec. 11, followed by a conclusion and future work in Sec. 12.

4. Background

In this section, we introduce Cassie, our main experimental platform, along with its dynamics model. We then frame the bipedal locomotion control problem as a Partially Observable Markov Decision Process (POMDP), laying the groundwork for training policies with RL. We also emphasize the importance of incorporating the robot's input and output (I/O) history in feedback control from both model-based control and model-free RL perspectives.

4.1. Cassie robot model

4.1.1. Floating-base coordinates. As illustrated in Figure 3, Cassie is a human-sized bipedal robot, standing at a height of 1.1 m and weighing 31 kg. On both the Left and Right (L/R) legs are 7 joints, consisting of the abduction q_1 , rotation q_2 , thigh q_3 , knee q_4 , shin q_5 , tarsus q_6 , and toe pitch q_7 . This results in a total of 14 joints ($\mathbf{q}_j \in \mathbb{R}^{14}$). Among those, $q_{1,2,3,4,7}^{L/R}$ are *actuated* by motors (which is denoted as $\mathbf{q}_m \in \mathbb{R}^{10}$), while the shin and tarsus joints ($q_{5,6}^{L/R}$) are *passive* and connected by leaf springs, as annotated in Figure 3. Cassie has a floating base $\mathbf{q}_b$ with 6 Degree-of-Freedom (DoFs) which are translational positions $q_{x,y,z}$ and rotational positions $q_{\phi,\theta,\psi}$. The generalized coordinates of the full system $\mathbf{q}$ can be represented as $\mathbf{q} = [\mathbf{q}_b, \mathbf{q}_j] \in \mathbb{R}^{20}$.

4.1.1.1. The observable states. Since the robot is a second-order mechanical system, we can denote the robot's states as its generalized coordinates $\mathbf{q}$ and their time derivatives $\dot{\mathbf{q}}$. However, there is only a part of states we can *reliably* measure or estimate using the robot's onboard sensors. We denote observable states as $\mathbf{o} \in \mathbb{R}^{26}$ and it contains motors positions and their velocities ($\mathbf{q}_m, \dot{\mathbf{q}}_m$), which can be respectively measured and estimated by the joint encoders. The base orientation $q_{\phi,\theta,\psi}$ can be measured by an IMU, and base linear velocity $\dot{q}_{x,y,z}$ can be estimated by an EKF as used in (Xie et al., 2020).

4.1.2. Full-order dynamics model. Cassie has a floating-base, a total of $n = 20$ DoFs, and $n_a = 10$ actuated joints. Its dynamics equation can be obtained by the Euler-Lagrange method:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \mathbf{B}\boldsymbol{\tau} + \boldsymbol{\kappa}_{sp}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\zeta}_{ext}, \quad (1)$$

where $\mathbf{M} \in \mathbb{R}^{n \times n}$, $\mathbf{C} \in \mathbb{R}^{n \times n}$, and $\mathbf{G} \in \mathbb{R}^n$ denote the generalized mass matrix, centrifugal and Coriolis matrix, and

generalized gravity, respectively. The right-hand side of (1) contains the system inputs which includes generalized control input (motor torques) $\boldsymbol{\tau} \in \mathbb{R}^{n_a}$ (distributed by $\mathbf{B} \in \mathbb{R}^{n \times n_a}$), state-dependent spring torques $\boldsymbol{\kappa}_{sp}(\mathbf{q}, \dot{\mathbf{q}})$, and generalized external force $\boldsymbol{\zeta}_{ext}$. The $\boldsymbol{\zeta}_{ext}$ groups all the external forces applied from the environment, including foot contact wrenches denoted as $\mathbf{J}_c^T \mathbf{F}_c$ and any joint-level friction or perturbation wrenches added to the robot. Specifically, for the contact wrenches, $\mathbf{J}_c(\mathbf{q}) \in \mathbb{R}^{n_c \times n}$ is the contact Jacobian and the dimension of contact wrenches (n_c) will vary when the robot has a different number of support legs with the ground (ranging from no stance leg to two stance legs).

4.1.3. System identification and adaptive control. For locomotion control of bipedal robots like Cassie, we can borrow ideas from system identification and adaptive control, which can adapt to changes in the dynamics of the robot. One approach is to leverage a sequence of the past system's input and output (I/O history) to identify the system parameters and change the control law over time (see Landau et al., 2011, Ch. 9). For example, given the dynamical system (bipedal robot) governed by (1), by utilizing a sequence of robot's input ($\boldsymbol{\tau}$) and robot's output ($\mathbf{o}$), with an assumption that the system states ($\mathbf{q}$ and their time derivatives) can be approximated by a sequence of $\mathbf{o}$ as derived in Lim et al. (2023), the modeling parameters in $\mathbf{M}$, $\mathbf{C}$, $\mathbf{G}$, $\boldsymbol{\kappa}$ and external forces $\boldsymbol{\zeta}_{ext}$ can be identified and therefore the control strategy in (1) can be adjusted accordingly. There are two main design choices in adaptive control: *indirect* and *direct* methods. In the *indirect* approach, system parameters are first explicitly estimated (see Ljung, 1998) and the control law is indirectly adjusted based on the identified parameters. In contrast, the *direct* approach alters the control law directly based on the system's I/O history. While both methods have their advantages and disadvantages as discussed in Annaswamy (2023), the *indirect* approach may struggle with unknown system parameters or the ones that are challenging to identify. Therefore, in this paper, we choose to develop a control policy through model-free RL by learning from the bipedal robot's I/O history to directly adapt to changes in the robot's full-order model (1). Our method is designed to align with the *direct* adaptive control category in contrast to the strategies like Teacher-Student (Lee et al., 2020) or RMA (Kumar et al., 2021) that need to estimate pre-selected system parameters, which can be viewed as *indirect* methods.

4.2. RL preliminaries

4.2.1. Legged locomotion control as a POMDP. The locomotion control on bipedal robots can be formulated as a Partially Observable Markov Decision Process (POMDP). At each timestep t , the environment is at state $\mathbf{s}_t$, the agent (i.e., the robot) makes an observation $\mathbf{o}_t$ from the environment,

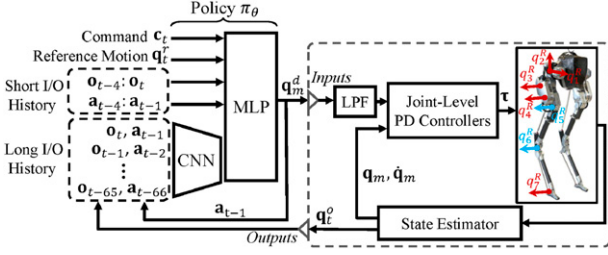


Figure 3. The proposed RL-based controller architecture that leverages a dual-history of input ($\mathbf{a}$) and output ($\mathbf{o}$) (I/O) from the robot. The control policy π_θ , operating at 33 Hz, processes a 2-second long I/O history. This data is initially encoded via a 1D CNN along its time axis before being merged with a base MLP. In addition, a short history spanning four timesteps is directly input into the base MLP, combined with skill-specific reference motion $\mathbf{q}_r^d$ and variable commands $\mathbf{c}_t$ that parameterize the tasks. The policy outputs desired motor positions $\mathbf{q}_m^d$ as the robot's actions, which are then smoothed using a low-pass filter (LPF). These filtered outputs are employed by joint-level PD controllers operating at 2 kHz to specify motor torques $\boldsymbol{\tau}$. This architecture is general for various locomotion skills like standing, walking, running, and jumping. This figure also annotates the generalized coordinates for Cassie, which include actuated joints ($q_{1,2,3,4,7}^{L/R}$, marked as red) and passive joints ($q_{5,6}^{S/R}$, marked as blue).

takes an action $\mathbf{a}_t$ and interacts with the environment, the environment transits to a new state $\mathbf{s}_{t+1}$, and the agent receives a reward r_t . Such a process will repeat until the end of the episode of length T . In a POMDP, the robot only has access to partial information of the environment, for example, the bipedal robot can only access the observable states $\mathbf{o}$ instead of the full environment states, which include the full coordinates $\mathbf{q}$ and their time derivatives as explained in Sec. 4.1.1. In a POMDP, the observation $\mathbf{o}_t$ is obtained by the observation function $O(\mathbf{o}_t|\mathbf{s}_t, \mathbf{a}_{t-1})$ conditioned on both the current environment state and action the agent has taken. The RL objective is to maximize the expected return, $\mathbb{E}[\sum_{t=0}^T \gamma^t r_t]$, by finding an optimal policy π^* that selects an action $\mathbf{a}_t$ at each time step. The expected return is the sum of discounted rewards collected by the agent throughout an episode, with γ representing a discount factor.

4.2.2. Solving POMDP with I/O history. Solving a POMDP can be formulated as finding the optimal policy π^* that maps the *process history* to the optimal action. The *process history* at timestep t contains the entire history of the agent's observations and actions, i.e., $\{\langle \mathbf{o}_t, \mathbf{a}_{t-1} \rangle, \langle \mathbf{o}_{t-1}, \mathbf{a}_{t-2} \rangle, \dots, \langle \mathbf{o}_1, \mathbf{a}_0 \rangle\}$ (Spaan, 2012). In the control context, it is the robot's I/O history. One method is to update the belief state from the process history with Bayesian filters at each timestep, and then transform the POMDP into a belief state MDP. Alternatives can be to formulate a Finite State Controller (FSC) that contains the memory of agent's past observations and actions through *internal states* to constitute and optimize a policy graph (Spaan, 2012, Ch. 4). While the full history is needed to

find an optimal policy, it is computationally intractable. Instead, a finite memory can be kept in the FSC and the POMDP can be solved *approximately* (see Meuleau et al., 1999). In all of these methods, the robot's process history (i.e., I/O history) is utilized for solving a POMDP. Therefore, we choose to train a policy that has a finite memory (with a fixed length h) of robot's I/O pair, i.e., $\pi(\mathbf{a}_t|\mathbf{o}_{t:t-h}, \mathbf{a}_{t-1:t-h-1})$. Furthermore, in this work, to enable the robot to accomplish a variety of goals, we parameterize tasks using commands $\mathbf{c}$, and the policy $\pi(\mathbf{a}_t|\mathbf{o}_{t:t-h}, \mathbf{a}_{t-1:t-h-1}, \mathbf{c}_t)$ is now conditioned on both the robot's I/O history and the given command $\mathbf{c}_t$.

4.2.3. Task parameterization. Different locomotion tasks are parameterized by a command and this parameterization may vary for different locomotion skills. For the walking skill, the command $\mathbf{c}_{\text{walk}} \in \mathbb{R}^4$ is defined as $[\dot{q}_{x,y}^d, q_{z,\psi}^d]$, which specifies the desired walking velocity in sagittal $\dot{q}_x^d$ and lateral $\dot{q}_y^d$ directions, desired walking height q_z^d , and desired turning yaw angle q_ψ^d , respectively. For the running skill, the command is represented by $\mathbf{c}_{\text{run}} = [\dot{q}_{x,y}^d, q_\psi^d] \in \mathbb{R}^3$. For jumping, the command $\mathbf{c}_{\text{jump}} \in \mathbb{R}^4$ is defined as $[q_{x,y,\psi}^d, e_z^d]$, which specifies the target planar position $q_{x,y}^d$, turned angle q_ψ^d , and change of elevation e_z^d in the vertical jump direction after the robot lands.

5. Bipedal locomotion controller with I/O history

In this section, we describe the proposed general control framework for bipedal robots, leveraging the robot's dual I/O history as a fundamental element to enable transfer to environments with uncertain dynamics. This control architecture is the backbone of the RL system developed in this work, which will show advantages in terms of control performance in both the simulation and the real world.

5.1. Control framework

Our locomotion control policy π_θ is represented by a deep neural network with parameters θ . The same policy architecture will be used for a variety of different locomotion skills, and as such, it is crafted to rely on minimal skill-specific design choices. As shown in Figure 3, the policy outputs the desired robot motor positions $\mathbf{q}_m^d \in \mathbb{R}^{10}$ for the robot, which is the agent's action $\mathbf{a}_t$. The action is first smoothed by a Low Pass Filter (LPF) (Peng et al., 2020), details of which are discussed in Appendix 1. The filtered actions are then used by joint-level PD controllers to calculate the motor torques $\boldsymbol{\tau}$ that will be applied to the robot's actuated joints. The policy is queried at a rate of 33 Hz, whereas the PD controllers operate at a higher frequency of 2 kHz.

The policy's input at each timestep t consists of four components: given command $\mathbf{c}_t$, reference motion $\mathbf{q}_t^r$, the robot's short I/O history $\langle \mathbf{o}_{t:t-4}, \mathbf{a}_{t-1:t-4} \rangle$, and the robot's long I/O history $\langle \mathbf{o}_{t:t-65}, \mathbf{a}_{t-1:t-66} \rangle$. The time-varying command $\mathbf{c}_t$, as defined in Sec. 4.2.3 represents the task the robot is to accomplish using the desired locomotion skill. The locomotion skill is specified by a *preview* of a skill-specific reference motion $\mathbf{q}_t^r$ at the current timestep t for the robot. It includes the incoming desired motor positions for the robot $\mathbf{q}_t^r = [\mathbf{q}_m^d(t+1), \mathbf{q}_m^d(t+4), \mathbf{q}_m^d(t+7)]$ which is sampled at timesteps that are 1, 4, and 7 ahead. The preview of the reference motion conveys the upcoming desired trajectory to the robot, aiding it in avoiding being shortsighted. If the desired base height is not included in command $\mathbf{c}_t$, such as for running and jumping, the current base height from the reference motion $q_z^r(t)$ is then also included in the $\mathbf{q}_t^r$.

5.2. The use of Robot's I/O history

In addition to the observations described above, the observations also include a history of the robot's inputs and outputs (I/O) to the control policy. As depicted in Figure 3, the *inputs* are represented by the desired motor positions (the robot's actions $\mathbf{a}$), while the *outputs* are represented by the observable states of the robot, denoted as $\mathbf{o}$ and described in Sec. 4.1.1. This history is processed through two streams, which we refer to as a dual-history approach. The first stream offers a brief, four-timestep history of the robot's I/O, which is directly provided as input into the base network. This short history, lasting approximately 0.1 seconds, provides the robot with recent feedback for real-time control. In addition to the short history, a longer I/O history, spanning 2 seconds, is also provided as input to the policy. This long history comprises of 66 pairs of robot I/O data ($\langle \mathbf{o}_{t-k}, \mathbf{a}_{t-k-1} \rangle$, $k \in [0, 65] \cap \mathbb{Z}$). This long I/O history contributes more to the identification of the system's dynamics, encompassing elements like the LPF, PD controllers, the robot itself governed by (1), and its state estimator. Such a structure allows the controller to effectively leverage information from both short-term and long-term I/O history. It is important to leverage both of these information. The long-term history is useful for system identification and inferring state estimates, especially for ballistic movements involved during the flight phase. The short-term history is also important from two perspectives: it can provide *explicit* feedback of recent measurements on the robot, which is critical for real-time control, and it can help the robot to determine the weight of the past information which sometimes may not be important. As we will show in Sec. 7, this dual-history structure leads to significant performance improvements for bipedal locomotion control.

5.3. Details of policy representation

As depicted in Figure 3, the policy architecture π_θ consists of two main components: a base network, modeled by a multilayer perceptron (MLP), and a long-term history encoder modeled by a 1D convolutional neural network (CNN), which computes an embedding of the long history that is provided as input to the base network. The base MLP has two hidden layers with 512 *tanh* units each. The 1D CNN encoder consists of two hidden layers. Their configurations, defined as [kernel size, filter size, stride size], are [6, 32, 3] and [4, 16, 2] with *relu* activation and no padding, respectively. The 66-timestep long I/O history is encoded via this CNN encoder through temporal convolutions along the time axis, and then compressed into a latent representation before being provided as input to the base MLP. The output layer of the base MLP consists of *tanh* units that specify the mean of the Gaussian distribution of the normalized action (w.r.t. the motor range). The standard deviation of the action distribution is specified by a fixed value 0.1I.

5.4. General policy structure for different skills

The control policy structure introduced in Figure 3 is a general design and can be widely applied to a large variety of locomotion skills, such as standing, walking, running, and jumping. To train policies for different locomotion skills, a user needs only to simply provide different reference motions and commands to the policy, while the underlying architecture of the policy remains unchanged. Throughout this paper, the same control policy architecture will be used for all experiments.

6. Multi-stage training for versatile locomotion controllers

Following the construction of the bipedal locomotion controller, our next step in this section involves developing a general framework for training the control policy through reinforcement learning. Much like the control structure, this training framework extends beyond a single, specific locomotion skill and is general to various skills. Such a framework is designed to train the robot in simulation and to transfer to the robot hardware without further fine-tuning.

6.1. Overview

It is challenging to train a robot to accomplish diverse tasks through a single control policy via RL. This challenge is further compounded when dealing with highly dynamic locomotion maneuvers where the robot has limited support, such as varying running speeds or jumping to different locations. In such scenarios, the robot often struggles to learn effectively and may adopt overly conservative strategies, such as merely standing, to circumvent the challenges.

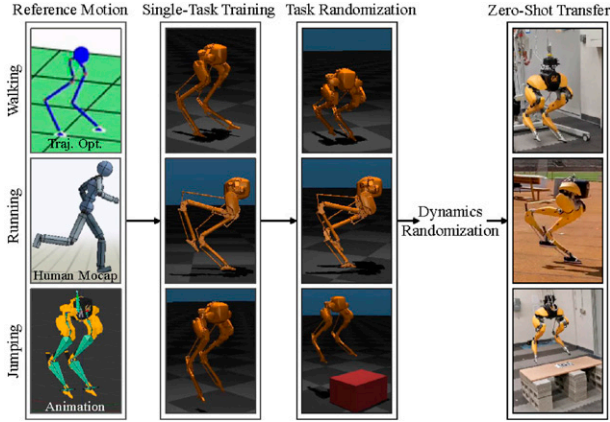


Figure 4. The multi-stage training framework to obtain a versatile control policy that can be zero-shot transferred to the real world. It starts with single-task training stage, where the robot is encouraged to mimic a single reference motion with a fixed goal. This is followed by task randomization stage, which expands the range of tasks the robot learns and fosters task generalization resulting in a versatile policy. Once the robot is adept at various locomotion tasks and their transitions, extensive dynamics randomization is incorporated to enhance policy robustness for sim-to-real transfer. This framework is suitable for diverse bipedal locomotion skills, including walking, running, and jumping, and for learning from different sources of skill-specific reference motions such as trajectory optimization, human mocap, and animation.

Therefore, we develop a multi-stage training strategy that incorporates a structured curriculum to facilitate the training of versatile locomotion control policies. This strategy can be summarized into three stages as shown in Figure 4: (1) single-task training, (2) task randomization, and (3) dynamics randomization. In the first stage, we concentrate on training the robot to acquire a locomotion skill from scratch, employing a fixed command. The primary aim is to equip the robot with the ability to master the skill itself, such as just walking forward, running forward, or jumping in place, while avoiding undesired maneuvering strategies. In the second stage, we introduce diverse commands to encourage the robot to perform a large variety of tasks using the skill it has acquired to develop a versatile policy. Following the robot's proficiency in a simple simulation environment, the third stage implements extensive randomization of dynamics parameters in the simulation. This process is designed to robustify the policy to ensure a successful zero-shot transfer from simulation to robot hardware.

The design of POMDP, including aspects like reward and episode design, may indeed differ across different stages to serve specific objectives. However, the overall multi-stage training scheme remains a general approach to developing different locomotion skills, and as we will see, this method requires only the change of skill-specific reference motion and hyperparameters for learning different skills.

6.1.1. Combining a standing skill. Learning a standing skill for the bipedal robot is useful for deployment in the real world,

where the robot may need to come to a stop after walking, running, or jumping. In the context of the aperiodic jumping skill, the robot learns to maintain a stance pose during the post-landing phase alongside the jumping skill. However, this standing skill is not introduced for periodic skills like walking and running. To address this gap, in Stage 2, we introduce an additional sub-stage to enable the robot to learn the transition to standing (and back) once it has acquired a versatile policy, with details introduced in Appendix 2. While previous approaches have explored using separate policies for such a transition, as demonstrated in Crowley et al. (2023), our work showcases the advantages of having a single policy for transitioning between standing and other locomotion skills. This approach can not only realize a rapid transition but also enable the robot to generalize the learned locomotion skill to significantly improve robustness during standing.

6.2. Reference motion

For each locomotion skill, we provide one or a set of reference motions, which provide examples of the type of locomotion maneuvers that the robot is to perform. As shown in Figure 4, our framework can accommodate diverse sources of reference motion, including reference motions from trajectory optimization, motion capture, and keyframe animations.

6.2.1. Trajectory optimization. For the walking skill, we leverage the trajectory optimization method to generate a library of reference motions that depict diverse periodic walking gaits based on the robot's full-order dynamics. The resulting *gait library* is parameterized by the walking commands $[\dot{q}_x^d, \dot{q}_y^d, \dot{q}_z^d]$ ranging from $[-1.0, -0.3, 0.65]$ to $[1.0, 0.3, 1.0]$, and consists of 1331 different reference motions. One reference motion is represented by a set of Bézier trajectories of each actuated motor with a fixed timespan of the walking period (0.8 seconds). A more detailed description of the process for generating the gait library is provided in (Li et al., 2020, Sec. III-B) and (Hereid et al., 2019). Note that we do not consider turning yaw command q_ψ^d when building the reference gait library.

6.2.2. Motion capture. The reference motion for the running skill is derived from motion capture data collected from a human actor (SFU, 2018). We retargeted the original human motion to the Cassie's morphology using inverse kinematics. This process involves matching the motion of key points, such as the pelvis and toes, between the human and the robot. More details on the retargeting process is available in (Li et al., 2020, Sec. II-B). Note that we only have *one single* reference motion for periodic running, with an average speed of 3 m/s and foot clearance of around 0.1 m during flight, without lateral or turning movements. We also retarget *one single* human motion of the transition from running to standing to serve as the reference motion for Cassie to transition between the two behaviors.

6.2.3. Animation. We provide the reference motion for the *jumping* skill using the animation technique. The robot jumping motion is directly hand-crafted in a 3D animation creation suite. Similar to running, we only provide a *single* jumping-in-place animation for reference motion, with an apex foot height of 0.5 m, a jumping timespan $T_J = 1.66$ seconds, and ending with a stance pose. For readers interested in creating animation for bipedal robots, please refer to Li et al. (2020).

Please note that we do not perform trajectory optimization to translate the kinematically feasible reference motion from motion capture or animation to be dynamically feasible for the robot.

6.3. Reward

We now formulate the reward function r_t that the agent receives at each timestep t in order to encourage the robot to perform the desired locomotion skills while completing the desired tasks. In this work, the reward r_t the agent receives is the weighted summation of several reward components $\mathbf{r}$, i.e., $r_t = (\mathbf{w}/\|\mathbf{w}\|_1)^T \mathbf{r}$ with the component vector $\mathbf{r}$ and weight vector $\mathbf{w}$ listed in Table 3. Each element in $\mathbf{r}$ shares the same format as:

$$r(\mathbf{u}, \mathbf{v}) = \exp(-\alpha \|\mathbf{u} - \mathbf{v}\|_2). \quad (2)$$

By maximizing (2), the robot is incentivized to minimize the distance between two vectors, $\mathbf{u}$ and $\mathbf{v}$. In (2), a different scaling factor $\alpha > 0$ is introduced in each term to normalize units, resulting in an output range of (0, 1].

6.3.1. Reward components. The reward r_t comprises three key terms: (1) motion tracking, (2) task completion, and (3) smoothing. Each of these terms consists of several individual reward components, each serving specific objectives, as detailed in Table 3.

6.3.1.1. Motion tracking. The *motion tracking* term is crafted to incentivize the agent to follow the provided reference motion for a specific skill. This objective is achieved through several components, including the motor position reward $r(\mathbf{q}_m, \mathbf{q}_m^r(t))$, the global pelvis height $r(q_z, q_z^r(t) + \delta_z)$, and the global foot height $r(\mathbf{e}_z, \mathbf{e}_z^r(t) + \delta_z)$ at each time step. The addition of δ_z in the reference vertical displacement takes into account variations in terrain height, such as when the robot encounters changing terrain while running (δ_z : = the time-varying terrain height) or when it jumps to different elevations (δ_z : = the time-invariant target elevated height). There is some privileged environment information used in the reward, such as the robot's global height q_z , foot height $\mathbf{e}_z$, or terrain height δ_z . These terms provide the robot with better knowledge of the environment

Table 3. The components $\mathbf{r}$ and their respective weights $\mathbf{w}$ in the reward function used for training the bipedal robot across various skills and training stages. A nominal weight vector is provided for different components, with adjustments necessary based on the skill and stage of training, primarily influenced by task diversity (training for single goal versus diverse goals) and the existence of a flight phase. The reward tuning process is relatively streamlined, as only 24 out of 72 reward terms vary across the three different bipedal locomotion skills over multi-stage training. Blank cells indicates no change on the corresponding nominal value.

Reward component $\mathbf{r}$	Weight $\mathbf{w}$						
	Nominal value	Walking skill		Running skill		Jumping skill	
		Stage 1	Stage 2, 3	Stage 1	Stage 2, 3	Stage 1	Stage 2, 3
Motion tracking							
Motion position: $r(\mathbf{q}_m, \mathbf{q}_m^r(t))$	15						−7.5
Pelvis height: $r(q_z, q_z^r(t) + \delta_z)$	5						−2
Foot height: $r(\mathbf{e}_z, \mathbf{e}_z^r(t) + \delta_z)$	10	−7	−7				
Task completion							
Pelvis position: $r(q_{x,y}, q_{x,y}^d)$	7.5	−1.5	−1.5			+5.5	+7.5
Pelvis velocity: $r(\dot{q}_{x,y}, \dot{q}_{x,y}^d)$	15					−15	−2.5
Pelvis orientation: $r(\cos(q_{\phi, \theta, \psi}, [0, 0, q_{\psi}^d]), 1)$	10	−2.5	+2.5	−5		+2.5	
Pelvis angular rate: $r(\dot{q}_{\phi, \theta, \psi}, [0, 0, \dot{q}_{\psi}^d])$	3				+4.5		+7
Smoothing							
Foot impact: $r(F_z, 0)$	10	−7				−5	
Torque: $r(\boldsymbol{\tau}, 0)$	3						
Motor velocity: $r(\dot{\mathbf{q}}_m, 0)$	0				+3		
Joint acceleration: $r(\ddot{\mathbf{q}}, 0)$	3						−3
Change of action: $r(\mathbf{a}_t, \mathbf{a}_{t+1})$	3			+2	+2	−3	+7

and current states during training, but are excluded from the actor's observation. We will frequently see such an exploitation of privileged information in the reward.

6.3.1.2. Task completion. We incorporate a *task completion* term into the reward to ensure that the robot accomplishes the assigned tasks using the acquired locomotion skill. Within this term, we motivate the robot to align its movement with the desired velocity and turning rate by including $r(\dot{q}_{x,y}, \dot{q}_{x,y}^d)$ and $r(\dot{q}_{\phi,\theta,\psi}, [0, 0, \dot{q}_{\psi}^d])$, respectively. Additionally, we introduce global pose tracking components $r(q_{x,y}, q_{x,y}^d)$ and $r(\cos(q_{\phi,\theta,\psi} - [0, 0, q_{\psi}^d]), 1)$ in the reward. Note that for the orientation tracking term, we opt to match 1 with the cos of the orientation error within the range of $[-\pi, \pi]$ to prevent singularities when the robot undergoes a transition between $-\pi$ and π . Furthermore, as this work does not consider changes in robot roll and pitch ($q_{\phi,\theta}$), we set $q_{\phi,\theta}^d$ and its rate of change $\dot{q}_{\phi,\theta}^d$ to zero, contributing to the stabilization of the robot's pelvis. For periodic walking and running skills, the desired velocities $\dot{q}_{x,y,\psi}^d$ are initially provided, and the position terms $q_{x,y,\psi}$ are calculated by integrating the corresponding commands over time. Conversely, for aperiodic jumping skills, the desired (time-invariant) landing targets $q_{x,y,\psi}$ are specified first, and average velocity terms are introduced to shape the sparse position reward as $\dot{q}_{x,y,\psi}^d = q_{x,y,\psi}^d / T_J$ where T_J is the jumping timespan.

6.3.1.3. Smoothing. Finally, we also add a *smoothing* term to discourage the robot from learning jerky behaviors. We incentivize the robot to reduce impact forces through $r(F_z, 0)$, reduce energy consumption via $r(\tau, 0)$, produce smooth motions by minimizing motor velocities $r(\dot{\mathbf{q}}_m, 0)$, damp out joint acceleration $r(\ddot{\mathbf{q}}, 0)$, and regulate changes in the actions $r(\mathbf{a}_t, \mathbf{a}_{t+1})$.

6.3.2. Reward weights. By employing different choices of weights $\mathbf{w}$, we can emphasize certain reward terms as more critical for the robot's performance while diminishing the significance of others, thereby influencing the robot's acquired maneuvers. In this work, despite covering different dynamic locomotion skills and including several training stages for each skill, we demonstrate that the reward components and their associated weight values align with a unified choice (i.e., the nominal value in Table 3) applicable to different skills and stages. This homogenization can be achieved with some adjustments in accordance with general principles governing the tuning of specific weight, as detailed in Table 3 and introduced below.

6.3.2.1. Weights across different stages. In Stage 1, which involves the initial training of a specific locomotion skill from scratch, our emphasis is placed on training the robot to master the desired skill. As a result, the weight of the *motion tracking* term takes precedence in the reward, encouraging the robot to closely mimic the reference motion. As the robot becomes proficient in the desired skill,

upon entering Stage 2, where the commands are randomized, we can adjust the weight of the *task completion* term to outweigh other terms. This adjustment serves to motivate the robot to accomplish different tasks, especially the ones beyond the reference motion, such as executing turning motions in addition to walking, various running and turning speeds that extend beyond the single running reference motion, and various landing targets beyond the single jumping-in-place animation. To prevent the robot from adopting overly conservative behavior, especially during initial training, the weight of the *smoothing* term remains relatively low. As the robot solidifies its locomotion skill, this term can be gradually increased to refine the robot's movements. But this term is always the least across multiple stages.

6.3.2.2. Weights across different skills. As detailed in Table 3, the variations in the weights among different locomotion skills are generally not substantial, with the primary distinctions arising from the presence of the flight phase in the skill. Notably, in skills such as running and jumping, where a significant flight phase is involved, the *motion tracking* term for foot height $r(\mathbf{e}_z, \mathbf{e}_z^r(t) + \delta_z)$ carries a larger weight compared to the walking skill. Furthermore, to encourage the robot to tackle the challenges caused by the flight phase while deviating from the reference motion to explore diverse tasks, we assign higher weights to the *task completion* term. The *smoothing* term largely remains consistent across skills, with some adjustments like the change of action term $r(\mathbf{a}_t, \mathbf{a}_{t+1})$ to strengthen the smoothing of aggressive movements for running and jumping.

6.4. Episode design

6.4.1. Unified approach. Across all of the diverse locomotion skills and training stages developed in this study, the episode design is consistent and unified. The episode duration is set to 2500 timesteps, corresponding to a total timespan of 76 seconds. In Stage 2, where variable tasks are introduced, we randomize the command after random time intervals, ranging from 1 second to 15 seconds. The specific ranges for uniformly randomized commands for each task are detailed in Table 6 in Appendix 3. Note that we employ a long horizon in an episode to enable the robot to explore more scenarios of transitions among different tasks.

One exception is made for the episode length in the first stage of aperiodic tasks like jumping. For such cases, the episode length is adjusted to cover a complete trajectory (e.g., 1.66 seconds for jumping motion), with a significant extension to enable the robot to learn to maintain the last standing pose after landing. In this work, the Stage 1 training of jumping employs 750 timesteps (22 seconds).

6.4.2. Early termination conditions. When training dynamic locomotion skills on bipedal robots, relying solely on reward design could be insufficient. The robot might still exhibit undesirable behavior despite achieving suboptimal

return, such as maintaining a stance pose without engaging in locomotion because the robot can easily attain part of the reward in this way. To address this, in addition to the standard termination conditions, such as the robot falling over ($q_z < 0.55\text{m}$) or the tarsus joints $q_6^{L/R}$ hitting the ground, we incorporate two additional termination conditions among all skills and stages, as elaborated below.

6.4.2.1. Foot height tracking tolerance. The episode will terminate early if the deviation between the robot's foot height and its reference motion exceeds a threshold E_e , i.e., if $|\mathbf{e}_z - \mathbf{e}_z^r(t) - \delta_z| > E_e$. This condition is empirically found to be particularly effective in promoting the development of locomotion skills involving flight phases, like running and jumping. Without this condition for *motion tracking*, especially during the initial stages of training, the robot tends to remain mostly on the ground. For skills lacking a flight phase, such as walking, this condition can still be incorporated, although it may not be necessary.

6.4.2.2. Task completion tolerance. We underscore the importance of *task completion* to the robot by introducing a condition based on the robot's deviation from the commanded base position and orientation, $|q_{x,y,\psi} - q_{x,y,\psi}^d| > E_t$, with E_t being the acceptable tracking error threshold. For walking and running, we consistently check this condition to encourage the robot to diminish accumulated tracking errors. In the case of jumping, this condition is evaluated after the robot's landing, stimulating the robot to jump to the specified target.

6.4.2.3. Tolerance across different stages. The error thresholds E_e and E_t may require adjustments as the training progresses, depending on the robot's proficiency in the desired locomotion skill. For instance, the foot height tracking error threshold E_e may start with a tight constraint in the initial stage (Stage 1) and progressively relax in subsequent stages. Ideally, this condition can be completely removed in the end. This strategy provides the robot more flexibility in exploring diverse maneuvers in the later stages, surpassing the capabilities of the provided reference motion. In contrast, the tracking error threshold E_t can be gradually reduced during training as the robot becomes more skilled in locomotion and can place a stronger emphasis on task completion. Despite the diversity of skills, we observe a consistent trend in such tolerance adjustments across the multiple training stages.

6.5. Dynamics randomization

As illustrated in Figure 4, in training Stage 3, we introduce randomized dynamics parameters in the simulation environment to train a policy that can stay robust and generalize the acquired locomotion skills to deal with the uncertainty in both *dynamics modeling* and *measurements*. The objective of this training is to enable successful transfer from simulation to real-world scenarios where the dynamics

parameters are uncertain. At each episode, dynamics parameters, as detailed in Table 4, are sampled from their respective uniform distributions.

Specifically, to address *modeling uncertainty*, we introduce extensive randomization of modeling parameters, including ground friction coefficient, robot joint damping ratio, and mass, inertia, and Center-of-Mass (CoM) position of each link. In the case of Cassie, which features passive joints connected by leaf springs, the introduction of $\pm 20\%$ randomized stiffness for the leaf springs has been important for successful sim-to-real transfer, particularly for skills like running and jumping that involve substantial compression of the springs. We also incorporate randomization into the PD gains of the joint-level PD controllers, adding a range of $\pm 30\%$ deviation from their default values. This randomization is applied independently to each joint-level PD controller. This stimulates the diversity in motor responses that can mimic the effects of uncertain motor dynamics seen in real-world scenarios, including motor aging and degradation, which is found effective in facilitating the sim-to-real transfer.

To tackle *measurement uncertainty*, we add a simulated noise to the observable states $\mathbf{o}_t$. The noise is modeled as a normal distribution with the mean uniformly sampled from the range specified in Table 4. Please note that, given the reliable sensors like joint encoders and IMU on Cassie's hardware, the range of measurement noise we applied is

Table 4. The range of dynamics randomization. Introducing simulated external perturbations to the robot's base or incorporating variable terrain is optional but only recommended after the robot has learned to handle general dynamics randomization. However, for highly dynamic skills like aperiodic jumping, external perturbations may not enhance robustness and could stop the robot from learning meaningful maneuvers.

Parameters	Range
Dynamics randomization (general)	
Ground friction coefficient	[0.3, 3.0]
Joint damping ratio	[0.3, 4.0] Nms/rad
Spring stiffness	[0.8, 1.2] $\times$ default
Link Mass	[0.5, 1.5] $\times$ default
Link inertia	[0.7, 1.3] $\times$ default
Pelvis (root) CoM position	[-0.1, 0.1] m in $q_{x,y,z}$
Other link CoM position	[-0.05, 0.05] m+default
Motor PD gains	[0.7, 1.3] $\times$ default
Motor position noise Mean	[-0.002, 0.002] rad
Motor velocity noise Mean	[-0.01, 0.01] rad/s
Gyro rotation noise	[-0.002, 0.002] rad
Linear velocity estimation error	[-0.04, 0.04] m/s
Communication delay	[0, 0.025] s
External perturbation (optional)	
Force & torque	[-20, 20] N & [-5, 5] Nm
Elapsed time interval (walking)	[0.1, 3.0] s
Elapsed time interval (running)	[0.1, 1.0] s
Randomized terrain (optional)	
Terrain type	Waved, slopes, stairs, steps

small. However, for robots equipped with lower-quality sensors, a wider simulated noise range may be necessary. Additionally, we simulate a communication delay, caused by a zero-order hold, between the policy and the robot's real-time computer, which influences the timing of sending actions and receiving observations. For highly dynamic motions, such as jumping and running, the delay plays a significant role in the sim-to-real gap, and the ability to tackle delays during real-time control is critical for stability.

The randomization strategy mentioned above is applied consistently across various locomotion skills. Additionally, we have also investigated the use of other sources of randomization during training. They are randomized perturbation and randomized terrains as detailed below.

6.5.1. Use of randomized perturbation. In our study, we explored the use of randomized external perturbation wrenches applied to the robot's pelvis, hypothesizing that it could create more diverse training scenarios and enhance robustness by deviating the robot from its nominal trajectories. This perturbation (typically being impulse), incorporating forces and torques applied for random durations as specified in Table 4, can be simulated during training. However, we find that while this approach can increase the robustness of policies in real-world applications, it introduces complex hyperparameters that are challenging to choose and complicates the training process. For walking, frequent perturbations led to an overemphasis on perturbed scenarios, impairing the robot's performance in normal walking gaits. In running, longer perturbations hindered the robot's ability to learn effective gaits. As a result, we employed different elapsed time intervals for walking and running, as detailed in Table 4. Moreover, in the case of jumping skills or transitions from locomotion to standing, these perturbations proved even more problematic, preventing the robot from acquiring meaningful jumping or standing abilities. Therefore, we chose to exclude external perturbation from the training for both jumping and transition-to-standing skills.

6.5.2. Use of randomized terrain. If the control policy aims to enable the robot to traverse uneven terrain, it is essential to simulate terrain changes during training. We developed an algorithm to randomize various types of terrains using parameterized height maps including wave terrain (characterized by sine functions), sloped terrain, monotonic stairs, and random steps, as listed in Table 4. It's important to note that our control policy doesn't incorporate vision, so the robot has to adapt to terrain changes based on its I/O history, making it a more challenging problem. Terrain randomization is recommended only after the robot has proficiently trained in other dynamics randomization to avoid excessive learning complexity. In this study, we introduced terrain randomization for running skills for example.

6.6. Training details

We develop the simulation of Cassie within the aforementioned environments using MuJoCo based on (DRL, 2023; Todorov et al., 2012). The training of all control policies is carried out using Proximal Policy Optimization (PPO) developed by (Schulman et al., 2017) in simulation. The control policy (actor) is as described in Sec. 5, with a value function represented by a 2-layered MLP that has access to ground truth observations. Given the varying complexities of different training stages and skills, the number of training iterations differs across stages and skills. The numbers of iterations and other hyperparameters are provided in Table 7 and Table 8, respectively, in Appendix 4.

Up to this point, design details of the proposed RL framework for learning versatile, robust, and dynamic bipedal locomotion skills have been introduced. We will next proceed to validate several key design components.

7. Advantages of policy architecture

In this section, we first evaluate the advantages of the proposed bipedal locomotion control framework in Sec. 5 through an extensive ablation study of its design choices. This evaluation unfolds from two key perspectives: (1) the learning performance in challenging simulation environments with extensively randomized dynamic parameters, and (2) the control performance during the sim-to-real transfer on a real robot, without any tuning. The simulation offers a controlled setting to test the capacity of the control architectures in learning control strategies for large changes in the system dynamics, while real-world experiments allow us to evaluate the control efficacy on the fixed but uncertain dynamics of the robot's hardware. As we will demonstrate, our proposed dual-history control architecture and training system result exhibits the best learning performance and sim-to-real transfer.

7.1. Baselines

We compare the proposed controller structure and other baselines listed below and illustrated in Figure 5. Those choices of baselines include ablations on several design choices proposed in this work: (1) choices of action space, (2) history of the robot's I/O or state feedback only, (3) history length (long or short history), (4) dual-history or long history only, (5) end-to-end training or policy distillation. The models used in the experiment include:

- **Ours (Figure 5(a)):** as detailed in Figure 3, our architecture leverages both short-term and long-term I/O history, with the long-term history encoded by a CNN, while the short-term history is directly fed into the base MLP. The CNN and MLP are trained jointly and the policy outputs specify the desired motor positions.
- **Residual (Figure 5(b)):** this architecture aligns with our proposed one, but it produces an output that represents a

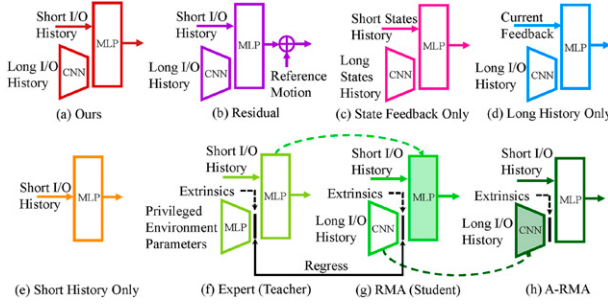


Figure 5. Illustration of our proposed and various baselines for RL-based control policy architectures for bipedal robot locomotion. A, **Ours** integrates both short and long-term I/O histories, with the base MLP and long history encoder jointly trained to specify motor positions. B, the **Residual** approach aligns with our architecture but adds a residual term to the reference motor position. C, the **State Feedback Only** baseline uses our model structure but relies solely on robot’s states history, excluding input history. D, the **Long History Only** approach depends on long I/O history without using short I/O history, while the **Short History Only** approach (E) focuses only on short-term I/O history, excluding the CNN encoder. The **RMA/Teacher-Student** method utilizes a two-phase policy distillation, with an expert (teacher) policy (F) guiding the training of an RMA (student) policy (G), which can be improved by **A-RMA** (H) which introduces an additional phase where the base MLP is finetuned while keeping the long I/O history encoder’s parameters fixed. Notably, all expert, RMA, and A-RMA policies in this study incorporate short I/O histories into the base MLP, which is a new modification in this work to enable equitable comparison with ours. All of these architectures have the command and reference motion as input to the base MLP, as detailed in Figure 3 and omitted for brevity.

residual term that is then added to the reference motor position at the current timestep, i.e., $q_m^d = \mathbf{a}_t + q_m^r(t)$. Such an approach is employed in prior works like Lee et al. (2020); Xie et al. (2020); Siekmann et al. (2020). It is worth noting that the robot “knows” the current reference motor position to add by taking the reference motion as input.

- **State Feedback Only** (Figure 5(c)): this variant retains the same model structure and action space as ours. However, it differs in its observation by relying solely on the historical states (robot’s output history), omitting the robot’s input history. Such a choice, *without* the use of short history, is more commonly seen in previous work, such as Lee et al. (2020); Siekmann et al. (2020, 2021a); Crowley et al. (2023).
- **Long History Only** (Figure 5(d)): this policy architecture relies only on a long-term I/O history encoded by the CNN. This configuration serves as a baseline in Kumar et al. (2021). As suggested by Peng et al. (2018), the base MLP has direct access to the robot’s immediate (last-timestep) state feedback. For brevity, in this work, *Long History Only* refers to the method that provides the latest observation (state feedback) along with the long history encoder, which can be modeled using different neural network architectures.

- **Short History Only** (Figure 5(e)): this policy relies solely on short-term I/O history, excluding the long-term I/O history CNN encoder. This is used for bipedal locomotion control in Li et al. (2021) and is more commonly seen in quadruped control like Huang et al. (2023); Escontrela et al. (2022); Feng et al. (2023).
- **RMA/Teacher-Student**: this architecture utilizes the policy distillation method that involves two training phases. First, an expert (teacher) policy (Figure 5(f)) is trained by RL that has access to privileged environment information (listed in Table 4). The privileged information is encoded into an 8D extrinsics vector by an MLP encoder. *This expert (or teacher) policy can only be used in simulation.* Second, the expert policy is employed to *supervise* the training of an RMA (student) policy (Figure 5(g)). The RMA policy copies the base MLP from the expert policy and only learns to leverage the long I/O history encoder to estimate the teacher’s extrinsic vector. Such a policy distillation method is used in Lee et al. (2020); Kumar et al. (2021) and widely adopted in quadrupedal locomotion control.
- **A-RMA** (Figure 5(h)): after the RMA training, an additional training phase is introduced. In this phase, the long I/O history encoder’s parameters remain fixed, while the base MLP is updated again through RL, as introduced by Kumar et al. (2022). *Notably, during the implementation, all expert, RMA, and A-RMA policies incorporate short I/O histories to the base MLP in this study, which is a new modification in this work to enable an equitable comparison with ours.*

For each locomotion skill studied in this work (walking, running, and jumping), using our proposed method and baseline methods, we obtained three policies trained by the identical multi-stage training framework introduced in Sec. 6. These policies were obtained from the same choice of hyperparameters, but from different random seeds. This was done for each of these policy architectures. In other words, $3 \times 3 \times 8 = 72$ different control policies are obtained, representing three locomotion skills, three random seeds, eight policy architectures, and evaluated below.

7.2. Learning performance

We begin our evaluation by benchmarking our method against various baselines in simulation by examining their respective learning curves. Our focus lies on the training performance during Stage 3, as shown in Figure 6, the most challenging stage that involves both randomized tasks and dynamics parameters. This particular emphasis on Stage 3 also stems from its critical role in sim-to-real transfer, a pivotal concern within the scope of controlling real bipedal robots in our study.

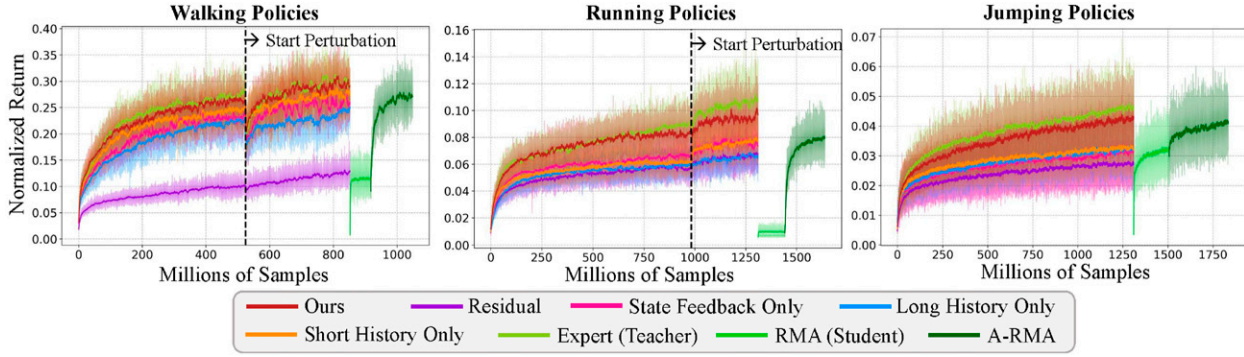


Figure 6. The learning performance using different policy structure designs illustrated in Figure 5. It is assessed during Stage 3 training, which incorporates both task and dynamics randomization. These curves represent the average normalized episodic return across three distinct policy trainings from different random seeds, with shaded regions indicating the range between minimum and maximum returns. Note that there is no perturbation training for the jumping policy as it prevents the robot from learning the dynamic jumping skill. Our proposed method consistently outperformed other baselines across different skills. Notably, our policy’s performance is comparable to that of the expert policy, which has access to privileged information but is not deployable in the real world. In contrast, the residual method shows the worst return. Using only a long history does not offer a clear benefit compared to a policy relying solely on a short history. In fact, the short history only policy even outperforms the long history only approach. Additionally, even with a dual-history approach like ours, omitting the robot’s input history (only using state feedback) results in no improvement over short I/O history only. The student or RMA methods exhibit significant regression loss in bipedal locomotion control; particularly in dynamic skills like running, RMA fails to learn. This suggests the necessity of the A-RMA stage for further training, though it requires considerably more training samples and yields slightly lower returns compared to our method.

7.2.1. Benchmark analysis. As depicted in Figure 6, the learning performance remains uniform across various locomotion skills, as outlined and analyzed below.

7.2.1.1. Choices of action. We observe that when the policy produces a residual term (purple curves) instead of directly specifying the desired motor position, the learning performance consistently deteriorates across various locomotion skills. Although the added reference motion to the action might accelerate the learning of the desired skill initially, this added reference motion could also introduce additional movements to the robot. Consequently, the policy expends more effort correcting the added movements rather than effectively controlling the robot, which becomes more problematic when the robot explores maneuvers beyond the reference motion. Hence, we recommend readers reconsider the use of residual learning in the context of locomotion control, regardless of whether the added action is optimized for dynamic feasibility (as in walking) or limited to kinematic feasibility (as in running or jumping).

We also consider another action space option: joint-level torque. RL has been used to learn torque control for quadrupedal and bipedal robots as demonstrated in Chen et al. (2023) and Kim et al. (2023) respectively. However, torque control’s high update frequency requirements, such as 250 Hz in Kim et al. (2023), restrict its ability to utilize extensive robot I/O history. Prior methods using torque control have thus been limited to a single timestep of robot feedback. Later, we will demonstrate that a longer I/O history, like 2 seconds, could improve the adaptivity of RL-based controllers, but recording and learning with such a long history is computationally expensive for high-frequency torque controllers, as the action (robot’s input)

history needs to update at the same frequency as the controller. Therefore, our benchmark focuses on policies that can operate effectively at lower frequencies.

7.2.1.2. Choices of observation. A comparison between our proposed method (red curves), which considers the robot’s I/O history, and a baseline that relies solely on the robot’s output history (pink curves) reveals an important finding: omitting the robot’s input (action) history leads to a decline in learning performance, even with an identical policy architecture and training approach. In other words, providing only the history of the robot’s state feedback in the observation is insufficient. This comparison illustrates the critical role of utilizing both the robot’s input and output history in developing a control policy through model-free RL. This combined I/O history is crucial for the control policy to perform system identification and state estimation to infer the robot’s system parameters and states, which enhances its adaptivity to uncertain dynamics and external perturbations. This importance becomes particularly pronounced in our work, where we tackle the control of a highly nonlinear high-dimensional system (bipedal robot) using limited and noisy, delayed measurements (partial observability).

7.2.1.3. Long history versus short history. When leveraging long I/O history alone (blue curves), the learning performance fails to surpass that of the policy utilizing only short I/O history (orange curves) or others. However, when we provide the base MLP with direct access to the short I/O history while having a long history encoder, as proposed in our approach, the resulting learning performance exhibits significant improvement. This enhancement is rooted in the

real-time control context, where the short history becomes crucial for information about the robot's very recent I/O trajectory. While the long history does contain this short-term information, it can become obscured after passing through an encoder. Hence, providing the policy with direct access to *explicit* short history within the base MLP complements the utilization of long history. Further discussion is presented in Sec. 11.1. Additionally, in [Appendix 5](#), we delve into the effects of varying history lengths for temporal encoding. We found that increasing the history length can enhance training performance, but improvements tend to plateau with longer histories and may even hinder training by introducing redundant information for locomotion control. The 2-second history tends to perform consistently well in all the skills developed in this work.

Remark 1. In [Appendix 6](#), we explore the effects of training performance using different temporal encoders such as TCN ([Lee et al., 2020](#)) and LSTM ([Siekmann et al., 2020](#)). We found that a dual-history approach consistently enhances learning performance for non-recurrent policies like TCN that encode an explicit history length. However, for recurrent policies like LSTM, the dual-history approach does not significantly aid learning. Additionally, recurrent policies tend to converge to more suboptimal policies than non-recurrent ones and are sensitive to hyperparameter tuning across different MDPs (locomotion skills).

It's important to clarify that our aim is not to dispute the use of a specific temporal encoding structure or the use of a long history. Instead, our emphasis lies in underlining the importance of incorporating short history when working with long history data, which can be encoded by many choices of structures.

7.2.1.4. Comparison with policy distillation methods. A comparison between our method that jointly trained the long history encoder with the base MLP (red curves) and policy distillation methods that separate the training of the base MLP and history encoder (green curves) highlights the significance of using a right training strategy. We observed that the RMA (student) policy exhibits significant degradation compared to the expert policy, primarily due to the unavoidable error when utilizing the robot's long history to estimate pre-selected environment parameters (encoded to the extrinsics vector). This decline in RMA's performance becomes particularly prominent when dealing with challenging locomotion skills, such as running, where it fails to learn. A-RMA, which continues fine-tuning the base MLP with the frozen estimation encoder, can enhance RMA performance but it still falls slightly short when compared to our proposed method, despite having significantly more training samples. In the case of running, where the encoder struggles to estimate environment parameters, A-RMA essentially reproduces the result of the short history-only policy (the orange one), avoiding the use of long history. It's

worth noting that although there are other potential ways to enhance RMA learning like [Fu et al. \(2023\)](#), our proposed method consistently demonstrates performance similar to the expert policy which is the theoretical upper bound for the student policy. Importantly, our approach is deployable in the real world, whereas the expert policy is not.

7.3. Case study: In-place walking experiments

We proceed to conduct a case study on the policies trained using various methods, as depicted in [Figure 5](#), in the real world. We choose to assess the walking policies for controlling Cassie to sustain an in-place walking gait in the real world without any tuning or global position feedback. This relatively straightforward scenario can provide valuable insights into the adaptivity of the trained policies. If the policy is unable to adapt to the dynamics of the real robot, it may result in obvious drift, even if it effectively maintains the robot's walking gait.

As shown in [Figure 7](#), the results highlight a significant contrast. Our proposed method demonstrates notably lower tracking errors and successfully maintains the robot's in-place walking, with minimal drift observed in both sagittal and lateral directions. Conversely, policies utilizing long history only, short history only, and dual-history with only state feedback, when trained under the same number of samples, result in substantial drift to the robot's left. Furthermore, RMA demonstrates the most obvious sagittal shift and it walks forward using a fast speed even with a zero velocity command. While A-RMA reduces this sagittal drift, it still experiences considerable lateral movement. Moreover, the residual policy fails to maintain a stable gait on the real robot. The corresponding video is recorded in [Vid. 3](#) in [Table 1](#).

Since we obtained three distinct policies trained from different random seeds, we test each of these policies for the same in-place walking in both the simulation using the robot nominal model and the real world, and compare the statistical results in [Figure 8](#). The robot's speed ($\|\dot{q}_{x,y}\|_2$) and base orientation ($q_{\phi,\theta,\psi}$) tracking errors (Mean Absolute Error, MAE) over a 10-second test are evaluated. Note that, besides the turning yaw (q_ψ) that is included in the command, we also evaluate the robot's tracking error for the pelvis (base) roll q_ϕ and pitch q_θ angles. This is done to see which control policy can better stabilize the robot to a small base roll and pitch movement as they are supposed to be zeros. As recorded in [Figure 8\(a\)](#), all policies trained by different methods exhibit similar good tracking performance in terms of slow speed (except RMA) and floating-base orientation when commanded to walk in place in simulation. However, upon transferring to real hardware ([Figure 8\(b\)](#)), other methods exhibit significant speed drift and floating base rotational tracking errors. For example, the long history only policy shows the worst performance in stabilizing the robot's pelvis with a large floating base orientation, yet it has minimal base oscillation in simulation.



Figure 7. Snapshots from experiments of the bipedal robot Cassie controlled by different policies trained by different methods illustrated in Figure 5. The test is to control the robot to walk in place, i.e., $(\dot{q}_x^d, \dot{q}_y^d, \dot{q}_\psi^d) = \mathbf{0}$. Each snapshot compiles three frames taken at the same 1st, 4th, 7th second after initialization. Our method exhibits minor drift (the robot does not leave its initial place as demonstrated in the figure), outperforming other approaches, which all result in notable sagittal and/or lateral drifts (marked by the white arrows). Furthermore, the residual policy failed to control the robot to maintain a stable gait. This benchmark underscores the advantages of our proposed architecture in adapting to the real robot’s dynamics for better tracking performance after zero-shot transfer. These results are consistent over different policies trained from different random seeds, which are reported in Figure 8.

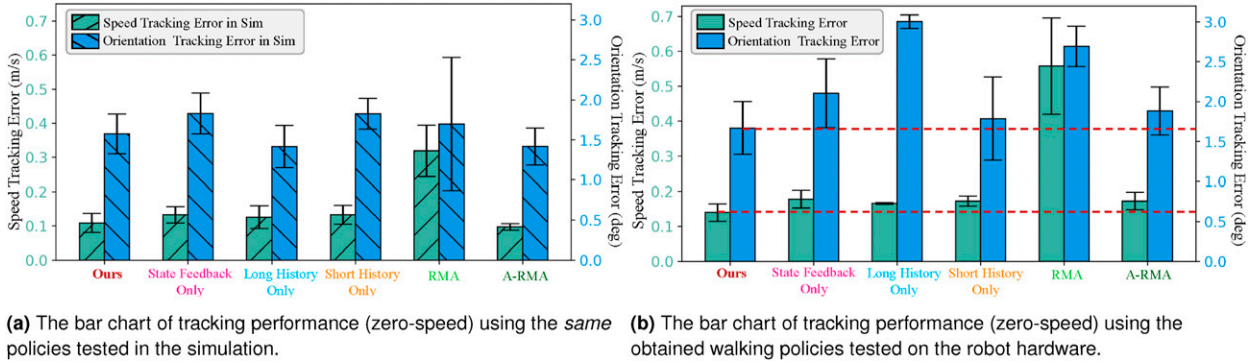


Figure 8. The bar chart of the tracking performance using different policies obtained by different methods (Figure 5) in the in-place walking experiments in the simulation (Figure 8(a)) and the real world (Figure 8(b)). The speed tracking error (Mean Absolute Error, MAE, of $\|\dot{q}_{x,y}\|_2$) and orientation tracking error (MAE of $q_{\phi,\theta,\psi}$) using the policies trained from *different random seeds* but same method are recorded. They are shown in green and blue bars, respectively. The bar height is the average tracking error over three random seeds and the error bar represents the standard deviation among these three tests. We exclude the residual approach as it cannot maintain a stable gait. Training from different random seeds, while all methods perform similarly well in the simulation, when transferring the same policies to the hardware, our method consistently demonstrates the minimal speed tracking error and orientation tracking error over other methods. This underscores the capacity of the proposed method in terms of adapting to the robot dynamics in the real world. (a) The bar chart of tracking performance (zero-speed) using the *same* policies tested in the simulation. (b) The bar chart of tracking performance (zero-speed) using the obtained walking policies tested on the robot hardware.

A-RMA, while best at maintaining the robot’s position in simulation, causes significant drift to the robot’s left on real hardware. In contrast, our method achieves better sim-to-real transfer performance with minimal degradation and consistently results in better control performance in terms of command tracking and stabilizing the floating base. This underscores the advantages of our approach in adapting to the dynamics of the robot hardware.

Consequently, we can conclude that the proposed method excels in bridging the sim-to-real gap, demonstrating better performance in effectively controlling the robot in the real-world environment.

7.4. Summary of results

For achieving dynamic bipedal locomotion skills through RL, the results in this section indicate the following design choices for the control policy structure:

- (1) Have the policy to directly specify motor-level commands rather than using a residual term;

- (2) Utilize a history of both the robot’s input and output, rather than relying solely on the robot’s state feedback;
- (3) When using the robot’s I/O history, favor the inclusion of long-term history, but complement it with short-term history in the base policy for enhanced performance, which leads to a dual-history approach; and
- (4) Train the base policy and the history encoder in an end-to-end manner, as it yields better performance, reducing training complexity and samples compared to policy distillation methods.

These design choices lead to our proposed policy structures, showcasing consistently best learning performance in simulation across various dynamic locomotion skills and delivering improved control results in the real world.

8. Source of adaptivity

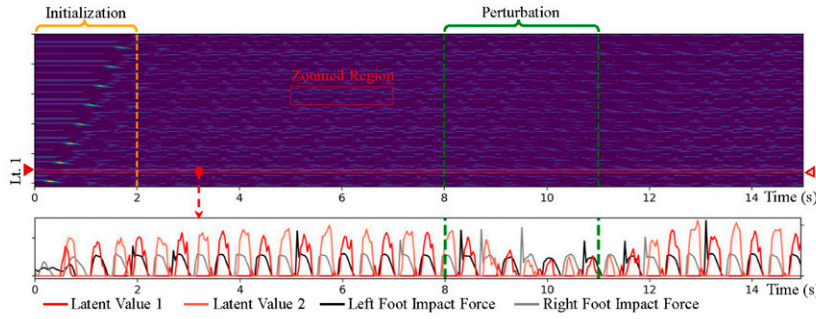
To understand why the proposed method shows advantages in achieving better learning performance in the

challenging scenario of performing dynamic locomotion skills with varying environment parameters, we delve into an examination of the latent representation provided by the encoder for long I/O history, as shown in Figure 9 and Appnedix 7. These results are obtained in simulation. As we will see, the encoder that utilizes long I/O history can capture both time-varying events and time-invariant

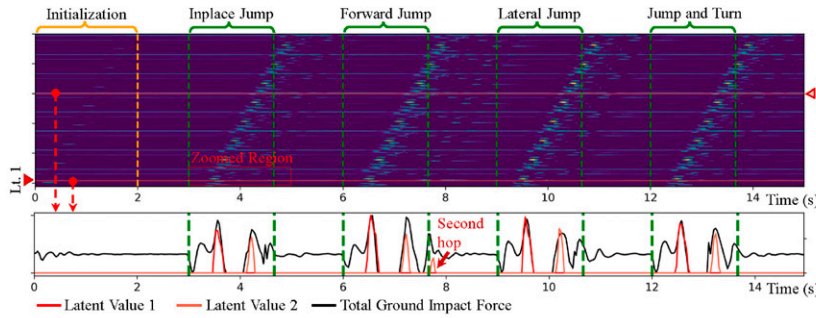
changes in dynamics parameters in all the locomotion skills we developed.

8.1. Time varying embedding

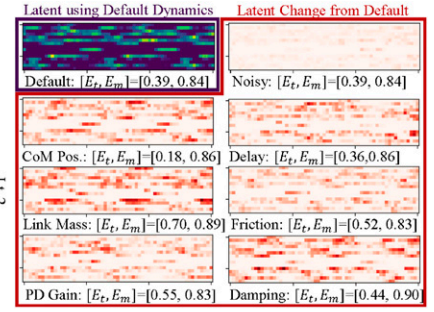
In this subsection, we observe that the latent embedding from the long I/O history encoder can effectively adapt to



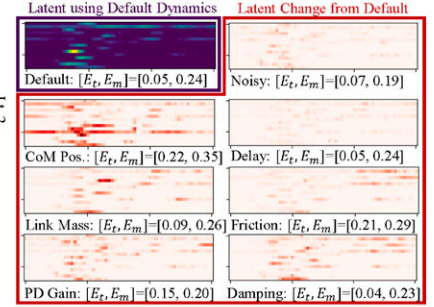
(a) (Top) Recorded latent representation after long-term I/O history encoder during running. (Bottom) Comparison of two selected dimensions (marked as red lines in the top plot) with recorded impact forces on each of the robot's feet.



(c) (Top) Recorded latent representation after long-term I/O history encoder during jumping. (Bottom) Comparison of two selected dimensions (marked as red lines in the top plot) with recorded total impact forces on both of the robot's two feet.



(b) The blue plot shows the robot's latent representation with default dynamics parameters during running. The red plots indicate changes in the same region under different dynamics.



(d) The blue plot shows the robot's latent representation with default dynamics parameters during jumping. The red plots indicate changes in the same region under different dynamics.

Figure 9. Adaptivity test in simulation (MuJoCo). We evaluate the latent representation profiles from the output of the long-term I/O history encoder during bipedal running (Figure 9(a)) and jumping (Figure 9(c)) where lighter color represents a larger relative value. After the initialization of the history encoder (first 2 seconds), variations in the latent embedding are noticeable for different activities like running versus jumping or standing, for specific jumping targets, and for the existence of external perturbations. Two latent values (indicated by red horizontal lines in the top plots) are plotted in the bottom plots to show that the latent space also captures the changes in the impact forces. To conduct an ablation study on time-invariant dynamics parameters, we examine the same regions inside red boxes (titled zoomed region) in Figure 9(a) and 9(c) and show these regions in Figure 9(b) and 9(d) when the robot is performing constant-speed running and in-place jumping respectively. The *Latent using Default Dynamics* shows these regions when then robot's nominal model is simulated without any changes to the dynamics or introducing noise or delays. The *Latent Change from Default* shows the delta change in the same regions when we adjust the CoM position of each link (marked as *CoM Pos.*), link mass (*Link Mass*), PD gains used on each motor (*PD Gain*), ground friction (*Friction*), joint damping ratio (*Damping*), communication delay (*Delay*), and noise levels (*Noisy*) one at the time. In these plots, a larger change is represented by a darker color. This shows that the latent embedding can reflect different changes in dynamics, while effectively filtering out noise which causes little changes. Despite significant environment changes, control performance metrics like task completion error (E_t) and motion tracking error (E_m) show minor degradation, showcasing adaptivity to dynamic changes using the proposed controller. We further visualize the corresponding saliency map in Appendix 8. (a) (Top) Recorded latent representation after long-term I/O history encoder during running. (Bottom) Comparison of two selected dimensions (marked as red lines in the top plot) with recorded impact forces on each of the robot's feet. (b) The blue plot shows the robot's latent representation with default dynamics parameters during running. The red plots indicate changes in the same region under different dynamics. (c) (Top) Recorded latent representation after long-term I/O history encoder during jumping. (Bottom) Comparison of two selected dimensions (marked as red lines in the top plot) with recorded total impact forces on both of the robot's two feet. (d) The blue plot shows the robot's latent representation with default dynamics parameters during jumping. The red plots indicate changes in the same region under different dynamics.

time-varying external disturbances or locomotion tasks, implicitly estimating contact events and/or external forces across all three distinct locomotion skills.

8.1.1. Periodic running. Using the running policy obtained by the proposed method, we recorded the latent embedding over a 15-second duration, initiating from a standing position and progressing to follow the constant running speed command of 3 m/s. A persistent backward perturbation force of 40 N was applied to the robot base from 8 to 11 seconds. The evolution of latent values over time is depicted in Figure 9(a).

Given that running is a periodic skill, the latent embedding also exhibits a periodic pattern once the gait has stabilized, as seen in Figure 9(a) after 2 seconds. Additionally, the presence of the perturbation introduces variations in the latent embedding (within the green dashed lines), showcasing the capability to capture time-varying disturbances.

Additionally, we found two intriguing latent dimensions highlighted by the red line and plots in Figure 9(a). These two latent signals exhibit a strong correlation with the impact force on the robot's left and right foot. Specifically, the values of these two latent dimensions vary in a tendency consistent with the ground truth impact force recorded in simulation, reaching zero when the corresponding force is zero (the respective foot is in a swing phase). This result highlights a crucial and demanding capability for controlling legged robots: contact estimation, as the trajectory breaks upon contact.

The advantage of implicit contact estimation becomes more evident with changes in these two latent dimensions in the presence of external perturbation (shown within the green dashed lines in Figure 9(a)). Despite the unchanged magnitude of the ground impact force, these two latent values shift to a lower envelope during the perturbation, followed by a recovery to the previous envelope after the perturbation is taken out. While an exact explanation for this change remains elusive as it is learned purely through data, it *may* be attributed to the notion that both external perturbation and ground reaction force can be treated as a generalized external force ζ_{ext} applied to the robot, as implied by the robot's full-order dynamics (1). The robot may have unconsciously learned to embed such external forces all together into these signals and leverage them in control, without explicit human engineering, through the provided long I/O history. A similar capacity for such a long history encoder for periodic walking skill is also observed, and we analyze it briefly in Appendix 7.

8.1.2. Aperiodic jumping. Despite the periodic running and walking skills, we also discovered a similar capacity of the I/O history encoder to capture time-varying information in aperiodic skills like jumping. As depicted in Figure 9(c), Cassie was commanded to execute different jumping tasks every 3 seconds, starting from an in-place jump to a 1.4-meter forward jump, followed by a

0.5-meter sideways jump and a jump and turn of -60° before standing.

The recorded time-evolving latent representation reveals a distinct contrast between jumping phases (that have more varying and non-zero signals) and standing phases (with less varying signals). Furthermore, for different jumping tasks, the latent values during the jumping phases differ, as illustrated in Figure 9(c).

Moreover, we discovered two latent dimensions that strongly correlate with contact events during jumping, as indicated by the red lines and plotted in Figure 9(c). In contrast to a common human expectation for a single binary variable of contact during jumping (either in contact or not), the robot learned to utilize two distinct signals: take-off event and landing event, estimated by Latent Value 1 and 2, respectively, as depicted in Figure 9(c). For example, the Latent Value 1 begins to increase and drops to zero just before the total contact force reaches zero (indicating the robot is starting to take off), while Latent Value 2 only becomes active at the moment the robot lands. Furthermore, during the forward jump the robot executes a small hop after it lands, Latent Value 2 exhibits an additional spike during that additional hop. Such separated signals for take-off and landing could provide more informative cues for controlling a bipedal jumping skill, given the difference in control complexity during the flight and landing phases.

8.2. Adaptive embedding for changes in dynamics

We now assess how the latent embedding changes with (time-invariant) variations in the robot dynamics model, specifically, alterations in the $\mathbf{M}$, $\mathbf{C}$, $\mathbf{G}$, κ , and Jacobians in (1), or with corrupted measurements (including noise and delays). Using the aforementioned running policy, we focus on a segment of the periodic pattern illustrated in Figure 9(a), highlighted by the red block. The zoomed-in pattern, observed when controlling the robot with the default dynamics model, is visualized in Figure 9(b).

We conduct an ablation study using the same running policy in the same task (tracking a 3 m/s command) with changes in the dynamics parameters. In particular, we adjust one specific modeling parameter one at a time, such as the link CoM position (increased by 8 cm for all links), link mass (30% greater for all links), joint damping ratio (8 times the default value), and PD gains used on each motor (40% higher than the default value), and lower ground friction (static friction ratio set to 0.5 for slippery ground). As shown in Figure 9(b), each change of these dynamics parameters results in a significant shift in the resulting latent embedding compared to the pattern obtained with the default model. However, evaluating control performance metrics, such as task completion (tracking error E_t between the robot's actual speed and desired speed) and motion tracking (motion deviation E_m from the robot's actual motor position and reference motion), reveals minimal change.

A similar ablation study is undertaken in jumping and walking skills, recorded in Figure 9(d) and Appendix 7, respectively. Specifically, within the in-place jumping task utilizing the obtained jumping policy, we modify the same dynamics parameters as in running, resulting in varying patterns of the latent embedding for different dynamics models. Despite these alterations, the control performance metrics, such as task completion E_t (the error in the landing position compared to the desired one) and motion tracking error E_m , also show minimal changes.

When assessing the influence of corrupted measurements on the latent embedding, the latency in control and observation (0.025 s) induced noticeable changes in running and walking, as showcased in Figures 9(b) and 32(b). Surprisingly, the introduction of significantly larger noise (2 times larger than the upper bound used during training as listed in Table 4) does not exert a considerable effect on the resulting latent embedding for all running, jumping, and walking skills. This result underscores the efficacy of the long history encoder in effectively filtering out the added zero-mean noise, which may be also brought by the CNN structure.

8.3. Summary of results

This ablation study highlights the adaptivity of the proposed controller. Empowered by the history encoder's ability to capture meaningful information from the I/O history, this controller can adapt to time-varying events such as external perturbations or contacts, time-invariant changes in dynamics parameters, and filter out measurement noise, and therefore, effectively execute the control task with minimal performance degradation. This capability explains why the proposed structure excels in the challenging training setting with a large range of randomization of dynamics parameters. Notably, in order to fully harness such advantages brought by the robot's long I/O history, a proper structure (dual-history approach) and training strategy (end-to-end training) are necessary, as discussed in Sec. 7.

9. Advantages of versatile policies and source of robustness

We now validate another key finding in this study, which is the source of robustness when using RL to obtain a locomotion controller, in both simulation and the real world. Our findings indicate that, in the context of bipedal locomotion control, a single versatile policy capable of executing various tasks can significantly improve the robustness compared to policies specialized in individual tasks. This is realized by task randomization and generalization.

9.1. Baselines

We conduct a benchmark among the proposed versatile policies and task-specific baselines across all four distinct

locomotion skills, including standing, walking, running, and jumping. For each of these four skills, we develop three policies, and they are:

- **Single Task:** The policy that is only trained with a single fixed task and dynamics randomization (excluding simulated perturbations). For walking, the task is to walk forward at 0.6 m/s; for running, the task is running at 3 m/s; for jumping, the task is to jump in place; and for standing, the task is to stand still.
- **Single-Task w/Perturbation:** The policy that is trained with the same fixed task, same dynamics randomization as the **Single-Task** policy, but is also trained with additional simulated perturbations.
- **Versatile (Ours):** The policy that is trained to handle the complete spectrum of commands (tasks) detailed in Table 6, with the dynamics randomization but *without* any perturbations. For the walking and running versatile policies, they also learned the transition between locomotion and standing.

All of them used the proposed policy structure as shown in Figure 3 and have trained with the same range of dynamics randomization introduced in Sec. 6.5 until convergence. For the policies trained with simulated perturbations, the range of perturbation is also the same as specified in Table 4.

The above-mentioned policies are examples trained with three different methods: (1) dynamics randomization (single-task policies trained *without* simulated perturbations), (2) perturbation training (single-task policies trained *with* perturbations during dynamics randomization), and (3) task randomization (versatile policies). *Usually, perturbation training is considered as an additional dynamics randomization.*

9.2. Source of robustness

We first start our validation in a simulation, which provides a controlled and safe environment to assess the robustness of the control policies. In this validation, we are going to show that the robustness of a versatile locomotion policy comes from two key factors: (1) the generalization of trained randomized dynamics parameters introduced by *dynamics randomization*, and (2) the generalization of various trained locomotion tasks, thereby endowing the robot with notable *compliance*, brought by *task randomization*.

For each locomotion control policy we've developed (including walking, running, and jumping), we test them under two distinct forms of uncertainties: (1) applying a consistent force on the robot's pelvis, and (2) introducing a significant deviation in the CoM position for all links of the robot, as depicted in Figure 10. It's important to note that these uncertainties exceed the bound used during training, as outlined in Table 4. For example, unlike the impulse perturbation used during training, here we apply a consistent perturbation, and the CoM position offset is greater than

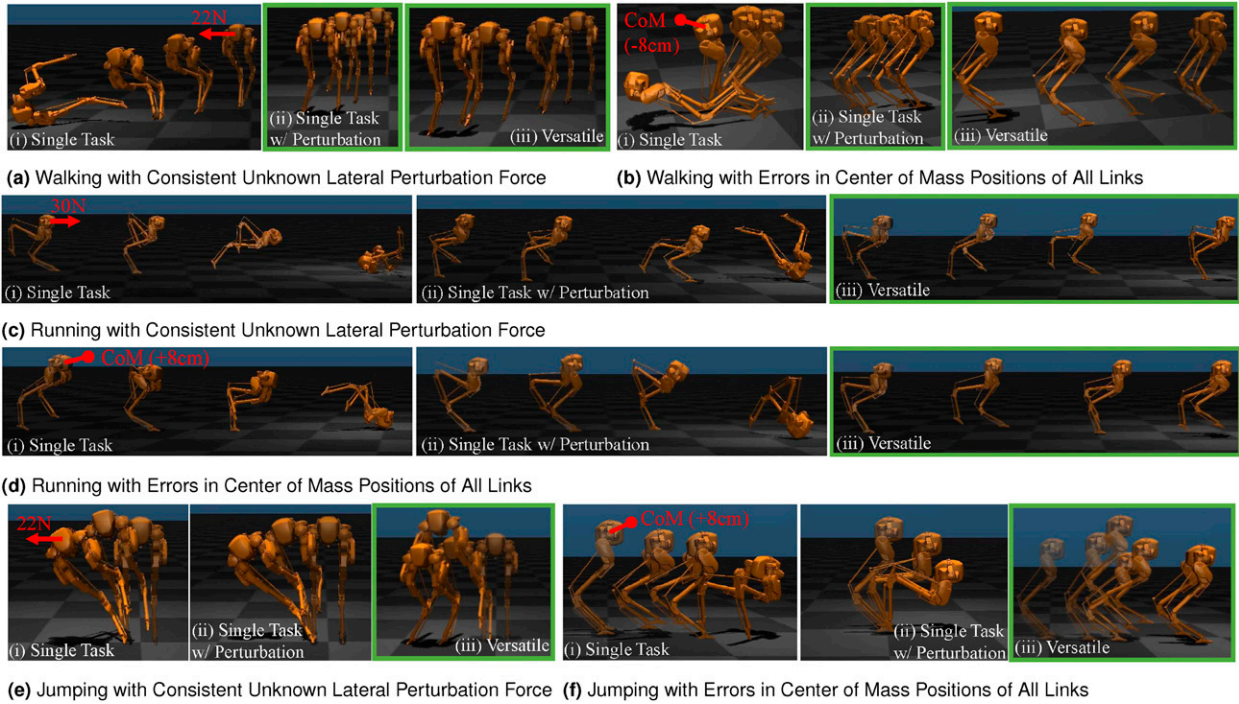


Figure 10. Robustness test in simulation (MuJoCo). We conduct an ablation study to evaluate the relationship between the robustness and task randomization using the baselines developed in Sec. 9.1. The green bounding box indicates the scenario where the policy successfully controlled the robot without falling over. The robot is subjected to consistent perturbation forces (with its direction marked by a red arrow) or CoM position offsets of all links (with the robot's entire CoM conceptually drawn as a red point) while performing fixed locomotion tasks like walking at 0.6 m/s, running at 3 m/s, or jumping in place. Policies focused on a single task, even with extensive dynamics randomization, failed in these out-of-distribution scenarios. With additional perturbation training, the robot can maintain stability during walking, as seen in Figure 10a(ii) and Figure 10b(ii). However, such policies are less effective for more dynamic skills like running or jumping, as illustrated in Figure 10c(ii) and Figure 10f(ii). In contrast, a versatile policy, trained across a variety of tasks, exhibits better robustness even without specific perturbation training. The robot changes its gait under external perturbations or CoM shifts, as demonstrated by using lateral (Figure 10a(iii)) and backward walking gaits (Figure 10b(iii)), highlighting the compliance of versatile controllers. This compliance is also observed during running and jumping (Figure 10c(iii) and Figure 10f(iii)), which is the key to success in these challenging robustness tests. (a) Walking with Consistent Unknown Lateral Perturbation Force. (b) Walking with Errors in Center of Mass Positions of All Links. (c) Running with Consistent Unknown Lateral Perturbation Force. (d) Running with Errors in Center of Mass Positions of All Links. (e) Jumping with Consistent Unknown Lateral Perturbation Force. (f) Jumping with Errors in Center of Mass Positions of All Links.

what was covered in the training scenarios. The results of these out-of-distribution tests are illustrated in Figure 10.

Consider the example of a walking task: when a robot is commanded to walk forward and faces a consistent lateral pulling force of 22 N, a single-task policy tailored for forward walking fails due to the lateral perturbation pushing the robot beyond its training distribution, as seen in Figure 10(a)(i). However, if the robot is trained with perturbation using a single-task policy, it can progress forward with minor lateral deviation under such force, shown in Figure 10(a)(ii). In contrast, using a versatile policy that is not trained with perturbations but trained with various walking tasks, the robot still stabilizes and performs the forward walking task, albeit differently. This is evident from Figure 10(a)(iii), where the robot significantly deviates to the right, using learned side walking skill to compensate for the external force, resulting in a *compliant* gait.

Similar results are observed with a CoM position offset of -8 cm in all links. Without training for perturbations or

other tasks, the robot, even with randomized CoM training, cannot handle this deviation, as shown in Figure 10(b)(i). When trained with perturbation, since the robot has learned to stabilize under a backward force, it uses such a learned control strategy to counter the backward CoM position offset and keep walking forward with a reduced speed, as depicted in Figure 10(b)(ii). Yet, using a versatile policy without perturbation training, the robot instead leverages backward walking gaits to offset the rearward CoM shift, as seen in Figure 10(b)(iii).

Please note that we are not trying to argue that perturbation training is inferior to task randomization. For example, in the walking task, robots trained with a single task and perturbations showed more favor ability to complete the assigned task (less tracking error shown in Figure 10(a)(ii) and 10b(ii)), compared to those controlled by versatile policies (Figure 10(a)(iii) and 10(b)(iii)). Therefore, incorporating external perturbations during dynamics randomization is still recommended following

task randomization. However, the enhancement in robustness from simulated dynamics is not as significant as that from task randomization, especially in more dynamic tasks like running and jumping.

As depicted in Figure 10(c)(i) and 10(c)(ii), under a constant forward perturbation of 30 N while running, policies trained only with single task fail to maintain stable gaits, regardless of extensive dynamics randomization and additional perturbation training. In contrast, robots using the versatile policy that has trained for faster running can adapt to such perturbations, as demonstrated in Figure 10(c)(iii). Similar results are observed in scenarios involving a +8 cm CoM position offset during running (Figure 10(d)), and in lateral perturbation (by using a lateral jump) and forward CoM position offset (by using a forward jump) cases in jumping (Figure 10(e) and 10(f)). Note that all of these versatile policies are not trained with simulated perturbations.

These studies highlight the two distinct sources of robustness of RL policies: (1) Training with specific simulated dynamics, including external perturbations, allows functioning within an expanded scenario range but limits the robot to the trained task; (2) Training with a diverse task set enables the robot to generalize learned tasks for greater robustness and compliance, even without extensive dynamics randomization.

9.3. Case study: Robust standing experiments

We conduct a case study in the real world to gain insights into the underlying source of robustness in control policies developed through RL. This case study centers on evaluating the performance of the standing skill.

In this scenario, as shown in Figure 11, we introduce an external forward perturbation to the robot's pelvis while it

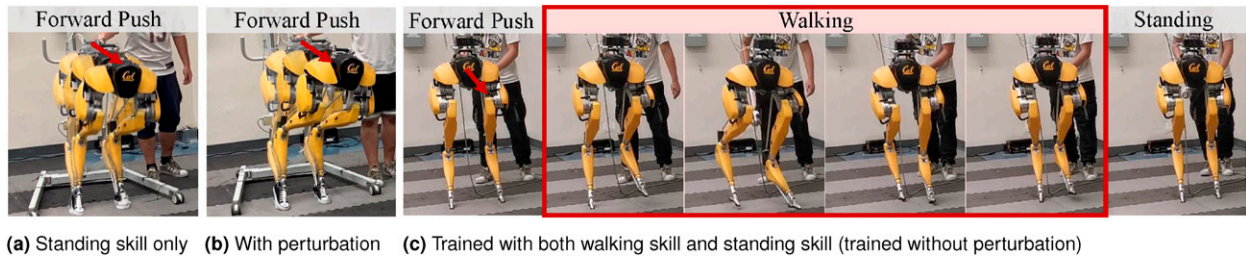


Figure 11. Robustness test for bipedal standing skill in the real world. If the robot is trained exclusively with the standing skill, regardless of whether it underwent extensive dynamics randomization with (Figure 11(b)) or without simulated perturbation (Figure 11(a)), the robot tends to fall when pushed beyond its support region. In contrast, the robot using our versatile policy, trained in both walking and standing skills as shown in Figure 11(c), demonstrates better robustness. Despite not being trained with perturbation during standing, the robot can spontaneously break contact when pushed forward and employ its generalized walking skills to regain its standing pose. The video of such a comparison is recorded in Vid. 3 in Table 1. (a) Standing skill only. (b) With perturbation. (c) Trained with both walking skill and standing skill (trained without perturbation).

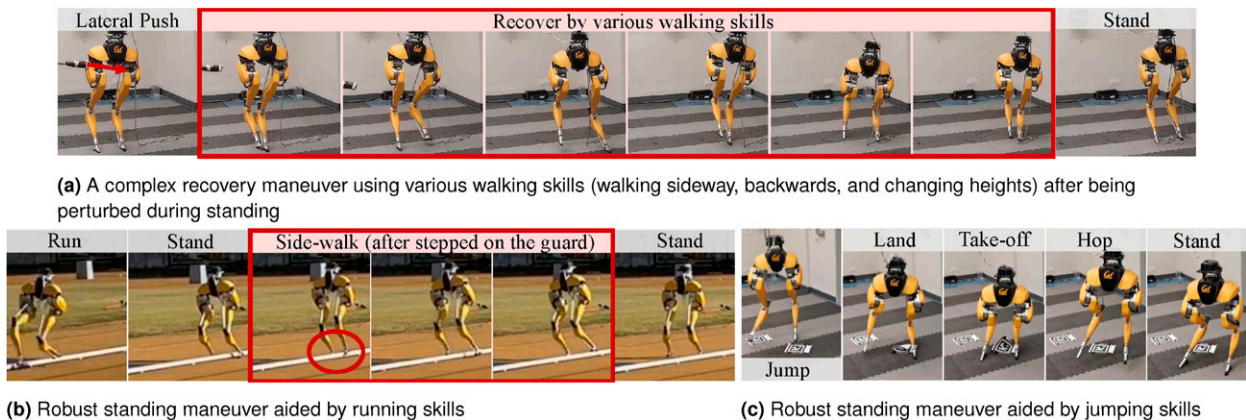


Figure 12. Robust bipedal standing skill realized by versatile policies in the real world. The robot, while standing, can intelligently break contact and engage learned locomotion skills as needed. For instance, as shown in Figure 12(a), when perturbed, the robot deviates from its standing command and autonomously employs a sequence of contacts and varying walking skills over a long horizon to return to a standing position. This robustness, a result of task randomization, enhances real-world deployment capabilities. For example, in transitions to standing, like in Figure 12(b), the robot uses side-stepping skills acquired during running to aid recovery from stepping on a track guard. Similarly, with the versatile jumping skill in Figure 12(c), the robot breaks contact after landing from an unstable state, adjusting its pose in the air for better landing, all without relying on any human-defined contact sequences. (a) A complex recovery maneuver using various walking skills (walking sideways, backwards, and changing heights) after being perturbed during standing. (b) Robust standing maneuver aided by running skills. (c) Robust standing maneuver aided by jumping skills.

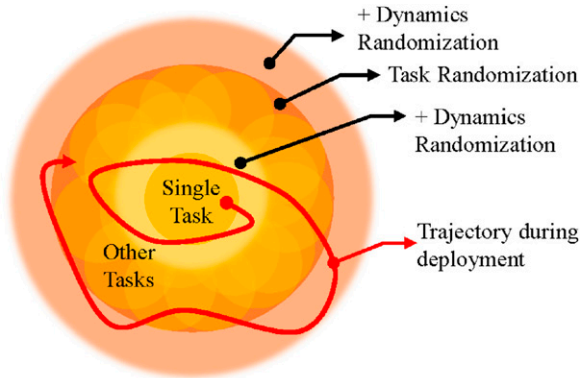


Figure 13. An illustration of the concept of training distributions using different methods to enhance robustness. During deployment, as conceptually illustrated by the red curve, we want the robot controlled by its RL-based policy to operate inside the training distribution of the robot's trajectories. When the training is focused on a single task, the training distribution is confined to nominal trajectories specific to that task, drawn as the yellow region. Incorporating extensive dynamics randomization, including simulated perturbations or varying terrains, can expand this distribution. However, this expansion is still centered around the fixed task. Task randomization significantly broadens the training distribution (to the orange region) by enabling the robot to learn and generalize various control strategies across different tasks (marked as different faded yellow regions). It is important to note that task randomization can be combined with dynamics randomization, further widening the training distribution and enhancing the policy's robustness.

maintains a standing pose. When policies are exclusively trained for the standing skill, regardless of whether they were trained with external perturbations, the robot keeps losing its balance if it is forced to lean beyond its support region, as recorded in Figure 11(a) and 11(b). Conversely, when we employ the versatile walking policy, which is also trained with the standing skill, and subject the robot to a similar forward perturbation while it is standing, as demonstrated in Figure 11(c), the robot initially leans forward. However, should the robot start leaning outside its support region, it demonstrates intelligent recovery maneuvers. It transits to a walking gait by breaking ground contact, executes several steps, including forward and backward walking, and then smoothly reverts to a standing pose again. Remarkably, this transition occurs without any human-provided commands, as the only reference motion provided is a standing motion. Also, we do not simulate external perturbation during the training of the standing skill using this walking policy.

The presence of a robust standing skill is highly advantageous in real-world deployments. For example, when the robot is laterally perturbed while standing, as shown in Figure 12(a), it utilizes its varied walking skills to recover and return to a stand. This complex sequence, transitioning through various walking maneuvers to lower its center of mass and then resuming a standing pose, highlights the policy's capability for long-horizon recovery maneuvers. Notably, this sophisticated

recovery was not specifically trained but is a natural outcome of the versatile policy. This benefit extends to other skills as well. When using the versatile running policy, as seen in Figure 12(b), the robot halts from running and steps on the track guard, as recorded in Figure 12(b). Without loss of balance, the robot promptly disengages from the guard and uses its acquired side-stepping skills when learning the running skill, and maintains a stable stance pose afterward. Similarly, with the versatile jumping policy (Figure 12(c)), after an unstable landing from a complex multi-axis jump, the robot executes a corrective hop, which is learned from diverse jumping tasks, to better correct itself in the air and to have a more stable landing configuration. These results can be better seen in the videos listed in Table 1.

9.4. Understand robustness from training distributions

The above-mentioned findings can be conceptually illustrated in Figure 13, similar to the notion of an *invariant set* in control theory. To effectively control the robot, an RL policy needs to address a range of disturbances encountered during deployment. A model-free RL policy can be particularly effective when the robot's trajectory lies within its training distribution, as the policy has been specifically trained and optimized for such scenarios. Single-task policies have a limited training distribution, focused only on that task, and training with random dynamics w/o perturbations can extend this distribution, as shown in Figure 13. However, as evidenced in Figure 10, dynamics randomization only modestly expands this distribution, primarily enhancing robustness within the single-task scheme. In contrast, task randomization broadens the training distribution significantly. By enabling the robot to handle diverse tasks, this approach can effectively expand the range of the trained robot's I/O trajectory. When faced with disturbances during deployment, the robot can still remain within this enhanced training distribution, as illustrated in Figure 13. When combined with dynamics randomization after the task randomization, as proposed in our work in Figure 4, the training distribution can further expand substantially and therefore enhance the robustness of the RL policy.

Remark 2. The range of dynamics randomization cannot be arbitrarily large as it will introduce significant challenges during training (the robot may fail to learn meaningful skills). The dynamics randomization range used in this work (Table 4) is challenging enough for Cassie to learn, which can be evident by the low converged training return in Fig. 6. Further adding dynamics randomization has limitations as suggested in Xie et al. (2021). Therefore, task randomization can be viewed as an “orthogonal” way to improve the robustness rather than further pushing the range of the dynamics randomization.

9.5. Summary of results

In conclusion, compared to policies focused solely on specific tasks, versatile policies exhibit significant improvements in robustness, which is validated in both simulation and the real world. This enhanced robustness stems from their ability to generalize learned tasks and to find better maneuvers to tackle unforeseen situations without being limited to adhering to given commands, ultimately leading to better stability. In RL-based locomotion control, one key source of robustness is the capacity to perform versatile tasks. Hence, task randomization, which diversifies the tasks during training, is recommended for future development on legged locomotion control.

10. Dynamic bipedal locomotion in the real world

We now extensively evaluate the obtained versatile policies for walking, running, and jumping in the real world. All of these policies are able to effectively control the robot in the real world without further tuning after being trained in simulation. Moreover, the main objective of the experiments is to evaluate two fundamental aspects of the proposed method: (1) the obtained policies' *adaptivity* to the actual robot dynamics in the real world by effectively controlling the real robot to achieve the assigned tasks, as in simulation; and (2) the policies' *robustness* in addressing different uncertainties in the real world by leveraging various learned and tasks.

10.1. Walking experiments

We now examine the versatile walking policy on Cassie. The following experiments are conducted by a *single* controller. As we will see, the obtained walking policy shows consistent control performance on the robot hardware over a long timespan with notable robustness and compliance.

10.1.1. Tracking performance. The policy demonstrates the capacity to reliably track varying, fast-changing commands, with consistent control performance over a long period that is more than a year. Such a result showcases the adaptivity of the proposed policy that adapts to the dynamics of the real robot, as discussed below.

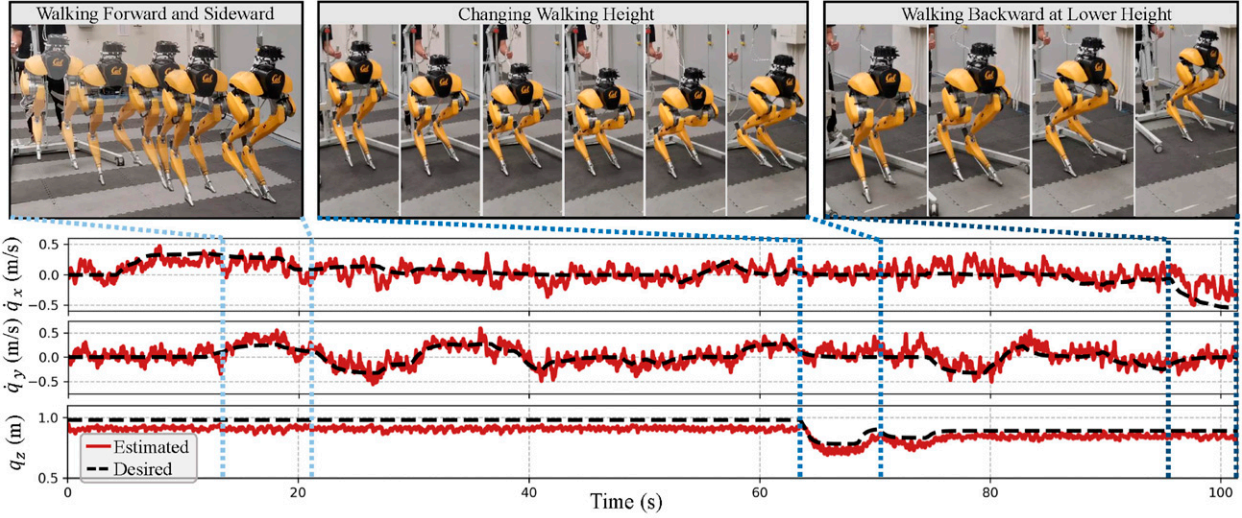
10.1.1.1. Variable commands. As recorded in Figure 14(a), the walking policy demonstrates efficient control over the robot's following of diverse commands, including variations in sagittal and lateral velocities ($\dot{q}_{x,y}$) and walking height (q_z). Throughout the test, the tracking errors, evaluated by Mean Absolute Error (MAE), remain reasonably low. The MAE in the ($\dot{q}_x, \dot{q}_y, q_z$) are (0.10 m/s, 0.10 m/s, 0.06 m) respectively. Additionally, as indicated in Figure 15, the policy exhibits reliable control, enabling the robot to accurately track varying turning commands, either clockwise or anti-clockwise.

10.1.1.2. Consistency over a long timespan. The robot hardware, especially for bipedal robots, keeps changing due to wear and tear over time. For instance, joint friction might vary due to the impacts during operation, and these changes can accumulate due to the bipedal robot's large number of DoFs and over extended periods. Consequently, controllers dependent on the control gains tuned on hardware, like most model-based controllers, often require manual updates to deal with these hardware changes. In contrast, our proposed RL-based walking policy can adapt to changing robot dynamics and control the robot without the need for tuning on the hardware. This adaptivity is evident as the policy consistently performs well over extended periods, maintaining its ability to track variable commands even after 325 and 492 days, as showcased in Figures 14(b) and 14(c), respectively. Compared with the test conducted earlier (Figure 14(a)), the changes of the tracking errors (MAE) in ($\dot{q}_x, \dot{q}_y, q_z$) during these two tests are (+0 m/s, -0.02 m/s, +0 m) and (+0.01 m/s, -0.01 m/s, -0.01 m), respectively, over a relatively long testing timespan (over 2 minutes) with different combinations of commands. Despite significant accumulated changes in the robot's dynamics over this period, the same controller from Figure 14(a) continues to effectively manage varying walking tasks with only minimal tracking error degradation.

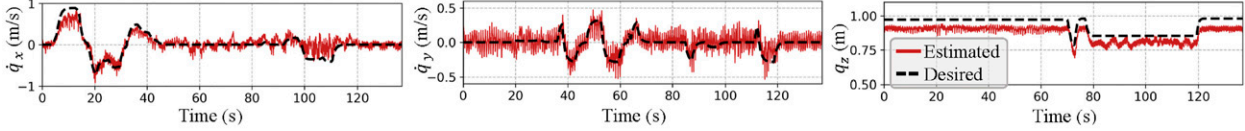
10.1.1.3. Fast walking. In addition to previously demonstrated moderate walking speeds, the policy exhibits the ability to control the robot to perform fast walking maneuvers, both forwards and backwards, as shown in Figure 16. The robot can transition from a standstill to rapidly achieve a forward walking speed with an average velocity of 1.14 m/s to track the command of 1.4 m/s, quickly returning to a standing position as commanded, as shown in Figure 16(a) with data recorded in Figure 16(c). Notably, during the transition from a stance pose with zero speed, the robot can quickly swing its legs forward with a considerable amount of acceleration while maintaining gait stability to follow the step-input command. Similar capacity is observed in the context of fast backward walking (Figure 16(b)), where the robot seamlessly shifts from a standing pose to perform a backward walking gait with an average velocity of -0.5 m/s, achieving the specified -1 m/s walking task before promptly returning to a standing position upon command.

10.1.2. Robust walking maneuvers. We further test the robustness of the walking policy, and the proposed policy shows notable robustness in addressing various changes in the environment, as showcased below.

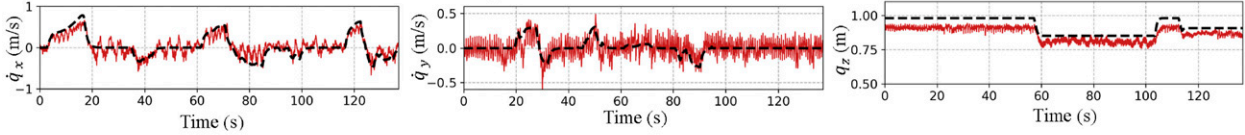
10.1.2.1. Uneven terrains (untrained). Although the walking policy has not been specifically trained for traversing uneven terrain in simulation, it displays considerable robustness to varying elevation changes during deployment. As depicted in Figure 17, the robot can



(a) Experiments on variable command tracking conducted on August 9, 2022. The tracking errors (Mean Absolute Error, MAE) in the (sagittal velocity $\dot{q}_x^d$, lateral velocity $\dot{q}_y^d$, walking height q_z^d) are (0.10 m/s, 0.10 m/s, 0.06 m), respectively.



(b) Experiments on variable command tracking conducted on June 30, 2023, 325 days after the one conducted Fig. 14a using the same controller without any tuning. The tracking errors (MAE) in the ($\dot{q}_x^d$, $\dot{q}_y^d$, q_z^d) directions are (0.10 m/s, 0.08 m/s, 0.06 m), respectively.



(c) Experiments on variable command tracking conducted on December 14, 2023, 492 days after the one conducted in Fig. 14a using the same controller without any tuning. The tracking errors (MAE) in the ($\dot{q}_x^d$, $\dot{q}_y^d$, q_z^d) directions are (0.11 m/s, 0.09 m/s, 0.05 m), respectively.

Figure 14. Figure 14(a) records a snapshot of the robot reliably tracking varying commands using the obtained versatile walking controller in the real world. The desired command and actual estimated values are recorded onboard during the experiments and drawn in the lower part. The dashed blue lines match the corresponding interval of frames in the real world. The robot can track different desired commands including changing sagittal velocity $\dot{q}_x^d$, lateral velocity $\dot{q}_y^d$, and walking height q_z^d , with a considerable accuracy that has a tracking error over a long test. Similar variable command tracking experiments are conducted after a long timespan, such as the one conducted after 325 days in Figure 14(c) and the one after 492 days in Figure 14(b). Although the robot hardware has been changing due to wear and tear after such a long time, the same RL-based controller can still effectively control the robot to track variable commands with similar tracking errors, without any tuning in the real world. (a) Experiments on variable command tracking conducted on August 9, 2022. The tracking errors (Mean Absolute Error, MAE) in the (sagittal velocity $\dot{q}_x^d$, lateral velocity $\dot{q}_y^d$, walking height q_z^d) are (0.10 m/s, 0.10 m/s, 0.06 m), respectively, (b) Experiments on variable command tracking conducted on June 30, 2023, 325 days after the one conducted Figure 14(a) using the same controller without any tuning. The tracking errors (MAE) in the ($\dot{q}_x^d$, $\dot{q}_y^d$, q_z^d) directions are (0.10 m/s, 0.08 m/s, 0.06 m), respectively, (c) Experiments on variable command tracking conducted on December 14, 2023, 492 days after the one conducted in Figure 14(a) using the same controller without any tuning. The tracking errors (MAE) in the ($\dot{q}_x^d$, $\dot{q}_y^d$, q_z^d) directions are (0.11 m/s, 0.09 m/s, 0.05 m), respectively.

effectively walk *backward* on small stairs or declined slopes. It's noteworthy that the robot lacks any terrain elevation sensors. Such ability stems from the policy's robustness to the change of contact timing or wrench while walking on varying elevations, facilitated by *not* requiring explicit estimation or control of the contact in the controller.

10.1.2.2. Robustness to random perturbations. In this test, we evaluate the robustness of the proposed versatile walking policy under two types of external perturbations: (1) impulse perturbation whose elapsed time is less than 1

second; (2) persistent perturbation that lasts more than 3 seconds (longer than the length of controller's I/O history).

For the impulse perturbation case, we introduced external perturbations over a short timespan from various directions to the robot while walking. As an example recorded in Figure 18(a), a substantial lateral perturbation force is applied to the robot while walking in place, causing a significant lateral velocity peak of 0.5 m/s. Despite this force, the robot swiftly recovers from the lateral deviation. As recorded in Figure 18(a), the robot adeptly moves in the

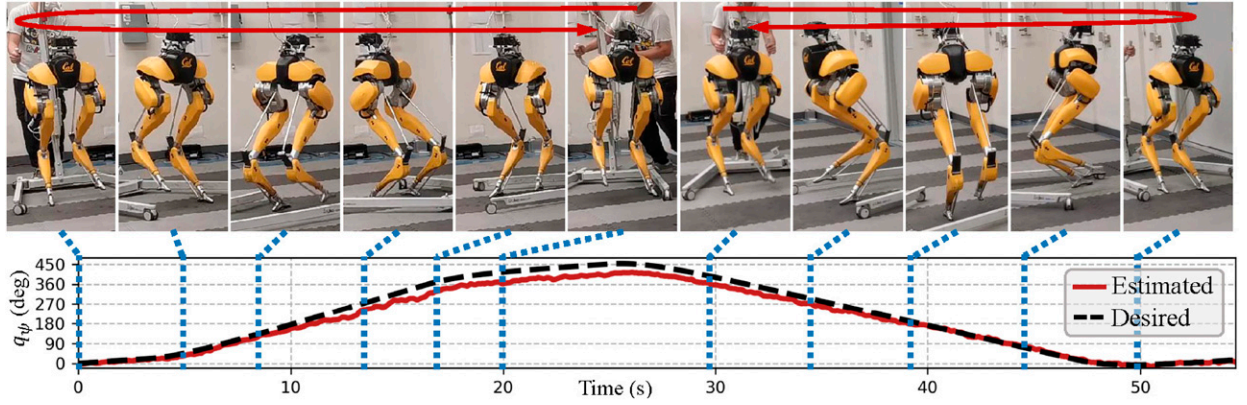


Figure 15. A snapshot from the real world demonstrating the robot reliably tracking various turning yaw commands q_{ψ}^d using the same controller used in Figure 14. Both the desired and actual turning angles are recorded in the lower part, with dashed lines matching the corresponding frames in the real world. The robot can execute full turns in both counterclockwise and clockwise directions.

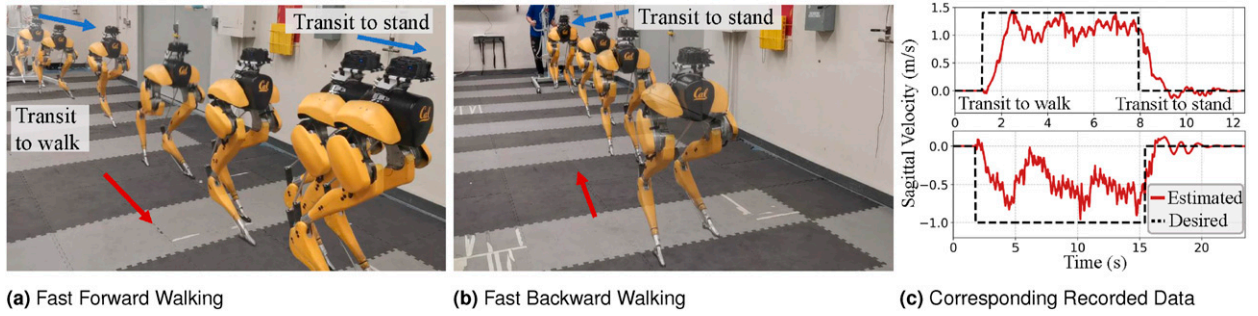


Figure 16. Snapshots of the robot transiting from standing to fast forward walking (Figure 16(a)) or backward walking (Figure 16(b)) and back to standing, using the same walking policy. Earlier frames appear more faded. The robot's commanded and actual sagittal velocities are recorded in Figure 16(c). This demonstrates the robot's capability to quickly switch from a stationary stance to dynamic fast walking and smoothly transition back to standing with a single command, even during dynamic maneuvers like fast walking. (a) Fast Forward Walking. (b) Fast Backward Walking. (c) Corresponding Recorded Data.

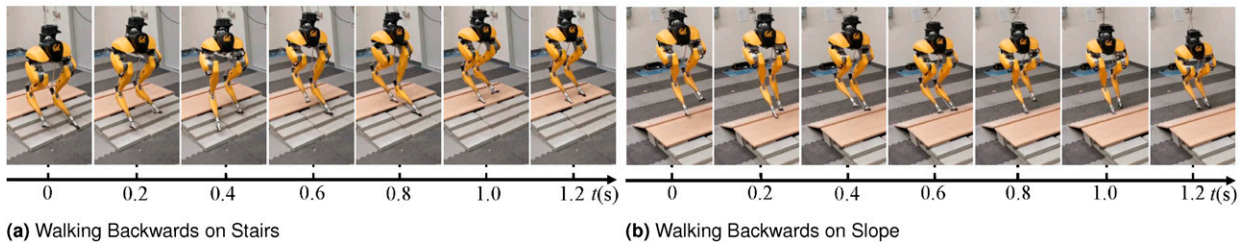


Figure 17. Snapshots of the robot Cassie performing a robustness test by walking *backwards* over small varied terrains such as stairs (Figure 17(a)) or slope (Figure 17(b)). Snapshots are aligned with time stamps. Although the policy wasn't specifically trained for terrain deviations, it successfully maintains stable walking gaits and robustness in the face of ground elevation changes in real-world scenarios as recorded in these set of snapshots. (a) Walking Backwards on Stairs. (b) Walking Backwards on Slope.

opposite lateral direction, effectively compensating for the perturbation and restoring its stable in-place walking gait.

During the test for persistent perturbation, a human exerts a persistent force on the robot base and drags the robot in random directions while the robot is commanded to walk in place. Examples can be seen in Figure 19(a), a persistent lateral dragging force is applied to Cassie's base while the robot is walking at normal height. In another test where the robot is walking at a lower height as

shown in Figure 19(b), the robot's base is subject to a persistent force with its direction changing randomly in the sagittal direction. During these tests, without losing balance, the robot shows compliance to these external forces by following the directions of these forces, despite being commanded to walk in place. These compliance experiments demonstrate the advantages of the proposed RL-based policy in controlling the bipedal robot for potential applications like safe human-robot interaction.

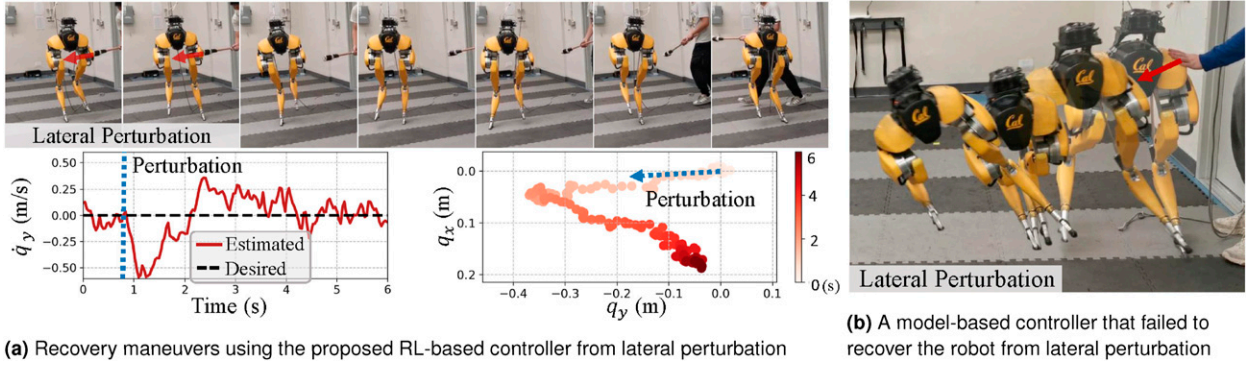


Figure 18. Robustness test of the robot walking against unknown perturbations. Using the proposed RL-based versatile policy, as seen in Figure 18(a), the robot, despite being pushed laterally and accelerated to -0.5 m/s, still maintains a stable walking gait and compensates such a lateral impulse by walking in the opposite direction. The corresponding lateral velocity $\dot{q}_y$ is recorded in the lower part of the figure. Additionally, the planar position (q_y, q_x) is estimated, with points appearing in progressively darker colors as they are recorded later in time. In contrast, using a model-based controller provided by Cassie’s manufacturer (Figure 18(b)), the robot crashes after a similar lateral push. (a) Recovery maneuvers using the proposed RL-based controller from lateral perturbation, (b) A model-based controller that failed to recover the robot from lateral perturbation.

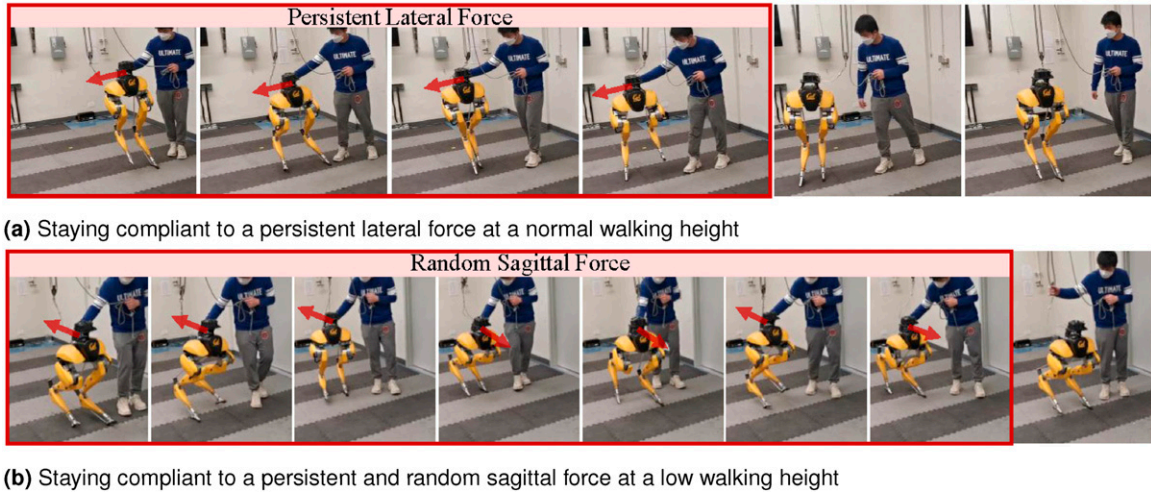


Figure 19. Snapshots of the compliance test using the proposed versatile walking policy in the real world. The red arrows illustrate the direction of the applied force. In Figure 19(a) where the robot is walking at a normal height, a human applies a persistent (longer than 3 seconds) lateral force on Cassie’s base and drags the robot to its right. In Figure 19(b) where the robot is at a low walking height, a human exerts a persistent but random force on the robot’s base and drags the robot back and forth in the sagittal direction. In these tests, the robot demonstrates compliance by walking along the persistent external force direction without losing balance, although being commanded to walk in place. After the force is removed, the robot returns to the commanded task. (a) Staying compliant to a persistent lateral force at a normal walking height, (b) Staying compliant to a persistent and random sagittal force at a low walking height.

More scenarios, including more impulse perturbations and persisting random perturbations, are recorded in the accompanying Vid. 3 in Table 1. In all conducted tests, the robot demonstrates its robustness, being consistent with the results observed in the simulation validation in Sec. 9. The underlying reason behind this lies in the robot’s extensive learning of various tasks such as lateral, forward, backward walking, turning, and more. This task randomization enables the robot to generalize the knowledge gained while learning from various tasks to recover from deviations occurring in the current commanded task.

10.1.2.3. Comparison with a model-based controller: We underline the advantages of the proposed walking policy by

comparing it with a model-based walking controller, specifically, a model-based controller utilized by the company that manufactured Cassie. As shown in Figure 18(b), when the robot is controlled by this model-based controller and subjected to lateral perturbation, this controller fails to maintain control, resulting in a crash. This is because that the model used in the controller does not consider the external perturbation (which is also hard to estimate), and the model-based controller cannot deal with such a large modeling error. The model-based controller also fails to stabilize the robot during a persistent perturbation test. These experiments are recorded in Vid. 3 in Table 1. A systematic perturbation test for legged robots was carried

out in a very recent work by Van Marum et al. (2024) and it would be interesting to incorporate such tests more widely. However, this is beyond the scope of this paper. Furthermore, as shown in the video, the model-based controller causes obvious lateral drifts on the robot (before the human operator perturbed it) when it is commanded to track a zero speed without manual gain tuning, while the proposed RL policy shows better tracking performance (that keeps the robot walking in-place) without training using real-world data. However, the model-based controller results in less and favorable stomping force than the RL policy. We hope these experiments could help the readers better understand the pros and cons of model-based optimal control and model-free RL methods for bipedal locomotion control.

10.1.3. Summary of results. In summary, the versatile walking policy derived from the proposed method is able to effectively control the bipedal robot Cassie to perform diverse tasks in the real world and be consistent over a long time of usage (more than a year). The results show that the robot can track different walking velocities, varying walking heights, turn in different directions, and perform fast walking and transition to and from standing. The policy also exhibits substantial robustness in the face of challenges such as changes in terrain elevation and external perturbations (including impulse and random but persistent forces).

10.2. Running experiments

We now evaluate the versatile running policies developed using the proposed methods in the real world. Three specific running policies were obtained: (1) a *general policy* trained on flat ground; (2) a *100-meter-dash finetuned policy*, trained after the convergence of the general policy with a focus on completing a 100-meter dash. This policy incorporated an additional termination condition to end the episode earlier if the robot could not complete the 100-meter dash within 25 seconds; (3) an *uneven-terrain policy*, developed after the convergence of the general policy, integrated with extra terrain randomization as listed in Table 4. All the policies have been trained with a variety of running and turning speeds as listed in Table 6 and transition to and from a standing skill. As we will see, using the obtained bipedal running policies, we achieved 400-meter dash within 2 min 34 s, 100-meter dash within 27.06 s, running inclines up to 10°, and more.

10.2.1. Running a 400-meter dash. We first evaluate the general running policy to complete a 400-meter dash on a standard outdoor running track, as demonstrated in Figure 20. Throughout this test, the robot was commanded to run at a consistent 3.5 m/s while responding to varying turning commands, given by a human operator to navigate the track. As shown in Figure 20(a) and the accompanying Vid. 2 in Table 1, the policy is able to smoothly transition from a standing position to a running gait (Figure 20(a)Ⓐ).

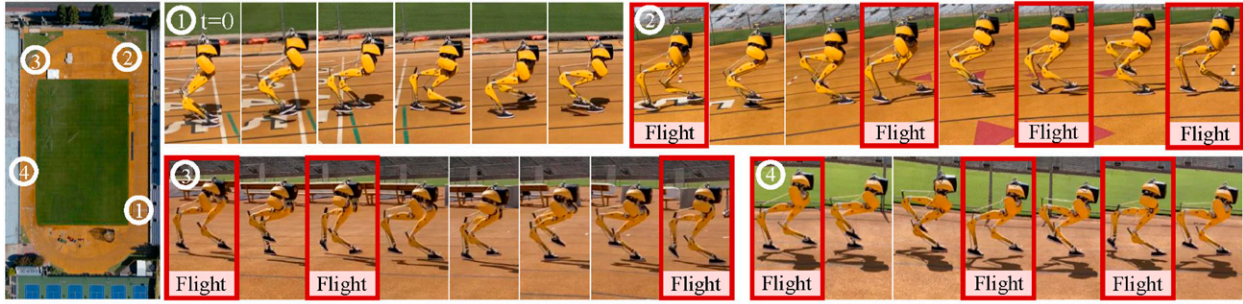
The robot managed to accelerate to an average *estimated* running speed of 2.15 m/s², reaching a peak estimated speed of 3.54 m/s, as recorded in Figure 20(b). The policy successfully maintained the desired speed consistently throughout the entire 400-meter run while accurately adhering to the varying turning commands. The average tracking error (MAE) in the turning angle q_ψ observed in Figure 20(b) is 5.95°. Throughout the 400m-dash, substantial flight phases were evident, as shown in Figure 20(a)Ⓑ–Ⓓ, despite the variable running speeds and turning angles. This consistent flight phase distinguishes our running gaits from a fast walking gait and presents significantly more challenges in control. However, the controller managed to sustain such a dynamic running gait stably over a significant duration (154 seconds).

Furthermore, during the 400-meter dash, although some lateral drift was detected, as shown in Figure 20(b), the robot can keep an average lateral velocity of 0.05 m/s. While the average lateral speed is relatively negligible, the robot did exhibit moments of having higher lateral speeds, exceeding 1 m/s at times, possibly due to irregular contact in the real-world environment. However, the policy's training in lateral running skills allowed the robot to correct these deviations, demonstrating the policy's robustness in maintaining stability even in the face of such unexpected lateral movements.

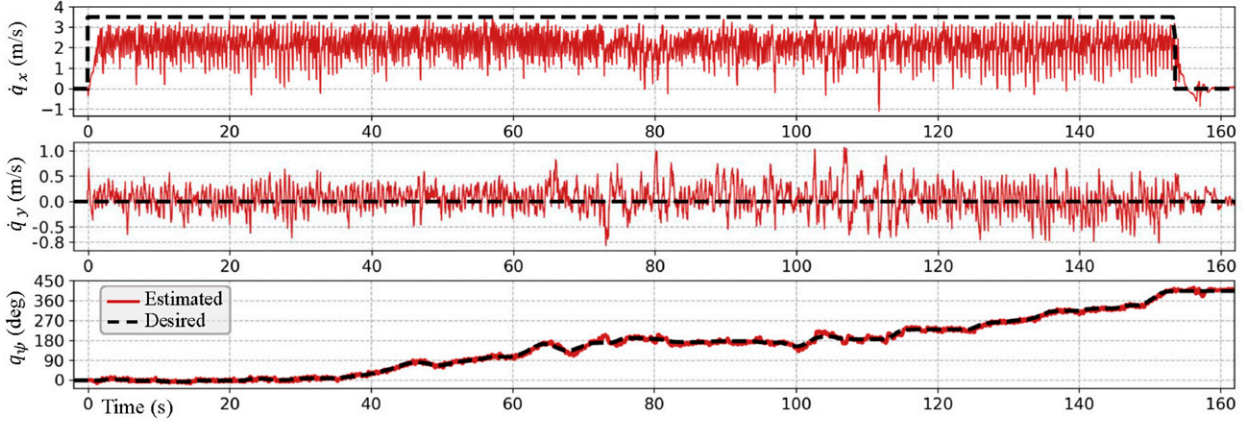
Cassie, controlled by the proposed running policy, successfully finished the 400-meter dash in 2 minutes and 34 seconds, and was able to transit to a standing pose afterward, as recorded in the video. This is a novel capacity of running a full 400-meter lap by a human-sized bipedal robot.

10.2.2. Tracking varying commands while running. Using the same versatile policy, the robot is also able to reliably track varying commands, such as sagittal velocity $\dot{q}_x$ shown in Figure 21(a) and lateral velocity $\dot{q}_y$, recorded in Figure 21(b), while running fast. The corresponding experiments are recorded in Vid. 4 in Table 1. Note that only one dimension of the command is changing during each of the tests conducted in Figure 20(b) (turning yaw q_ψ), 21a (sagittal velocity), 21b (lateral velocity). By comparing with the recorded logs among these tests, we observed that the command changing in one dimension will not affect the control performance on tracking a constant in other dimensions. This indicates that the dimensions on $\dot{q}_x$, $\dot{q}_y$, and q_ψ are *decoupled*, which is intriguing as this is realized in fast running control of a highly-nonlinear bipedal robot in the real world.

We further conduct a sharp turning test where the robot is given a step change of the yaw command, from 0 directly to 90°, as recorded in Figure 21(c). The robot can respond to such a step command and finish a 90-degree sharp turn within five steps in 2 seconds, using a natural running gait shown in Figure 21(c). Remarkably, the robot is not specifically trained for this sharp-turn scenario, as only smooth

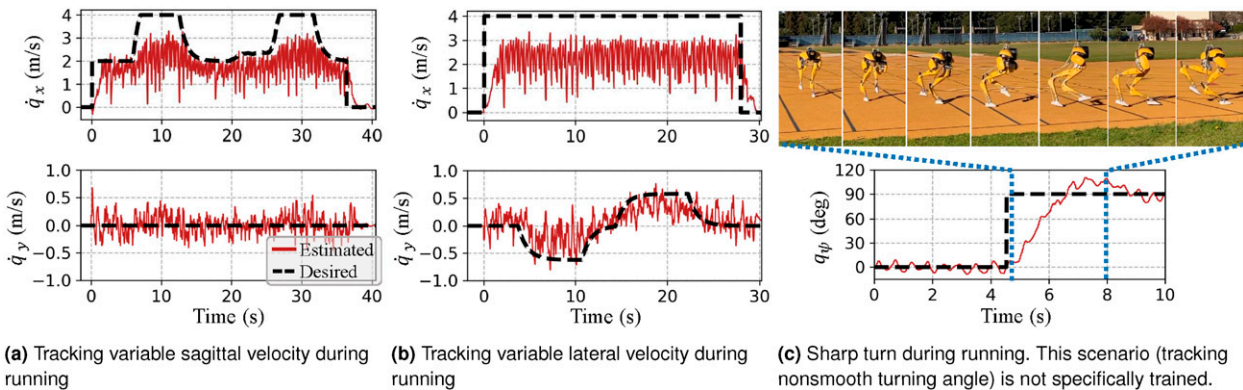


(a) Snapshot of 400-meter dash. The number matches the snapshot of the robot and the corresponding position on the running track.



(b) Recorded data during the 400-meter dash. The robot's sagittal velocity $\dot{q}_x$, lateral velocity $\dot{q}_y$ and turning yaw angle q_ψ are shown.

Figure 20. Utilizing a versatile running policy we developed, Cassie successfully completed a 400-meter dash in the bipedal robot regime with a time of 2 minutes and 34 seconds. The dash is completed in the Edwards Stadium at UC Berkeley. This is made possible using a single running policy that enabled the robot to transition from a standing pose to fast running gaits of 2.15 m/s at average and 3.54 m/s at peak. The robot also maintains that speed with an average lateral speed of 0.05 m/s (with zero lateral speed command) throughout the dash. Additionally, the robot is able to accurately follow varying turning commands with an average tracking error (MAE) of 5.95° . This precise turning command tracking was crucial for the successful completion of the 400-meter dash, as it required the robot to reliably turn while running. During running, we consistently observed noticeable flight phases as examples seen in Figure 20(a). The robot is also able to transit back to standing after it finished the dash, as recorded in the sagittal velocity log (Figure 20(b)) and Vid. 2 in Table 1. (a) Snapshot of 400-meter dash. The number matches the snapshot of the robot and the corresponding position on the running track, (b) Recorded data during the 400-meter dash. The robot's sagittal velocity $\dot{q}_x$, lateral velocity $\dot{q}_y$ and turning yaw angle q_ψ are shown.



(a) Tracking variable sagittal velocity during running

(b) Tracking variable lateral velocity during running

(c) Sharp turn during running. This scenario (tracking nonsmooth turning angle) is not specifically trained.

Figure 21. Real-world experiments on tracking variable commands while running using the same control policy that completed the 400 m-dash in Figure 20. Using the versatile policy, the robot is able to track varying sagittal velocity $\dot{q}_x$ (Figure 21(a)) and lateral velocity $\dot{q}_y$ (Figure 21(b)). Furthermore, the robot is able to perform a sharp turn while running, as demonstrated in Figure 21(c). The robot can turn to 90° within 2 seconds, using five steps, as recorded in the logs and corresponding snapshots in Figure 21(c). Notably, the robot is not specifically trained for this sharp-turn scenario during training. These experiments are also recorded in Vid. 4 in Table 1. (a) Tracking variable sagittal velocity during running, (b) Tracking variable lateral velocity during running, (c) Sharp turn during running. This scenario (tracking nonsmooth turning angle) is not specifically trained.

turning yaw angle command whose changing rate is bounded to 30 deg/s is given during training, as listed in Table 6. The versatile policy can generalize the learned various turning tasks and direct transfer to accomplish such a sharp-turn test. This further highlights the advantage of obtaining a versatile policy as previous work like Yu et al. (2022) requires training a separated policy to perform similar sharp-turn while walking.

10.2.3. Running a 100-meter dash. We now assess the running policy fine-tuned for the 100-meter dash, documented in Figure 22. We conducted the test 3 times, results of which are detailed in Table 5 and available in Vid. 1 and Vid. 4 in Table 1. With the deployment of the proposed running policy, the robot accomplished the 100-meter dash with around 28 seconds, achieving a fastest run time of 27.06 seconds. As depicted in Figure 22(a), the robot quickly transitioned from a stationary standing pose to a fast running gait within 1.8 seconds, using the transition maneuvers we commonly observed in human running athletes. During the cruising phase (Figure 22(b)), the robot maintained a rapid running gait and reached a peak estimated speed of 4.2 m/s, showing notable flight phases, as the data recorded in Figure 22(c).

10.2.4. Running on uneven terrains (trained). We proceed to examine the running policy fine-tuned for uneven terrains. As observed in Figure 23(a), the robot controlled by the proposed running policy managed to effectively traverse terrains with different slopes, all without the need for explicit terrain height estimation or external sensors. The terrain tests encompassed challenging variations, beginning with a 7° inclined slope in the sagittal plane, followed by a 3° slope in the lateral plane, a steeper 10° inclined slope in the sagittal plane, and concluding with a flat ground. Despite these difficult terrains, the robot maintained a stable running gait, as indicated by the converged limit cycles of the robot's thigh and knee joints ($q_{3,4}^{L/R}$) showcased in Figure 23(b) across these varied elevations. The robot's ability to change its gait to adapt to different terrains is derived from its usage of the robot's input/output history, implicitly encapsulating information about contact events and positioning. Notably, even while traversing these challenging terrains, the robot retained its flight phase, as showcased in Figure 23(a), signifying its ability to maintain a running gait, avoiding a degradation to walking gaits that offer more support but reduced speed. This is the first implementation of running (with flight phases) over large uneven terrains by a human-sized bipedal robot.

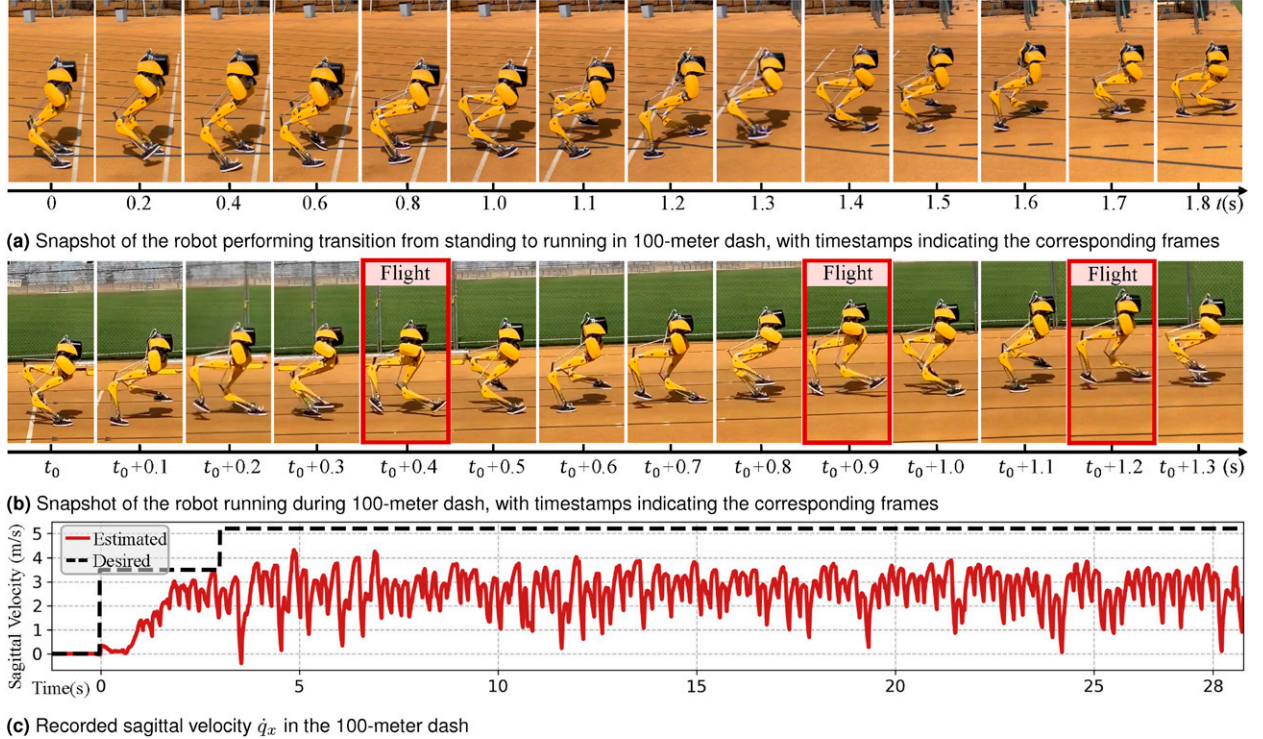


Figure 22. Using the finetuned running policy, Cassie finishes the 100-meter dash with a fast running gait. Figure 22(a) shows that the robot is able to transit from stationary stance to a rapid running gait, accelerating from 0 m/s to 3 m/s within 2 seconds with aggressive maneuvers. During the cruise phase shown in Figure 22(b), the robot maintains a fast yet stable running gait. The corresponding commanded and *estimated* sagittal velocity $\dot{q}_x$ is recorded in Figure 22(c). The robot finished this dash within 28 seconds, with an average speed of 3.57 m/s. (a) Snapshot of the robot performing transition from standing to running in 100-meter dash, with timestamps indicating the corresponding frames, (b) Snapshot of the robot running during 100-meter dash, with timestamps indicating the corresponding frames, (c) Recorded sagittal velocity $\dot{q}_x$ in the 100-meter dash.

10.2.5. Robust running maneuvers. During the real-world deployment, the robustness of the developed running policies in handling various perturbation scenarios was observed. For example, in Figure 24(a), the robot is controlled by its 100-meter-dash fine-tuned policy when an abrupt impulse perturbation force, produced by the safety cord, causes the robot to suddenly drop speed and lean to the left and twist. Despite this unforeseen event, the robot, trained on simulated perturbations and diverse running tasks including turning, is able to maintain stability and quickly recover back to a stable running gait. A similar test is also conducted using the general running policy while a lateral perturbation is applied to the robot, as recorded in Figure 24(b). Without losing balance, the robot exerts a lateral running gait to compensate such a lateral perturbation. These experiments are recorded in Vid. 4 in Table 1. These scenarios underline the robustness of the *versatile* running policies obtained by the proposed *task randomization*, especially beneficial when addressing unexpected disturbances while the robot is running at high speeds in real-world scenarios.

Table 5. Record of the completion time of 100-meter dash in three trials in the real world.

Trial	Completion time (s)
1	27.06
2	27.99
3	28.28

10.2.6. Summary of results. In summary, the running policies derived from the proposed method exhibit effective control over the bipedal robot Cassie in real-world scenarios. They showcase the capacity to manage various running and turning velocities, adapt to changes in terrain based solely on proprioceptive feedback, and seamlessly transition from and to a standing pose. Because of these running policies, Cassie achieves a notable peak speed of 4.2 m/s in the 100-meter dash which was completed within 27.06 seconds, accomplishes a 400-meter dash in 2 minutes 34 seconds, and traverses over uneven terrains, while showing robustness to unexpected distributions.

10.3. Jumping experiments

We now evaluate the proposed versatile jumping policies. We obtained two policies: (1) the *flat-ground* policy that specializes in jumping to different locations and executing turns on level ground ($e_z^d = 0$); and (2) *discrete-terrain* policy that focuses on leaping to various elevated platforms without executing turns ($q_\psi^d = 0$). The rationale behind having these two separate policies stems from the challenges posed by the dynamic jumping skill. In our empirical findings, we found it is difficult to train a robot to perform both jumping onto elevated platforms while turning simultaneously. As we will see, the proposed jumping policies achieve a large variety of different bipedal jumps including a 1.4-meter long jump and jumping onto a 0.44-meter elevated platform.

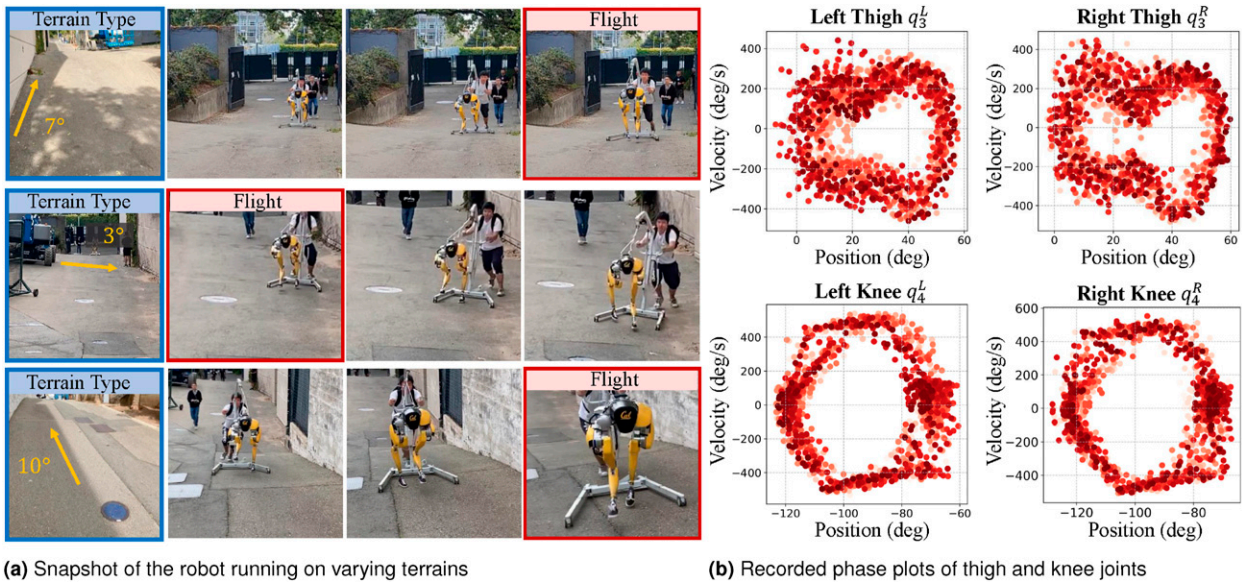


Figure 23. The robot runs over various terrains without any prior knowledge of the terrain specifics or elevation estimates. Figure 23(a) records the robot running on different types of terrains with different slopes, starting from 7° incline in the sagittal direction, followed by a 3° slope laterally, then a 10° sagittal incline, and finally transitioning to flat ground. The left-most column of the figure showcases these different terrain types. Despite the varying terrain, the robot consistently adapts and maintains a stable running gait with noticeable flight phases, as shown in the right columns of the figure. Figure 23(b) records the corresponding phase plots of the robot thigh and knee joints ($q_{3,4}^{L/R}$ vs $\dot{q}_{3,4}^{L/R}$, which play dominant roles in locomotion control), showing converged limit cycles. (a) Snapshot of the robot running on varying terrains. (b) Recorded phase plots of thigh and knee joints.

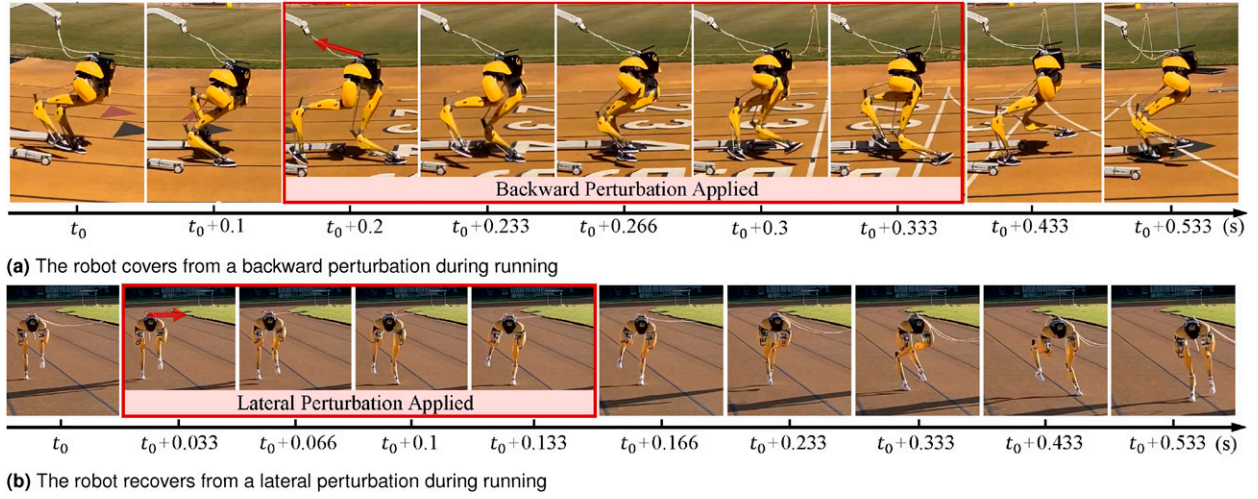


Figure 24. Snapshot that captures the moment when the robot, while running a 100-meter dash with a fast speed, is unexpectedly perturbed, demonstrating the robustness of the versatile running policy we developed. The frames are aligned with the corresponding timestamps. The robot experiences a sudden decrease in speed and begins to lean laterally along with a significant drift in its base yaw angle, due to being pulled backward by the safety cord. Despite these deviations from its nominal running trajectory, the robustness of the policy, which is enhanced by training with extensive dynamics randomization and versatile tasks (including slower-speed running with lateral and turning commands), enables the robot to maintain stability and correct itself back. (a) The robot covers from a backward perturbation during running. (b) The robot recovers from a lateral perturbation during running.

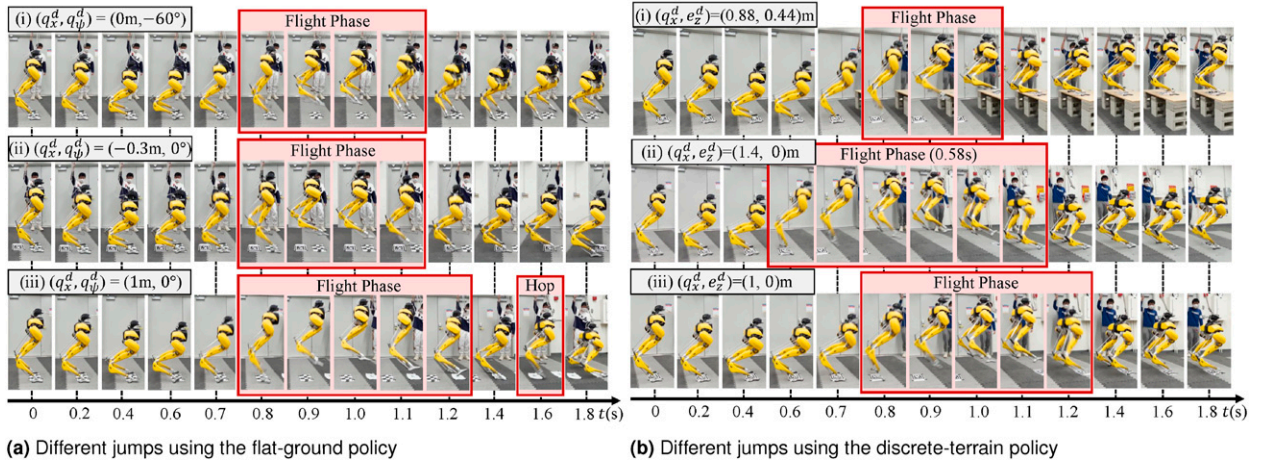


Figure 25. Snapshots of the bipedal robot Cassie performing various aggressive jumping maneuvers using two single versatile policies. Each frame is aligned with the corresponding timestamp, and paper tags in the figures represent the desired landing target. In Figure 25(a), using the *flat-ground* policy, (i) the robot is able to perform in-place jump while turning, (ii) backward jump, and (iii) forward jump, and land accurately on the target (paper tag). During the 1-meter ahead jump, the robot interestingly performs an extra forward hop after its initial landing, reaching the target on this second attempt. This additional maneuver was not observed during simulations that utilized the robot's nominal model. In Figure 25(b), using the *discrete-terrain* policy, (iii) the robot is also able to perform 1-meter forward jump. Moreover, (ii) the robot can perform a large standing long jump that lands accurately at 1.4-meter ahead, and (i) jump to a high platform that is elevated 0.44-meter above and 0.88-meter ahead. (a) Different jumps using the flat-ground policy. (b) Different jumps using the discrete-terrain policy.

10.3.1. Jump and turn. We first assess the performance of the *flat-ground* policy, as documented in Figure 25(a). Using this single jumping policy, the robot executes various given target jumps, such as jumping in place while turning 60° (Figure 25(a)(i)), jumping backward to land 0.3-meter behind (Figure 25(a)(ii)), and jumping to a target placed 1 meter ahead (Figure 25(a)(iii)), by just changing the

command to the policy. The robot is able to accomplish the task by landing precisely on the target which is marked as a paper tag on the ground. Upon closer observation, the robot can adjust its take-off pose in order to follow different commands. For instance, it leans backward when required to land on the rear target, and leans forward when executing a forward jump. The robot is also capable of executing a

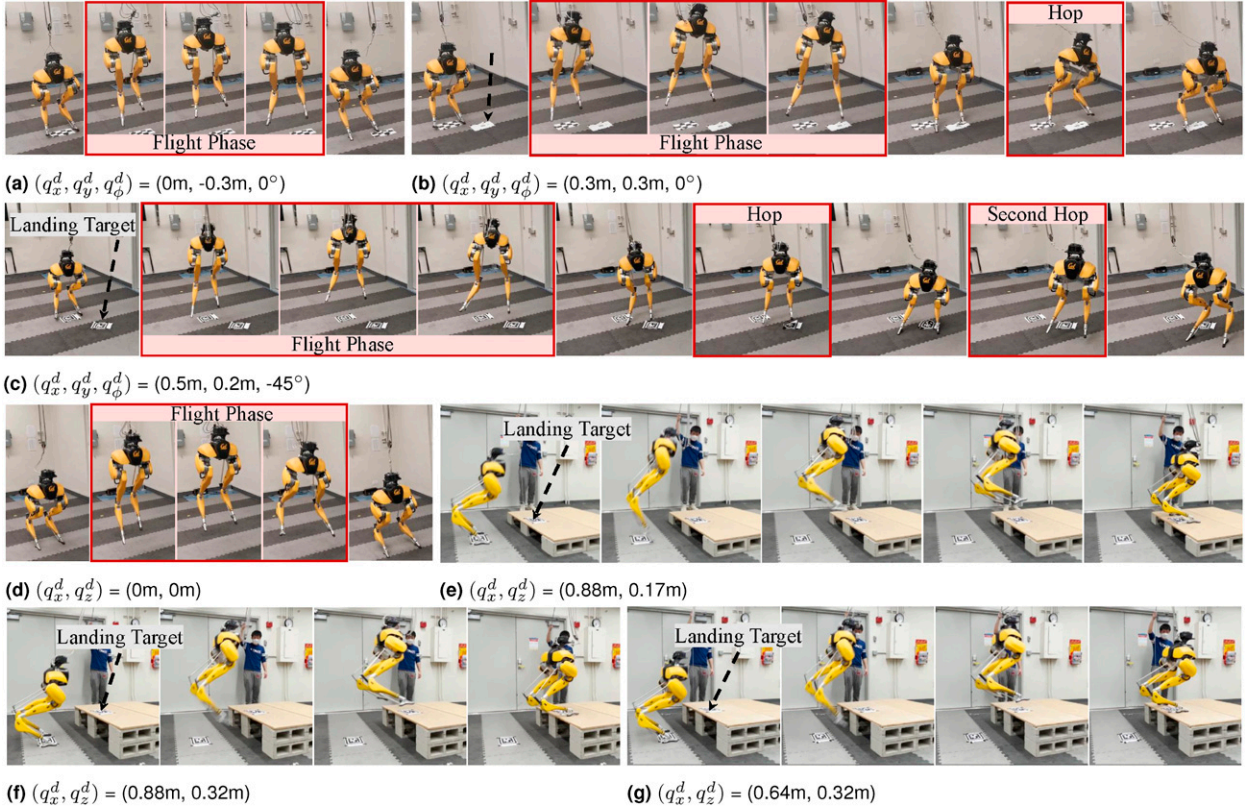


Figure 26. Snapshots of more diverse bipedal jumps using the proposed versatile jumping policies. The paper tag on the ground indicates the landing target. Using the same flat-ground policy used in Figure 25(a), the robot is also able to perform a lateral jump (0.3 m to its right in Figure 26(a)), diagonal jump (0.3 m ahead and 0.3 m to its left in Figure 26(b)), and a complex jumping maneuver that blends forward (0.5 m), lateral (0.2 m) and turning (-45°) at the same time (Figure 26(c)). Using the same discrete-terrain policy used in Figure 25(b), the robot can also jump in place (Figure 26(d)), jump to targets that are placed (Figure 26(e)) 0.88-meter ahead and 0.17-meter above, (Figure 26(f)) 0.88-meter ahead and 0.32-meter above, and (Figure 26(g)) 0.64-meter ahead and 0.32-meter above, respectively. (a) $(q_x^d, q_y^d, q_\phi^d) = (0\text{m}, -0.3\text{m}, 0^\circ)$, (b) $(q_x^d, q_y^d, q_\phi^d) = (0.3\text{m}, 0.3\text{m}, 0^\circ)$, (c) $(q_x^d, q_y^d, q_\phi^d) = (0.5\text{m}, 0.2\text{m}, -45^\circ)$, (d) $(q_x^d, q_z^d) = (0\text{m}, 0\text{m})$, (e) $(q_x^d, q_z^d) = (0.88\text{m}, 0.17\text{m})$, (f) $(q_x^d, q_z^d) = (0.88\text{m}, 0.32\text{m})$, (g) $(q_x^d, q_z^d) = (0.64\text{m}, 0.32\text{m})$.

more diverse set of jumps that combine movements along different axes. For instance, Figure 26(c) records a representative jump where the robot performs a multi-axes jump where it's required to jump forward, laterally, and make a turn at the same time. We report more diverse types of jumps (12 different jumps in total) in Appendix 10 and Vid. 5 in Table 1, and all of these jumps are realized by this single flat-ground jumping policy.

10.3.2. Jump to elevated platforms. We then evaluate the discrete-terrain policy to control the robot's jumping ability over various distances and elevations. As showcased in Figure 25(b), the robot exhibits the capability to jump precisely to targets placed at different positions, like 1 meter ahead (Figure 25(b)(iii)) or 1.4 meters ahead (Figure 25(b)(ii)), and at varying elevations, including 0.44 meters elevated (Figure 25(b)(i), considering the robot height is only 1.1 meter). Using the single discrete-terrain policy, the robot is able to adjust its take-off maneuvers for different jumping targets and efficiently manages its angular momentum after the large impact upon landing to maintain stability. In addition to standing long jumps (1.4 meters forward) and standing high jumps

(0.44 meters elevated), we conducted a range of varied jumping tasks within this range, including jumping in place (with foot clearance around 0.4 meters at its apex height) and jumping over different sagittal distances onto various elevated platforms. These tasks were all conducted using the same policy, as shown in Figure 26. Notably, the standing long jump over 1.4 meters and standing high jump to a 0.44-meter elevated platform (while using the same controller) are the novel locomotion capacities in the human-sized bipedal robot regime.

10.3.3. Robust jumping maneuvers. We illustrate the robustness of the jumping policy obtained from the proposed method with an example shown in Figure 27. Employing the flat-ground policy, we commanded the robot to perform an in-place jump. At the apex of the jump, we applied a backward impulse perturbation, as depicted in Figure 27(a). This significant perturbation caused the robot to deviate from its nominal in-place jumping trajectory (as seen in Figure 27(b)), resulting in a significant change in its pose upon landing. Consequently, the robot landed with a backward lean and its toes pitched up trying to stabilize the robot. This pose presented a significant challenge, given that the

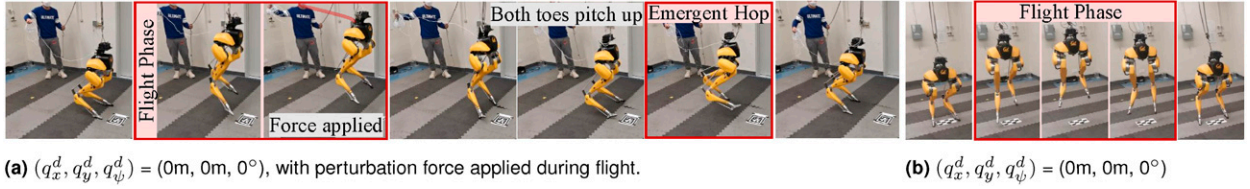


Figure 27. Snapshots of the robustness test during jumping using the versatile flat-ground policy. In Figure 27(a), an unexpected backward perturbation is applied while at its apex height during an in-place jump. This contrasts with Figure 27(b), where the robot performs the same task without perturbation, serving as a baseline. Despite leaning backward upon landing due to the perturbation, the robot spontaneously executes a backward hop, a maneuver it learned during other tasks like backward jumping, and succeeds to maintain balance after landing. Such a robustness is achieved through *task randomization* alone, as the jumping policy was not trained with simulated perturbation. (a) $(q_x^d, q_y^d, q_\psi^d) = (0m, 0m, 0^\circ)$, with perturbation force applied during flight, (b) $(q_x^d, q_y^d, q_\psi^d) = (0m, 0m, 0^\circ)$.

robot became underactuated and was nearly losing balance. However, since the jumping policy has been trained to execute backward jumps, as previously showcased in Figure 25(a)(ii), the robot rapidly adjusted its intended landing trajectory and executed a backward hop. By this adjustment, the robot is able to better correct its pose in the air to achieve a more favorable configuration upon its next landing.

Note that the jumping policies were never trained explicitly to handle perturbations. Therefore, this experiment highlights the advantage of a versatile jumping policy. The robot demonstrates the ability to generalize its learned diverse tasks to devise a better maneuver and contact plan rather than strictly adhering to the given task during real-world deployment. This serves as a detailed report of successful recovery from a perturbation while a bipedal robot is executing a jump in the real world, illustrating the importance of the *task randomization*.

Remark 3. *Using versatile policies for walking, running, and jumping, the robot exhibits the capability to develop its own contact strategy online, deviating from the contact plans (implicitly) provided by reference motions. This ability enhances stability and robustness. For instance, in jumping experiments, the robot often breaks contact after landing and executes small hops to achieve a better landing configuration, as seen in Figure 27. This capacity is also evident in other skills, such as standing (Figure 11(c)), walking (with significant double-support phases contrary to the single-support in reference motions, as in Figure 17(b)), and running (where the robot does not strictly follow the contact sequence of the periodic running reference motion, as seen in Figure 22). This emergent capability aligns with what contact-implicit optimization, like in Posa et al. (2014); Drnatch and Zhao (2021); Landry et al. (2022), aims to achieve. While these prior methods have only achieved such optimization offline for bipedal robots, our work realizes this online on a real bipedal robot.*

Remark 4. *We note that during the jumping experiments, the robot occasionally oscillates while standing after a large jump. This is due to the challenges of learning a single RL policy to learn both dynamic aperiodic jumping*

skill and stationary standing skill. The features learned during a jump may favor high-acceleration motion and cannot easily alter the policy's behavior for perfectly stationary standing. This also provides an insight into the challenges of obtaining a single unified policy to learn all different locomotion skills (to combine dynamic and stationary skills). But we want to note that, the jumping policy is still able to damp out such an oscillation during standing, though with a longer noticeable time, as demonstrated in the supplementary video (Vid. 5).

10.3.4. Summary of results. We demonstrated 19 distinct bipedal jumps in the real world, encompassing a range of landing locations, turns, and elevations. These jumps were conducted using just two versatile policies: the *flat-ground* policy and the *discrete-terrain* policy. Through these extensive real-world experiments, we illustrated two key aspects of the proposed method, using the jumping skill as an exemplary case. First, we showcase the *adaptivity* of the proposed policy. The robot exhibits the capability to land precisely on a designated target after a flight phase, all without requiring global position feedback. To achieve this, the robot must generate a precise amount of momentum at take-off. By accomplishing these diverse jumping tasks, the RL-based control policy showcases its adaptivity to the robot's hardware dynamics, mainly by leveraging the long-term I/O history, to control the robot and achieve accurate take-off translational velocities to land at the target. Second, we highlight the *robustness* of the policy. Despite being trained without perturbation, the robot demonstrates an ability to respond to unexpected perturbations during jumping by employing learned tasks for agile recovery.

11. How to train your bipedal robot: A discussion

After detailing our methodology, analysis, and experiments on robust, dynamic, and versatile control policies for a spectrum of bipedal locomotion skills via RL, we now turn to a discussion of the key lessons learned throughout this development. Our objective is to provide valuable and generally applicable insights that could steer future research

in the use of RL for locomotion control in complex dynamic systems, particularly in bipedal and humanoid robots.

The following section is structured as follows: (1) We first discuss the importance of utilizing the robot's I/O history, and how to effectively use it (by using a dual-history policy to learn direct adaptive control) in Sec. 11.1. (2) In Sec. 11.2, we show that while properly using robot's I/O history empowers the controller with adaptivity, the robustness of the controller can be improved by task randomization. (3) In Sec. 11.3, we provide several "bonuses" that RL can bring forth, such as motion inference and optimization, as well as contact planning. We wrap up this discussion with a small debate on general versus task-specific control policies.

11.1. How to use the Robot's history?

11.1.1. Robot's long I/O history is important. In existing RL-based locomotion control literature, there is no convergence on how to use the robot's history: short or long, history of only state feedback or of state and action pair. Our research demonstrates the clear benefits of providing a long history of both robot's inputs and outputs, especially in the case of high-dimensional highly nonlinear systems like bipedal robots. This approach is grounded in the robot's full-order dynamics (1), suggesting that a sequence of I/O history can be more informative in identifying the dynamic system to control. This assertion is supported by our ablation study (Figure 6), where policies incorporating a longer I/O history outperformed those with shorter history or states-only long history in handling randomized dynamics parameters in realizing different locomotion skills. Additionally, analyzing the latent embedding from the long I/O history encoder (Figure 9) revealed that it could implicitly capture information such as contact events, external forces, and changes in system dynamics parameters, and more. This finding is intriguing, as it shows that optimization by model-free RL, with a proper utilization of robot's I/O, can learn to extract vital information for dynamic control.

11.1.2. Short history complements long history. While a long I/O history can improve control performance, its effectiveness is limited without proper formulation. Our ablation study (Figure 6) shows that using only a long I/O history does not outperform policies with a short history, aligned with the report from Singh et al. (2023). We discover that to fully exploit the benefits of long history, it's effective to provide a short I/O history directly to the base MLP, bypassing the long history encoder, termed as *dual history* approach. This modification markedly improves learning performance across various locomotion skills (Figure 6) and outperforms other approaches in real-world experiments (Figure 7). While long history has been introduced in RL-based locomotion control in prior work, the complementary use of short history is a novel approach. Many previous efforts lack this addition to the base policy,

potentially overlooking a critical element that is the *recent* I/O history for controlling complex robots in *real time*. When only using long I/O history, recent events may become obfuscated in the latent embedding after the long-history encoding, and this can be addressed by incorporating an *explicit* short I/O history alongside the long-history encoder. This explicit recent I/O feedback can also help with the non-Markovian property when we opt to use a long robot I/O history. Furthermore, while the long history is better for system identification and state estimation, the short-term features are better for denoising the high-frequency estimates from the long-history encoder. Although this approach might not be essential for simpler systems like stationary robotic arms (Peng et al., 2018) or quadrupedal robots (Lee et al., 2020) as observed in prior studies, for more complex dynamic systems like the bipedal robots in our work, the benefits of this dual history approach are clearly pronounced. Additionally, while the dual-history approach could boost the learning using non-recurrent policy that is trained to leverage the history sequence with an explicit length, we found it is hard to aid the training of recurrent policy in terms of the bipedal locomotion control. Details are presented in Appendix 6.

11.1.3. Are we learning a robust or adaptive controller?. Adaptivity in RL is not a new concept, and there are notable efforts from the control theory community to bridge adaptive control with RL, like Annaswamy (2023). On the other hand, many studies on RL-based locomotion control have demonstrated robustness in real-world applications, attributed by some to the robustness of the policy trained with dynamics randomization and by others to the adaptivity in RL. Many in this community are curious about which attribute actually plays a critical role. This curiosity motivated us to conduct an extensive empirical study in this work to investigate both the adaptivity and robustness of the RL policy. We show that in the RL-based controller, these two aspects can originate from different sources: adaptivity can arise from the proper use of the robot's I/O history, while robustness can be introduced by task randomization. Dynamics randomization occupies a middle ground: in order to be robust to different dynamics, the policy can be trained with randomized dynamics to learn to effectively utilize the history (if provided) in adapting to the dynamics shifts.

11.1.4. What do we want to learn, direct or indirect adaptive control?. Recognizing the value of properly employing long I/O history, we provide a discussion about an alternative method: using the history to estimate selected environmental parameters for control. This approach, known as the Teacher-Student (TS) training strategy (Lee et al., 2020) or RMA (Kumar et al., 2021), can be seen as an RL-version of *indirect* adaptive control. However, for real-time dynamic locomotion control, our findings suggest that a *direct* adaptive control method, which integrates long I/O history directly into the controller without explicitly estimating model parameters, yields better performance, supported by the results in Sec. 7. The underlying reason behind

this becomes clearer when considering the capabilities of the history encoder (Figure 9) obtained by our proposed method. It not only adapts to time-invariant modeling parameters but also captures crucial information deemed important by the robot, like time-variant contact events. This contrasts with TS or RMA methods, which are limited to estimating modeling parameters and might encounter estimation errors or failures in challenging tasks like bipedal running (Figure 6). Compared to TS and RMA which separate training for the base control policy and history encoder, end-to-end training without expert supervision gives the robot more freedom to explore and exploit useful information.

11.2. Versatility improves robustness

Besides the adaptivity brought by the proper use of the robot's long I/O history, another key benefit of using RL for locomotion control is robustness. When policies are trained with *dynamics randomization*, they can stay robust to environmental changes, like during the sim-to-real transfer. However, our research reveals that the robustness of RL policies extends beyond this, with *task randomization* emerging as another key strategy.

Our ablation study (Figure 10) supports this. While dynamics randomization expands the range of trajectories a robot is trained on within a specific task, it doesn't dramatically alter the training distribution. For instance, extensive randomization in a standing policy doesn't equip the robot with new locomotion skills like walking or jumping for recovery (Figures 11 and 12). Task randomization, however, imparts robustness differently. By training across diverse tasks, such as varying walking speeds or jumping targets, the robot learns to generalize and exploit these learned tasks, aiding recovery even without specific dynamics randomization, as demonstrated in Figure 10. Interestingly, the robot's response to disturbances differs based on the source of robustness. With extensive dynamics randomization, it tends to adhere to the commanded task, whereas using a versatile policy trained with task randomization, it shows more *compliance* to external disturbances, deviating from its given command.

Although some previous RL-based locomotion research unintentionally achieved versatile policies and attributed their success of robustness primarily to dynamics randomization, our study distinguishes the sources of robustness. We recommend additional task randomization in future RL applications for robust locomotion control.

11.3. Exploring what RL can enable for legged locomotion

In this subsection, we aim to offer broader insights into the potential (and limitations) of RL for controlling complex dynamic systems.

11.3.1. Trajectory optimization and motion inference. By using RL for controlling high-dimensional bipedal robots,

reference motions are often used to achieve natural gaits, as demonstrated in our work. While there are prior attempts using trajectory optimization based on robot dynamics model to obtain a reference motion, like Bogdanovic et al. (2022) and the walking policy in this work, we also show that such a "pre-optimization" could be unnecessary for RL-based control. For example, our work with bipedal running and jumping, using only kinematically feasible retargeted human motion and animation, shows that robots can learn to stay close to reference motions and maintain dynamic stability through trial and error, as long as the action space is not corrupted by the added reference motion (as the residual approach in Figure 6). This indicates the ability of RL to jointly learn trajectory optimization and real-time control. Furthermore, our work demonstrates that RL also enables robots to infer motions beyond the provided references. For example, the robot can learn to vary running or jumping maneuvers from single reference motions, achieved by task randomization and goal-conditioned policy. However, using a single reference motion has its limits. For instance, if we only provide a reference forward jumping motion, it becomes challenging for the robot to learn a backward jump based solely on task completion reward. Therefore, selecting an effective single reference motion may require it to be relatively unbiased, such as the jumping-in-place motion used in this work.

11.3.2. Motion (and contact) planning. In our real-world experiments (Sec. 10), we observed numerous instances where the robot demonstrated the ability to perform a sequence of varied motions with varying contact sequences for recovery. For example, when perturbed from a standing position, the robot exerted a variety of different walking gaits in order to return to a stand, all without requiring an online motion or contact sequence scheduler (Figure 12(a)). This reflects a longer-horizon motion planning capability, distinct from its real-time locomotion control occurring at 33 Hz. This is further supported by jumping experiments like Figures 26(c) and 27(a). These observations indicate that RL policies can autonomously develop contact sequences for different tasks during deployment when provided with the unvaried reference motion. This capability also suggests that specifying a contact sequence to the policy can be less advantageous, as the (legged) robot will have less freedom to explore more optimal contact patterns for enhanced stability and robustness. *In short, the advantage of utilizing RL for legged locomotion control where contact is crucial is in the ability to control without explicitly considering contact.*

11.3.3. The possibility of learning a unified control policy. Given that all the policies developed in this work use the same control architecture and training procedure, it is possible to obtain a unified policy for different skills. This possibility is further supported given that in all dynamic walking, running, and jumping policies, we have combined them with a distinguished skill: the stationary standing. For

example, the policy that realized the transition among running and standing skills shows the potential to learn different skills (such as slower speed walking) within this spectrum. Adding the standing skill with the other is realized by using a carefully designed MDP and an additional substage for the new skill training as introduced in [Appendix 2](#). However, if using this relatively simple method, it is challenging to keep adding different new skills as the robot may suffer from the catastrophic forgetting problem.

Using an adversarial motion prior (AMP) as in [Peng et al. \(2021\)](#) could potentially help to obtain a unified control policy for diverse locomotion skills without specifically tuning motion tracking rewards. However, applying AMP to aggressive real-world bipedal locomotion remains a challenge. GAN-styled methods are prone to mode-collapse, and can struggle to imitate aggressive motions that occur over brief periods. The actor policy can “cheat” the discriminator by mimicking a part of a jump, without performing the motion in its entirety. This is further compounded by the large sim-to-real gap for bipedal robots.

Another potential way to develop such a unified policy can be done by continual RL to keep learning new skills like [Liu et al. \(2023\)](#) or imitation learning from offline datasets collected by skill-specific policies like [Huang et al. \(2024\)](#). However, increasing the robustness and overcoming the sim-to-real gap using these methods for bipedal robots is still an open question. A practical way could be learning to transition among different pre-trained skill-specific policies, such as [Lee et al. \(2018\)](#). In either way, this proposed method for skill-specific policies in this work serves as a solid starting point.

11.3.4. Generalization versus precision. In our research, we have showcased many examples of RL’s generalization capabilities in dynamic locomotion control, spanning various tasks and dynamics parameters. While control precision has not been a primary focus, it becomes evident in skills like bipedal jumping. In these tasks, the robot successfully jumps to specified targets (Sec. 10.3), requiring precise translational velocity at take-off for accurate landing. However, attaining perfect precise control using one policy that handles a wide variety of tasks and dynamics variations remains an open question. For example, it is challenging to track a specific sagittal velocity with minor errors in fast-running tasks in the real world ([Figure 22\(c\)](#)). Yet, the benefits of a generalized locomotion control policy, like foundation models, lie in providing a solid starting point for further fine-tuning on specific downstream tasks, like precision control. On the flip side, a controller optimized for precision might be restricted to the finetuned task. We present these trade-offs for readers to consider, highlighting the balance between generalization and precision in RL-based dynamic locomotion control.

12. Conclusion and future work

In conclusion, this work presents a comprehensive study on using deep reinforcement learning to develop versatile,

robust, and dynamic locomotion controllers for bipedal robots. This work introduces a dual-history approach that integrates the robot’s input/output (I/O) history into RL-based controllers and highlights its significance. We demonstrate the adaptivity and robustness of the proposed RL-based controller, particularly underscoring how a well-designed long I/O history encoder can adapt to both time-invariant dynamics changes and time-variant events such as varying contacts. Additionally, task randomization, which encourages the robot to explore a broader range of scenarios by accomplishing different tasks, significantly enhances robustness, complementing the robustness achieved through traditional dynamics randomization. The proposed method is validated thoroughly on the bipedal robot Cassie, successfully realizing versatile and robust walking, running, and jumping skills in the real world. These experiments include several state-of-the-art results, including walking control with consistent performance over a long timespan (459 days), versatile running capabilities demonstrated in a 400-meter dash and across challenging terrains, and a large repertoire of jumping tasks including the furthest 1.4-meter forward jump and 0.44-meter high jump.

In the future, we hope this method can be extended to humanoid robots that can also leverage upper-body motions for agility and stability. Additionally, integrating depth vision directly into the locomotion controller can be realized in a straightforward way where an additional depth encoder can work alongside the robot’s I/O history encoder in our proposed control architecture. We encourage the readers to also try this setup for visual control on their bipedal robots. With the advancements made in this work, we think a large portion of practical problems in realizing effective locomotion control for human-sized bipedal robots can be addressed. For the bipedal robotics community, an exciting direction would be the combination of bipedal locomotion and bimanual manipulation to tackle long-horizon locomanipulation tasks, opening new possibilities in the field.

Acknowledgements

The authors thank Xuxin Cheng for the help in the initial development of this work. The authors thank Daniel Wong for the design of running shoes for Cassie. The authors would also like to thank Bike Zhang, Jiaming Chen, Lizhi Yang, Xiaoyu Huang, Yiming Ni, Yufeng Chi, Akshay Thirugnanam, Yuanzhuo Li, Dr Jun Zeng, and Dr Ayush Agrawal for their help with the experiments.

Declaration of conflicting interests

The author(s) declared no potential conflict of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was in part supported by NSF Grant CMMI-1944722,

The AI Institute, and Canadian Institute for Advanced Research (CIFAR).

ORCID iD

Zhongyu Li  <https://orcid.org/0000-0001-8040-1098>

Supplemental Material

Supplemental material for this article is available online.

Notes

1. While there are attempts from industry (like Boston Dynamics as patented in [Deits et al. \(2022\)](#)) using model-based OC that tackled similar problems, detailed reports are limited. As a research paper, our focus is on published findings.
2. We note that there is a nontrivial state estimation error on the sagittal velocity when the robot is running at a high speed. The average speed of finishing 400 meter in 154 seconds is 2.6 m/s while the estimated average speed is 2.15 m/s. Such an error is more obvious in the 100-meter dash discussed later. Therefore, we emphasize the logs are estimated values and not true values. We further elaborate this in [Appendix 9](#).

References

- Agrawal A (2022) *Model-based design for legged robots: predictive control and reinforcement learning*. PhD Thesis. Berkeley: UC Berkeley.
- Annaswamy AM (2023) Adaptive control and intersections with reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems* 6: 65–93.
- Bogdanovic M, Khadiv M and Righetti L (2022) Model-free reinforcement learning for robust locomotion using demonstrations from trajectory optimization. *Frontiers in Robotics and AI* 9: 854212.
- Boroujeni MG, Daneshmand E, Righetti L, et al. (2021) A unified framework for walking and running of bipedal robots. In: 2021 20th International Conference on Advanced Robotics (ICAR), Ljubljana, Slovenia, 06–10 December 2021, pp. 396–403, IEEE.
- Bouyarmane K and Kheddar A (2011) Using a multi-objective controller to synthesize simulated humanoid robot motion with changing contact configurations. In: 2011 IEEE/RSJ International Conference on Intelligent Robots and Systems, San Francisco, CA, USA, 25–30 September 2011, pp. 4414–4419, IEEE.
- Caron S, Kheddar A and Tempier O (2019) Stair climbing stabilization of the hrp-4 humanoid robot using whole-body admittance control. In: 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019, pp. 277–283, IEEE.
- Castillo GA, Weng B, Zhang W, et al. (2022) Reinforcement learning-based cascade motion policy design for robust 3d bipedal locomotion. *IEEE Access* 10: 20135–20148.
- Chen S, Zhang B, Mueller MW, et al. (2023) Learning torque control for quadrupedal locomotion. In: 2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids), Austin, TX, USA, 12–14 December 2023, pp. 1–8.
- Cheng X, Kumar A and Pathak D (2023) Legs as manipulator: pushing quadrupedal agility beyond locomotion. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), 29 May–02 June 2023, London, UK, pp. 5106–5112.
- Chignoli M, Kim D, Stanger-Jones E, et al. (2021) The mit humanoid robot: Design, motion planning, and control for acrobatic behaviors. In: 2020 IEEE-RAS 20th International Conference on Humanoid Robots (Humanoids), Munich, Germany, 19–21 July 2021, pp. 1–8, IEEE.
- Crowley D, Dao J, Duan H, et al. (2023) Optimizing bipedal locomotion for the 100m dash with comparison to human running. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), London, UK, 29 May–02 June 2023, 12205–12211, IEEE.
- Da X, Harib O, Hartley R, et al. (2016) From 2d design of underactuated bipedal gaits to 3d implementation: walking with speed tracking. *IEEE Access* 4: 3469–3478.
- Dai H, Valenzuela A and Tedrake R (2014) Whole-body motion planning with centroidal dynamics and full kinematics. In: 2014 IEEE-RAS International Conference on Humanoid Robots, Madrid, Spain, 18–20 November 2014, pp. 295–302, IEEE.
- Daneshmand E, Khadiv M, Grimminger F, et al. (2021) Variable horizon mpc with swing foot dynamics for bipedal walking control. *IEEE Robotics and Automation Letters* 6(2): 2349–2356.
- Dao J, Green K, Duan H, et al. (2022) Sim-to-real learning for bipedal locomotion under unsensed dynamic loads. In: 2022 International Conference on Robotics and Automation (ICRA), Philadelphia, PA, USA, 23–27 May 2022, pp. 10449–10455, IEEE.
- Deits R and Tedrake R (2014) Footstep planning on uneven terrain with mixed-integer convex optimization. In: 2014 IEEE-RAS International Conference on Humanoid Robots, Madrid, Spain, 18–20 November 2014, pp. 279–286, IEEE.
- Deits R, Kuindersma S, Kelly MP, et al. (2022) Robot movement and online trajectory optimization. US Patent App. 17/358,628.
- DRL (2023) cassie-mujoco-sim. <https://github.com/osudrl/cassie-mujoco-sim>.
- Drnack L and Zhao Y (2021) Robust trajectory optimization over uncertain terrain with stochastic complementarity. *IEEE Robotics and Automation Letters* 6(2): 1168–1175.
- Escontrela A, Peng XB, Yu W, et al. (2022) Adversarial motion priors make good substitutes for complex reward functions. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Kyoto, Japan, 23–27 October 2022, pp. 25–32, IEEE.
- Feng G, Zhang H, Li Z, et al. (2023) Genloco: generalized locomotion controllers for quadrupedal robots. In: Conference on Robot Learning, Auckland, New Zealand, 14 December 2022, pp. 1893–1903, PMLR.
- Fernbach P, Tonneau S, Stasse O, et al. (2020) C-croc: continuous and convex resolution of centroidal dynamic trajectories for legged robots in multicontact scenarios. *IEEE Transactions on Robotics* 36(3): 676–691.
- Fevre M, Wensing PM and Schmiedeler JP (2020) Rapid bipedal gait optimization in casadi. In: 2020 IEEE/RSJ International

- Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, 24 October 2020–24 January 2021, pp. 3672–3678, IEEE.
- Fu Z, Kumar A, Malik J, et al. (2021) Minimizing energy consumption leads to the emergence of gaits in legged robots. In: 5th Annual Conference on Robot Learning, London, UK, 08 November 2021.
- Fu Z, Cheng X and Pathak D (2023) Deep whole-body control: learning a unified policy for manipulation and locomotion. In: Conference on Robot Learning, Auckland, New Zealand, 14 December 2022, pp. 138–149, PMLR.
- Gong Y and Grizzle JW (2022) Zero dynamics, pendulum models, and angular momentum in feedback control of bipedal locomotion. *Journal of Dynamic Systems, Measurement, and Control* 144(12): 121006.
- Gong Y, Hartley R, Da X, et al. (2019) Feedback control of a cassie bipedal robot: walking, standing, and riding a segway. In: 2019 American Control Conference (ACC), Philadelphia, PA, USA, 10–12 July 2019, pp. 4559–4566, IEEE.
- Goswami D and Vadakkepat P (2009) Planar bipedal jumping gaits with stable landing. *IEEE Transactions on Robotics* 25(5): 1030–1046.
- Haarnoja T, Ha S, Zhou A, et al. (2019) Learning to walk via deep reinforcement learning. In: Robotics: Science and Systems (RSS), Freiburg im Breisgau, 22–26 June 2019.
- Hereid A, Hubicki CM, Cousineau EA, et al. (2018) Dynamic humanoid locomotion: a scalable formulation for hzd gait optimization. *IEEE Transactions on Robotics* 34(2): 370–387.
- Hereid A, Harib O, Hartley R, et al. (2019) Rapid trajectory optimization using c-frost with illustration on a cassie-series dynamic walking biped. In: 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Macau, China, 03–08 November 2019. pp. 4722–4729.
- Huang X, Li Z, Xiang Y, et al. (2023) Creating a dynamic quadrupedal robotic goalkeeper with reinforcement learning. In: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 1–5 October 2023, pp. 2715–2722.
- Huang X, Chi Y, Wang R, et al. (2024) Diffuselo: real-time legged locomotion control with diffusion from offline datasets. *arXiv preprint arXiv:2404.19264*.
- Hwangbo J, Lee J, Dosovitskiy A, et al. (2019) Learning agile and dynamic motor skills for legged robots. *Science Robotics* 4(26): eaau5872.
- Ibanez A, Bidaud P and Padois V (2014) Emergence of humanoid walking behaviors from mixed-integer model predictive control. In: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, Chicago, IL, USA, 14–18 September 2014, pp. 4014–4021, IEEE.
- Ji G, Mun J, Kim H, et al. (2022) Concurrent training of a control policy and a state estimator for dynamic and robust legged locomotion. *IEEE Robotics and Automation Letters* 7(2): 4630–4637.
- Kajita S, Kanehiro F, Kaneko K, et al. (2001) The 3d linear inverted pendulum mode: a simple modeling for a biped walking pattern generation. In: Proceedings 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems. Expanding the Societal Role of Robotics in the the Next Millennium (Cat. No. 01CH37180), volume 1, Maui, HI, USA, 29 October–03 November 2001, pp. 239–246, IEEE.
- Kalashnikov D, Varley J, Chebotar Y, et al. (2021) Mt-opt: continuous multi-task robotic reinforcement learning at scale. *arXiv preprint arXiv:2104.08212*.
- Kim D, Berseth G, Schwartz M, et al. (2023) Torque-based deep reinforcement learning for task-and-robot agnostic learning on bipedal robots using sim-to-real transfer. *IEEE Robotics and Automation Letters* 8: 6251.
- Kojima K, Kojio Y, Ishikawa T, et al. (2019) A robot design method for weight saving aimed at dynamic motions: design of humanoid jaxon3-p and realization of jump motions. In: 2019 IEEE-RAS 19th International Conference on Humanoid Robots (Humanoids), Toronto, ON, Canada, 15–17 October 2019, pp. 586–593, IEEE.
- Kuindersma S, Permenter F and Tedrake R (2014) An efficiently solvable quadratic program for stabilizing dynamic locomotion. In: 2014 IEEE International Conference on Robotics and Automation (ICRA), Hong Kong, China, 31 May–07 June 2014, pp. 2589–2594, IEEE.
- Kumar A, Fu Z, Pathak D, et al. (2021) Rma: rapid motor adaptation for legged robots. *Robotics: Science and Systems*.
- Kumar A, Li Z, Zeng J, et al. (2022) Adapting rapid motor adaptation for bipedal robots. In: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Kyoto, Japan, 23–27 October 2022, pp. 1161–1168.
- Landau ID, Lozano R, M'Saad M, et al. (2011) *Adaptive Control: Algorithms, Analysis and Applications*. Berlin, Germany: Springer Science & Business Media.
- Landry B, Lorenzetti J, Manchester Z, et al. (2022) Bilevel optimization for planning through contact: a semidirect method. *Robotics Research: The 19th International Symposium ISRR*. Berlin, Germany: Springer, 789–804.
- Le Cleac'h S, Howell TA, Yang S, et al. (2024) Fast contact-implicit model predictive control. *IEEE Transactions on Robotics* 40: 1617.
- Lee Y, Sun SH, Somasundaram S, et al. (2018) Composing complex skills by learning transition policies. In: International Conference on Learning Representations, New Orleans, Louisiana, USA, 6–9 May 2019.
- Lee J, Hwangbo J, Wellhausen L, et al. (2020) Learning quadrupedal locomotion over challenging terrain. *Science Robotics* 5(47): eabc5986.
- Li Z, Cummings C and Sreenath K (2020) Animated cassie: a dynamic relatable robotic character. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, 24 October 2020 - 24 January 2021, pp. 3739–3746.
- Li Z, Cheng X, Peng XB, et al. (2021) Reinforcement learning for robust parameterized locomotion control of bipedal robots. In: 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–05 June 2021, pp. 2811–2817, IEEE.
- Li Y, Li J, Fu W, et al. (2023a) Learning agile bipedal motions on a quadrupedal robot. *arXiv preprint arXiv:2311.05818*.

- Li Z, Peng XB, Abbeel P, et al. (2023b) Robust and versatile bipedal jumping control through reinforcement learning. In: Robotics: Science and Systems XIX, Daegu, Republic of Korea, 10–14 July 2023.
- Lim D, Kim MJ, Cha J, et al. (2023) Proprioceptive external torque learning for floating base robot and its applications to humanoid locomotion. In: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 01–05 October 2023.
- Liu S, Xu M, Huang P, et al. (2023) Continual vision-based reinforcement learning with group symmetries. In: Conference on Robot Learning, Atlanta, USA, 06 November 2023, pp. 222–240, PMLR.
- Ljung L (1998) System identification. *Signal Analysis and Prediction*. Berlin, Germany: Springer, 163–173.
- Ma WL, Kolathaya S, Ambrose ER, et al. (2017) Bipedal robotic running with durus-2d: bridging the gap between theory and experiment. In: Proceedings of the 20th international conference on hybrid systems: computation and control, Pittsburgh, PA, USA, April 18–20, 2017, pp. 265–274.
- Marcucci T, Gabbicini M and Artoni A (2016) A two-stage trajectory optimization strategy for articulated bodies with unscheduled contact sequences. *IEEE Robotics and Automation Letters* 2(1): 104–111.
- Margolis GB and Agrawal P (2023) Walk these ways: tuning robot control for generalization with multiplicity of behavior. In: Conference on Robot Learning, Auckland, New Zealand, 14 December 2022, pp. 22–31, PMLR.
- Margolis G, Yang G, Paigwar K, et al. (2022) Rapid locomotion via reinforcement learning. In: Robotics: Science and Systems, New York City, NY, USA, June 27–1 July 2022.
- Meduri A, Shah P, Viereck J, et al. (2023) Biconmp: a nonlinear model predictive control framework for whole body motion planning. *IEEE Transactions on Robotics* 39(2): 905–922.
- Meuleau N, Peshkin L, Kim KE, et al. (1999) Learning finite-state controllers for partially observable environments. In: Proceedings of the Fifteenth conference on Uncertainty in artificial intelligence, Stockholm Sweden, July 30–1 August 1999, pp. 427–436.
- Miki T, Lee J, Hwangbo J, et al. (2022) Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics* 7(62): eabk2822.
- Moro FL and Sentis L (2019) Whole-body control of humanoid robots. In: *Humanoid Robotics: A Reference*. Dordrecht: Springer.
- Orin DE, Goswami A and Lee SH (2013) Centroidal dynamics of a humanoid robot. *Autonomous Robots* 35: 161–176.
- Peng XB, Andrychowicz M, Zaremba W, et al. (2018) Sim-to-real transfer of robotic control with dynamics randomization. In: 2018 IEEE international Conference on Robotics and Automation (ICRA), Brisbane, QLD, Australia, 21–25 May 2018, pp. 3803–3810.
- Peng XB, Coumans E, Zhang T, et al. (2020) Learning agile robotic locomotion skills by imitating animals. In: Robotics: Science and Systems (RSS), held virtually, 12–16 July 2020.
- Peng XB, Ma Z, Abbeel P, et al. (2021) Amp: adversarial motion priors for stylized physics-based character control. *ACM Transactions on Graphics* 40(4): 1–20.
- Posa M, Cantu C and Tedrake R (2014) A direct method for trajectory optimization of rigid bodies through contact. *The International Journal of Robotics Research* 33(1): 69–81.
- Pratt J, Koolen T, De Boer T, et al. (2012) Capturability-based analysis and control of legged locomotion, part 2: application to m2v2, a lower-body humanoid. *The International Journal of Robotics Research* 31(10): 1117–1133.
- Qi H, Chen X, Yu Z, et al. (2023) Vertical jump of a humanoid robot with cop-guided angular momentum control and impact absorption. *IEEE Transactions on Robotics* 39: 3154–3166.
- Radosavovic I, Xiao T, Zhang B, et al. (2024) Real-world humanoid locomotion with reinforcement learning. *Science Robotics* 9(89): eadi9579.
- Raibert MH, Brown HB Jr and Chepponis M (1984) Experiments in balance with a 3d one-legged hopping machine. *The International Journal of Robotics Research* 3(2): 75–92.
- Reher J and Ames AD (2021) Control lyapunov functions for compliant hybrid zero dynamic walking. *arXiv preprint arXiv:2107.04241*.
- Rodriguez D and Behnke S (2021) Deepwalk: omnidirectional bipedal gait by deep reinforcement learning. In: 2021 IEEE international conference on robotics and automation (ICRA), Xi'an, China, 30 May–05 June 2021, pp. 3033–3039.
- Rummel J, Blum Y, Maus HM, et al. (2010) Stable and robust walking with compliant legs. In: 2010 IEEE International Conference on Robotics and Automation, Anchorage, AK, USA, 3–7 May 2010, pp. 5250–5255, IEEE.
- Schulman J, Wolski F, Dhariwal P, et al. (2017) Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Sentis L and Khatib O (2006) A whole-body control framework for humanoids operating in human environments. In: Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006, Orlando, FL, USA, 15–19 May 2006, pp. 2641–2648, IEEE.
- SFU (2018) Sfu motion capture database. <https://mocap.cs.sfu.ca/>.
- Shao Y, Jin Y, Liu X, et al. (2021) Learning free gait transition for quadruped robots via phase-guided controller. *IEEE Robotics and Automation Letters* 7(2): 1230–1237.
- Siekmann J, Valluri S, Dao J, et al. (2020) Learning memory-based control for human-scale bipedal locomotion. In: Robotics Science and Systems, Corvallis, Oregon, USA, 12–16 July, 2020.
- Siekmann J, Godse Y, Fern A, et al. (2021a) Sim-to-real learning of all common bipedal gaits via periodic reward composition. In: 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–05 June 2021, pp. 7309–7315, IEEE.
- Siekmann J, Green K, Warila J, et al. (2021b) Blind bipedal stair traversal via sim-to-real reinforcement learning. In: Robotics: Science and Systems, Held Virtually, July 12–16, 2021.
- Singh RP, Xie Z, Gergondet P, et al. (2023) Learning bipedal walking for humanoids with current feedback. *IEEE Access*.
- Smith L, Kew JC, Peng XB, et al. (2022) Legged robots that keep on learning: fine-tuning locomotion policies in the real world. In: 2022 International Conference on Robotics and

- Automation (ICRA), Philadelphia, PA, USA, 23–27 May 2022, pp. 1593–1599, IEEE.
- Smith L, Kostrikov I and Levine S (2023) Demonstrating a walk in the park: learning to walk in 20 minutes with model-free reinforcement learning. *Robotics: Science and Systems (RSS) Demo* 2(3): 4.
- Spaan MT (2012) Partially observable markov decision processes. *Reinforcement learning: state-of-the-art*. Berlin, Germany: Springer, 387–414.
- Sreenath K, Park HW, Poulakakis I, et al. (2011) A compliant hybrid zero dynamics controller for stable, efficient and fast bipedal walking on mabel. *The International Journal of Robotics Research* 30(9): 1170–1193.
- Sreenath K, Park HW, Poulakakis I, et al. (2013) Embedding active force control within the compliant hybrid zero dynamics to achieve stable, fast running on mabel. *The International Journal of Robotics Research* 32(3): 324–345.
- Takenaka T, Matsumoto T and Yoshiike T (2009a) Real time motion generation and control for biped robot-1 st report: walking gait pattern generation. In: 2009 IEEE/RSJ International Conference on Intelligent Robots and Systems, St. Louis, MO, USA, 10–15 October 2009, pp. 1084–1091, IEEE.
- Takenaka T, Matsumoto T, Yoshiike T, et al. (2009b) Real time motion generation and control for biped robot-2 nd report: running gait pattern generation. In: 2009 IEEE/RSJ International Conference on Intelligent Robots and Systems, St. Louis, MO, USA, 10–15 October 2009, pp. 1092–1099, IEEE.
- Todorov E, Erez T and Tassa Y (2012) Mujoco: a physics engine for model-based control. In: 2012 IEEE/RSJ international conference on intelligent robots and systems, Vilamoura-Algarve, Portugal, 07–12 October 2012, pp. 5026–5033.
- van Marum B, Shrestha A, Duan H, et al. (2024) Revisiting reward design and evaluation for robust humanoid standing and walking. *arXiv preprint arXiv:2404.19173*.
- Vukobratović M and Borovac B (2004) Zero-moment point—thirty five years of its life. *International Journal of Humanoid Robotics* 01(01): 157–173.
- Wensing PM and Orin DE (2016) Improved computation of the humanoid centroidal dynamics and application for whole-body control. *International Journal of Humanoid Robotics* 13(01): 1550039.
- Wensing PM, Posa M, Hu Y, et al. (2023) Optimization-based control for dynamic legged robots. *IEEE Transactions on Robotics* 40: 43.
- Westenbroek T, Castaneda F, Agrawal A, et al. (2022) Lyapunov design for robust and efficient robotic reinforcement learning. In: 6th Annual Conference on Robot Learning, Auckland, New Zealand, 14 December 2022.
- Westervelt ER, Grizzle JW and Koditschek DE (2003) Hybrid zero dynamics of planar biped walkers. *IEEE Transactions on Automatic Control* 48(1): 42–56.
- Wu J, Xue Y and Qi C (2023a) Learning multiple gaits within latent space for quadruped robots. *arXiv preprint arXiv:2308.03014*.
- Wu P, Escontrela A, Hafner D, et al. (2023b) Daydreamer: world models for physical robot learning. In: Conference on Robot Learning, Auckland, New Zealand, 14 December 2022, pp. 2226–2240, PMLR.
- Xie Z, Berseth G, Clary P, et al. (2018) Feedback control for cassie with deep reinforcement learning. In: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Madrid, Spain, 1–5 October 2018, 1241–1246, IEEE.
- Xie Z, Clary P, Dao J, et al. (2020) Learning locomotion skills for cassie: iterative design and sim-to-real. *Proceedings of the Conference on Robot Learning* 100: 317–329.
- Xie Z, Da X, Van de Panne M, et al. (2021) Dynamics randomization revisited: a case study for quadrupedal locomotion. In: 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–05 June 2021, pp. 4955–4961, IEEE.
- Xiong X and Ames AD (2018) Bipedal hopping: reduced-order model embedding via optimization-based control. In: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Madrid, Spain, 1–5 October 2018, pp. 3821–3828, IEEE.
- Xiong X and Ames A (2022) 3-d underactuated bipedal walking via h-hip based gait synthesis and stepping stabilization. *IEEE Transactions on Robotics* 38(4): 2405–2425.
- Yang W and Posa M (2021) Impact invariant control with applications to bipedal locomotion. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Prague, Czech Republic, 2021, pp. 5151–5158, IEEE.
- Yang W and Posa M (2023) Impact-invariant control: maximizing control authority during impacts. *arXiv preprint arXiv:2303.00817*.
- Yang L, Li Z, Zeng J, et al. (2022) Bayesian optimization meets hybrid zero dynamics: safe parameter learning for bipedal locomotion control. In: 2022 International Conference on Robotics and Automation (ICRA), Philadelphia, PA, USA, 23–27 May 2022, pp. 10456–10462, IEEE.
- Yu W, Kumar VC, Turk G, et al. (2019) Sim-to-real transfer for biped locomotion. In: 2019 IEEE/RSJ international conference on intelligent robots and systems (IROS), Macau, China, 03–08 November 2019. pp. 3503–3510.
- Yu F, Batke R, Dao J, et al. (2022) Dynamic bipedal turning through sim-to-real reinforcement learning. In: 2022 IEEE-RAS 21st International Conference on Humanoid Robots

(Humanoids), Ginowan, Japan, 28–30 November 2022, pp. 903–910, IEEE.

Yunt K and Glocker C (2006) Trajectory optimization of mechanical hybrid systems using sumt. In: 9th IEEE International Workshop on Advanced Motion Control, 2006, Istanbul, Turkey, 27–29 March 2006, pp. 665–671, IEEE.

Zhu Y, Pan Z and Hauser K (2021) Contact-implicit trajectory optimization with learned deformable contacts using bilevel optimization. In: 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May–05 June 2021, pp. 9921–9927.

Appendix

1. Advantages of using an action filter

We used a Low Pass Filter (LPF) after the policy output, acting as an action filter. The LPF used in this work is a Butterworth low pass filter with a cut-off frequency of 4 Hz. LPF has been widely applied in control engineering including model-based optimal control methods (Gong and Grizzle, 2022) and model-free RL methods (Escontrela et al., 2022; Li et al., 2021; Peng et al., 2020; Smith et al., 2022) for legged locomotion. LPF can further smooth out the action from the RL-based policy, which further complements the smoothing rewards listed in Table 3. We conducted an ablation study by training a jumping-in-place policy (single-task training) *without* LPF, which still results in jumping but with worse learning performance (converged return) due to the resulting jittering motion, as shown in Figure 28. If we take out the LPF, we will need to increasing the weight on the smoothing term to highlight the importance of the smoothness of the entire motion. However, if the smoothing term takes too much weight in the entire reward composition, the robot may easily learn a suboptimal behavior by being stationary without exploring dynamic skills such as jumping. Therefore, LPF can help with reducing such reward tuning burden. Furthermore, since LPF

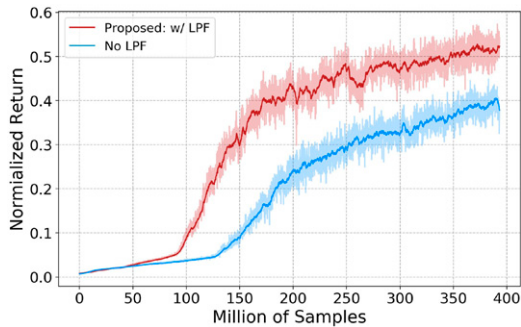


Figure 28. Ablation study on the use of Low Pass Filter (LPF) as an action filter for the training of the in-place jumping skill from scratch. Without using the LPF, the training return (blue curve) is much lower than the one using LPF (ours, red curve). These two policies are obtained by using the exact hyperparameters and training settings. The underlying reason is that it is harder for the RL-based policy to damp out high-frequency jittering motion without the use of LPF.

only helps to damp out the high-frequency changes from the action (policy's output), the entire system may still have oscillation (e.g., due to a large joint velocity), having the smoothing reward is still necessary to smooth the entire closed-loop system. These two parts complement each other.

2. Learning to combine a standing skill

Enabling the robot to learn standing along with other locomotion skills is useful for real-world applications, as it is essential to retrieve the physical robot hardware (while the robot is not moving).

For the walking skill, after the robot has mastered the diverse walking tasks after the task randomization stage, we added a sub-stage. In the episode of this sub-stage of training, a standing skill is commanded after a random timespan of walking, and *will last until the end of episode*. The robot is informed to perform standing by two changes: (1) the reference motion input to the control policy is changed to a nominal standing motion (the preview of reference motion is now a stack of three same standing motor positions), and (2) the reward is formulated to encourage performing stationary standing skill. The reward for standing has the same formulation used for training dynamic locomotion skills listed in Table 3, and the weights of smoothing terms, such as motor velocity and change of action, are increased with other terms remaining unchanged. The increased smoothing weight is to encourage the robot to maintain a stationary standing pose. After the robot has acquired the skills of walking, standing, and the transition from an arbitrary state from walking to standing, we can move on to dynamics randomization. Starting from this stage, the standing skill will be only introduced in the middle of the training of walking and last for a random finite timespan. In

Table 6. Command range for different Skills.

Task parameters	Range
Walking	
Sagittal velocity $\dot{q}_x^d$	$[-1.5, 1.5]$ m/s
Lateral velocity $\dot{q}_y^d$	$[-0.6, 0.6]$ m/s
Turning velocity $\dot{q}_\psi^d$	$[-45, 45]$ deg/s
Walking height q_z^d	$[0.65, 1.0]$ m
Running	
Sagittal velocity $\dot{q}_x^d$	$[2.0, 5.0]$ m/s
Lateral velocity $\dot{q}_y^d$	$[-0.75, 0.75]$ m/s
Turning velocity $\dot{q}_\psi^d$	$[-30, 30]$ deg/s
Jumping	
Sagittal landing location q_x^d	$[-0.5, 1.5]$ m
Lateral landing location q_y^d	$[-1.0, 1.0]$ m
Turning direction at landing q_ψ^d	$[-100, 100]$ deg
Change of elevation e_z^d	$[-0.5, 0.5]$ m

Table 7. Number of training iterations used for different locomotion skills. Each iteration collects a batch of 65,536 samples.

Walking				
Single-task	Task randomization	Combining standing	Dynamics randomization	Added perturbation training
6000	8000	2000	8000	5000
Running				
Single-task	Task randomization	Combining standing	Dynamics randomization	Added perturbation training
6000	18,000	5000	15,000	5000
Jumping				
Single-task	Task randomization			Dynamics randomization
6000	12,000			20,000

this way, the robot can also learn the transition back from standing to walking.

For the running skill, the POMDP design for this sub-stage is similar to the one for walking. However, since transiting from fast running to standing is more challenging, we provide an additional reference motion of transition from running to standing, retargeted from human mocap. This could facilitate the training during the transition phase. For the jumping skill, since it is an aperiodic skill and the robot is required to maintain a standing pose after landing, we do not need such an additional sub-stage of training.

This method may also be useful to combine different locomotion skills (like combining all walking, running, and jumping) using one single policy. This could potentially solve the problem of developing multi-skill policy for highly-dynamic legged locomotion control. We note this as a possible extension of this work.

3. command range for different skills

The training ranges for the command of three locomotion skills (walking, running, and jumping) developed in this work are detailed in Table 6. During a training episode, the given command is drawn uniformly from the listed range.

4. Hyperparameters used in training

The number of training (PPO) iterations used to develop different locomotion skills is detailed in Table 7. Furthermore, the hyperparameters of PPO are reported in Table 8, and they are consistent over different locomotion skills.

5. Comparison of use of different history lengths

We conducted an ablation study to investigate the effects of varying history lengths on a non-recurrent policy that

Table 8. Hyperparameters used in PPO training. These are consistent among different locomotion skills.

Hyperparameter	Value
PPO iteration batch size	65,536
PPO clip rate	0.2
Optimization step size (both actor and critic)	$1e^{-4}$
Optimization batch size	8192
Optimization epochs	2
Discount factor (γ)	0.98
GAE smoothing factor (λ)	0.95

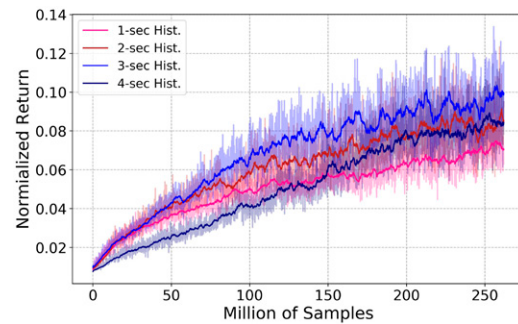


Figure 29. Learning performance using different lengths of the robot's I/O history when training a single-task running policy with dynamics randomization. All of these policies used the proposed dual-history-based policy. When increasing the explicit length of robot history from 1 second (pink curve), 2 seconds (red curve, as used in this work), and 3 seconds (blue curve), we observe an increase in learning performance. However, if we keep increasing the history length, such as to 4 seconds (dark blue curve), the improvement of the learning performance may get saturated.

explicitly requires a specified history length. Using bipedal running training as an example, Figure 29 illustrates the learning performance of training a single-task running policy (which has completed the first-stage training) with extensive dynamics parameters and varying lengths of robot I/O history.

As suggested by Figure 29, increasing the history length, such as from 1 second to 2 seconds, or from 2 seconds to 3 seconds, consistently enhances learning performance. This improvement occurs because a longer I/O history provides more information on the robot's dynamics parameters, aiding in better state estimation. However, continually extending the history length may not always be beneficial, as it can introduce redundant information that the robot must filter out. This is evidenced by a drop in learning performance when history length increases from 3 seconds to 4 seconds, as shown in Figure 29. In bipedal locomotion control, where remembering events that happened further earlier is less crucial, a longer history could lead more training samples for the robot to distill useful information. Therefore, we recommend using a history length that spans a relatively short timespan, such as the 2 seconds used in this work, for more stable training and relatively good performance in learning bipedal locomotion control. For readers interested in adopting this method for their robots, finding the optimal history length could be beneficial for specific tasks, but we suggest starting with a 2-second history, which has been tested extensively in this work.

6. Comparison of use of different temporal encoders

To explore the effects of different neural network structures encoding the robot's I/O long history, we conducted an ablation study and benchmarked policies based on TCN and LSTM. TCN, a non-recurrent structure that encodes temporal information, is used in Lee et al. (2020), while LSTM, a recurrent structure, is utilized in other bipedal locomotion control works like Siekmann et al. (2020). To eliminate potential confounding factors such as poor implementation, we adopted the training algorithms (PPO and recurrent PPO) and training hyperparameters open-sourced from Siekmann et al. (2020), which underpin many subsequent studies such as Siekmann et al. (2021b,a); Crowley et al. (2023). The MDP design (training environment) employed the walking and jumping scenarios developed in this work.

As shown in Figure 30(a), providing the base policy with a short 4-timestep I/O history alongside the TCN encoder significantly improves learning performance compared to using TCN alone (while still providing the base policy with immediate last state feedback, which is the form used in Lee et al. (2020)). This suggests that the dual-history approach can consistently enhance learning performance for non-recurrent structures that encode an explicit history length. Note that in this comparison, we use the same TCN implementation as detailed in Table S5 in Lee et al. (2020).

Conversely, as depicted in Figure 30(b), the LSTM-based policy shows no significant improvement whether a short history is provided or not. Additionally, it tends to converge to a lower return plateau compared to the TCN-based policy, even though the TCN used hyperparameters tuned for LSTM. This might indicate that LSTM encoders may learn to focus more on recent short history and pay less attention to older history.

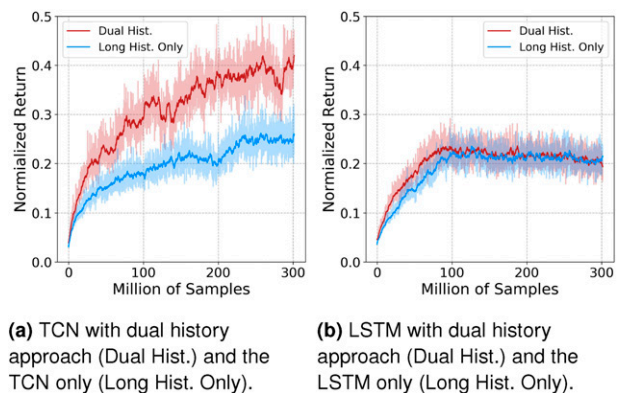


Figure 30. Learning performance using different neural network architectures to encode the robot's I/O history when training a single-task walking policy with dynamics randomization (walking forward at a fixed speed, after finishing single-stage training). For both Long Hist. Only methods, we still provide explicit immediate state feedback alongside the temporal encoder. As shown in Figure 30a, using the proposed dual-history approach by providing an explicit short I/O history alongside the TCN encoder, the learning performance is much better than the TCN only. The TCN encodes 2-second robot I/O history and has three layers with filter sizes of [34,34,34], a kernel size of 5, a dilation base of 2, and a stride size of 2, with ReLU activation, as suggested in Lee et al. (2020). Figure 30(b) shows that the dual-history approach will not help with the LSTM-based policy. The LSTM encoder has 1 layer of 128 units. However, both TCN with dual-history approach and Long Hist. Only perform better than the LSTM-based policy while using the hyperparameters tuned for LSTM. It suggests that LSTM may only learn to leverage a recent short history and converge to a more suboptimal policy. (a) TCN with dual history approach (Dual Hist.) and the TCN only (Long Hist. Only), (b) LSTM with dual history approach (Dual Hist.) and the LSTM only (Long Hist. Only).

Although LSTM hidden states are only reset at episode ends, they may quickly converge to a suboptimal policy focusing primarily on recent history, explaining why providing additional short history does not enhance performance and why LSTM underperforms compared to TCN that was explicitly trained to utilize a long history. This observation aligns with findings reported in Lee et al. (2020) and Singh et al. (2023).

Moreover, extending the LSTM-based policy to learn jumping skills resulted in training failure, even using the same architecture and hyperparameters from Siekmann et al. (2020) that reproduced the same walking skill developed in this work, as illustrated in Figure 31. Please note that we are not asserting that it is impossible for LSTM to learn aperiodic skills like jumping, but it is difficult without carefully tuning hyperparameters during training. In contrast, our method provides a straightforward and general approach for learning various skills, using unified hyperparameters across all tasks. After finalizing the training for walking, we did not adjust any hyperparameters for jumping or running.

In summary, the dual history approach consistently helps learning performance for policies recording explicit robot history (non-recurrent policies like TCN

and 1D CNN), but does not improve recurrent policies like LSTM. Moreover, recurrent policies are sensitive to hyperparameter tuning across different MDPs (such as locomotion control) and may more readily converge to suboptimal policies. Therefore, dual-history-based non-recurrent policy could be more favorable for bipedal locomotion control cases.

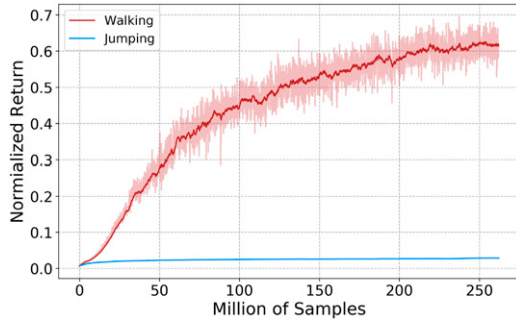


Figure 31. Learning performance for training walking (red curve) and jumping (blue curve) skills *from scratch* using the same neural network including the LSTM architecture with the same hyperparameters used in Siekmann et al. (2020). It shows that while the LSTM can learn the walking skill, it may struggle to learn highly dynamic locomotion skills like jumping (shown as a flat curve). This is not to suggest that LSTM cannot learn aperiodic jumping skills, but rather to highlight its sensitivity to hyperparameter tuning across different MDPs (different locomotion skills). This contrasts with the non-recurrent-based policy like the 1D CNN used in this work, which employs a unified set of hyperparameters for all different skills due to the ease of training.

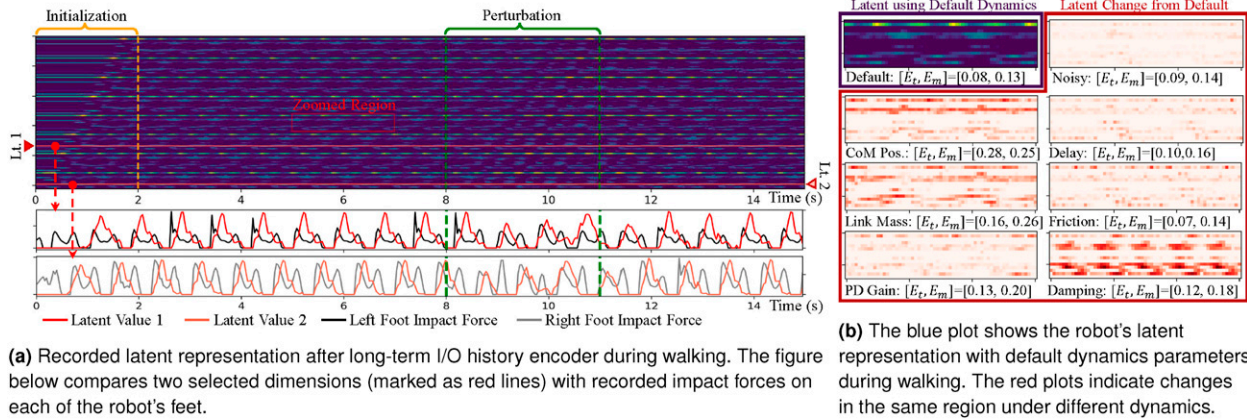


Figure 32. Adaptivity test on the obtained walking policy in simulation (MuJoCo). The robot is commanded to walk forward at 0.6 m/s with no lateral or turning movement at normal height (0.95 m). Figure 32(a) records the latent embedding after the policy's long I/O history encoder over 15 seconds. An external backward perturbation force of 30 N is applied on the robot base from 8 to 11 seconds. We can observe the changes in the latent embedding with the existence of the perturbations. The image below shows a strong correlation between the two selected latent dimensions with the robot's impact force or contact event. Figure 32(b) shows the change of the latent embedding (the same zoomed region marked as the red bounding box in Figure 32(a)) with respect to the change of different dynamics parameters during the same walking task. These ablation studies are the same as running conducted in Figure 9(b). The control performance metrics, tracking error E_t and motion tracking error E_m , show small changes with a large change in the dynamics parameters. (a) Recorded latent representation after long-term I/O history encoder during walking. The figure below compares two selected dimensions (marked as red lines) with recorded impact forces on each of the robot's feet, (b) The blue plot shows the robot's latent representation with default dynamics parameters during walking. The red plots indicate changes in the same region under different dynamics.

7. Latent visualization of walking policy

The results of adaptivity test on the obtained walking policy by the proposed method are presented in Figure 32. The adaptivity test is similar to the ones conducted on the running policy discussed in Sec. 8. It evaluates the change of latent embedding after the long I/O history encoder when the robot is commanded to walk at a constant forward speed of 0.6 m/s. The findings are consistent with the ones observed from the tests on other skills reported in Sec. 8: we show that the latent embedding is able to capture the time-variant changes like external perturbation and contact events (Figure 32(a)) and time-invariant dynamics shifts (Figure 32(b)). During the ablation study on the dynamics shifts, the latent embedding differs when dynamics parameters change, but the resulting control performance shows little change. This indicates the adaptivity of the walking policy.

8. Saliency map

To further understand how the policy adjusts its action based on the environment changes, we visualize the saliency map in Figure 33 as used in Lee et al. (2020). The saliency map reflects how important each input dimension is to the neural network output (i.e., where does the policy focus).

As shown in the saliency map of the MLP base, which produces the final policy action on the entire input, the robot focuses more on the short I/O history, particularly the most recent observation, highlighting the importance of the short I/O history, during both running (Figure 33(a)) and walking (Figure 33(b)).

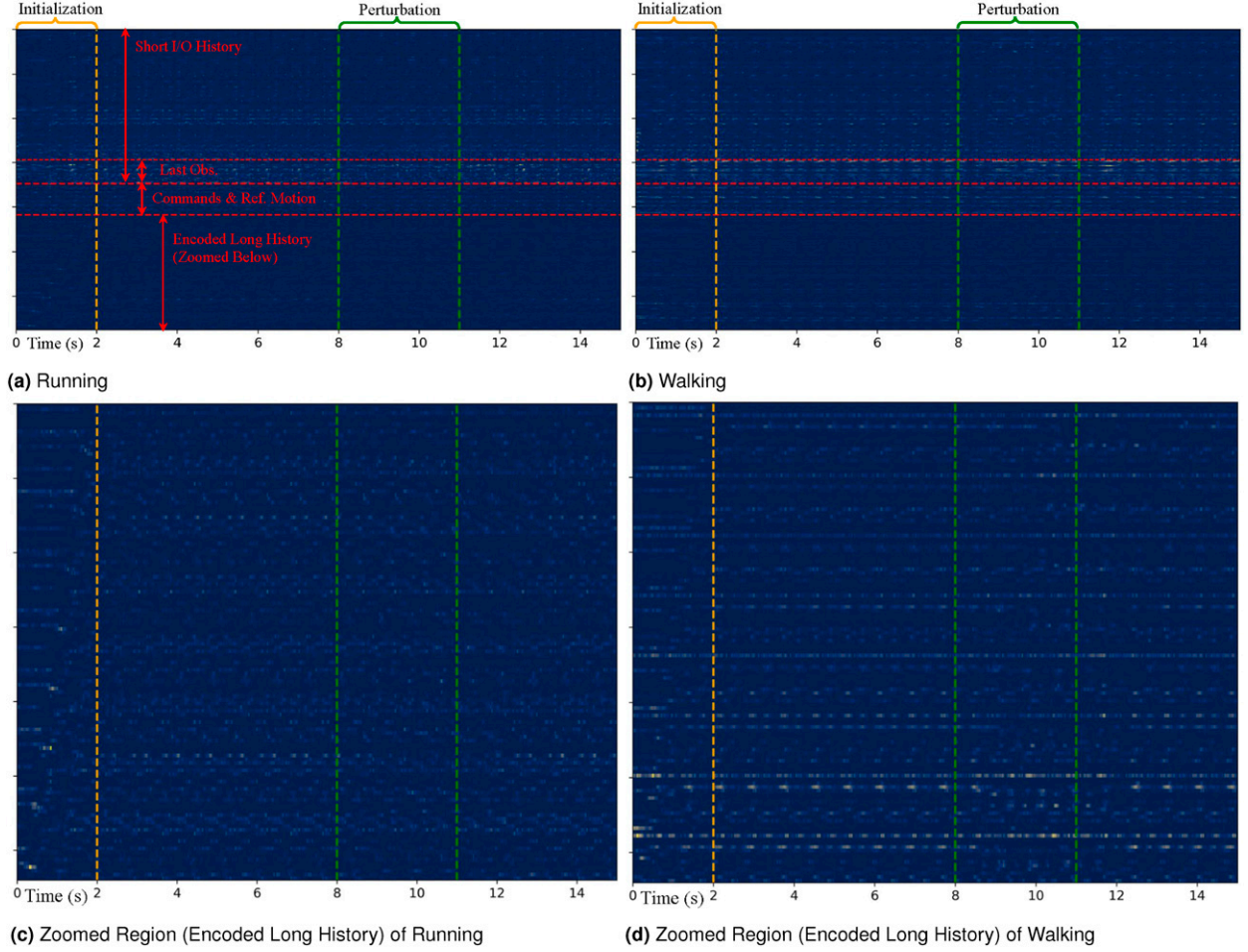


Figure 33. The saliency map of the MLP base with respect to the entire input while the robot is (a) running or (b) walking with the existence of a perturbation from 8 to 11 seconds in a 15-second test. The zoomed regions of the saliency map w.r.t. latent embedding from the long history encoder are shown in detail in (c) for running and (d) for walking. These are the same tests reported in Figures 9 and 32. The pixels with a lighter color represent the MLP base focus more on this value (higher saliency). These maps show that the MLP base, which produces the policy action, focuses more on the short I/O history, especially the latest observation, highlighting the importance of short I/O history in the input. Moreover, if we take a close look at the encoded long I/O history (c and d), the robot can also focus on different regions of the latent embedding to deal with the change in environments and therefore adjust its output. Therefore, it indicates that the inclusion of long I/O history is also beneficial, which also provides insights on the benchmark conducted in Figure 6. (a) Running, (b) Walking, (c) Zoomed Region (Encoded Long History) of Running, (d) Zoomed Region (Encoded Long History) of Walking.

We can take a close look at the zoomed regions (encoded long history) of the entire saliency map in the same test of the running (Figure 9(a)) and walking (Figure 32(a)). The saliency maps with respect to the encoded long history have different focuses on different parts of the embeddings from the long-history encoder with the existence of the external perturbation during running (Figure 33(c)) or walking (Figure 33(d)). This further suggests that the proposed RL policy could utilize the long history information to adjust the action for different control scenarios, i.e., the long I/O history is useful.

From the extensive ablation study in Figure 6, we observed differences in learning performance and sim-to-real transfer depending on the use of history. It will result in degradation by either removing the short I/O history, even keeping the most recent observation that drew the most

attention, which is Long History Only in Figure 5(d), or removing the long I/O history (Short History Only, Figure 5(e)). These saliency maps on the entire input space provide insights into the reason behind this.

9. Errors of estimator in high-speed running

In this section, we present the estimation errors of the state estimator (EKF) we used in the high-speed running. In Figure 34, we compare the robot ground-truth sagittal velocity $\dot{q}_x$ during running obtained from simulation and the corresponding estimated velocity from the estimator. During high-speed running (larger than 3 m/s), we observe a large estimation error from the estimator to which we only have access during real-world experiments. This suggests

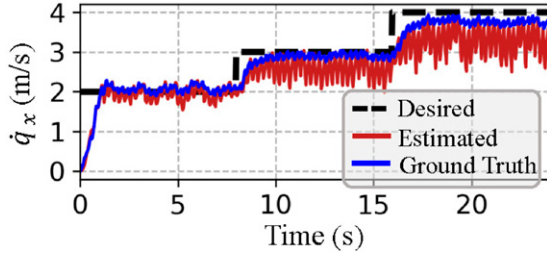


Figure 34. The large estimation error using the robot onboard velocity estimator (based on EKF) during high-speed running *in simulation*. In this figure, the robot is controlled by the proposed running policy to track variable commands (black dashed line) in simulation, the estimated sagittal velocity $\dot{q}_x$ is recorded as the red line while the robot's actual running speed is recorded as the blue line. The ground-truth speed is obtained from the simulator. Although showing accurate results under slow speed (such as 2 m/s), the estimated velocity shows a significant error in the high-speed region (above 3 m/s) compared to the ground-truth speed. The robot's actual speed tends to be the upper envelope of the estimated speed. In real-world experiments, we only have access to the state estimator whose result we can report, such as the running speed tracking results in Figure 20(b). This comparison shows that the robot's actual running speed in the real world is faster than the reported estimated value and closer to the command.

that the robot's actual running speed in the real world is faster than the recorded estimated speed, and may be the upper envelope of the estimation. This large estimation error is further supported when comparing the estimated speed and the robot's actual average speed (traveled distance/elapsed time). These suggest a smaller actual tracking error during high-speed running than the experiment logs suggested. This also suggests the necessity of including this estimator during training rollout to let the robot train with the observation data produced by this inaccurate estimator. But still, the robot tends to have a noticeable tracking error during running in the high-speed region due to the sim-to-real gap and robot hardware limitation. We note that developing a reliable state estimator for dynamic bipedal locomotion skills using RL could be an interesting future work.

10. Diverse bipedal jumps

Using the same flat-ground jumping policy that realized various bipedal jumps demonstrated in Sec. 10.3, we further evaluate the capacity to jump to other different targets in the real world, as showcased in Figure 35.

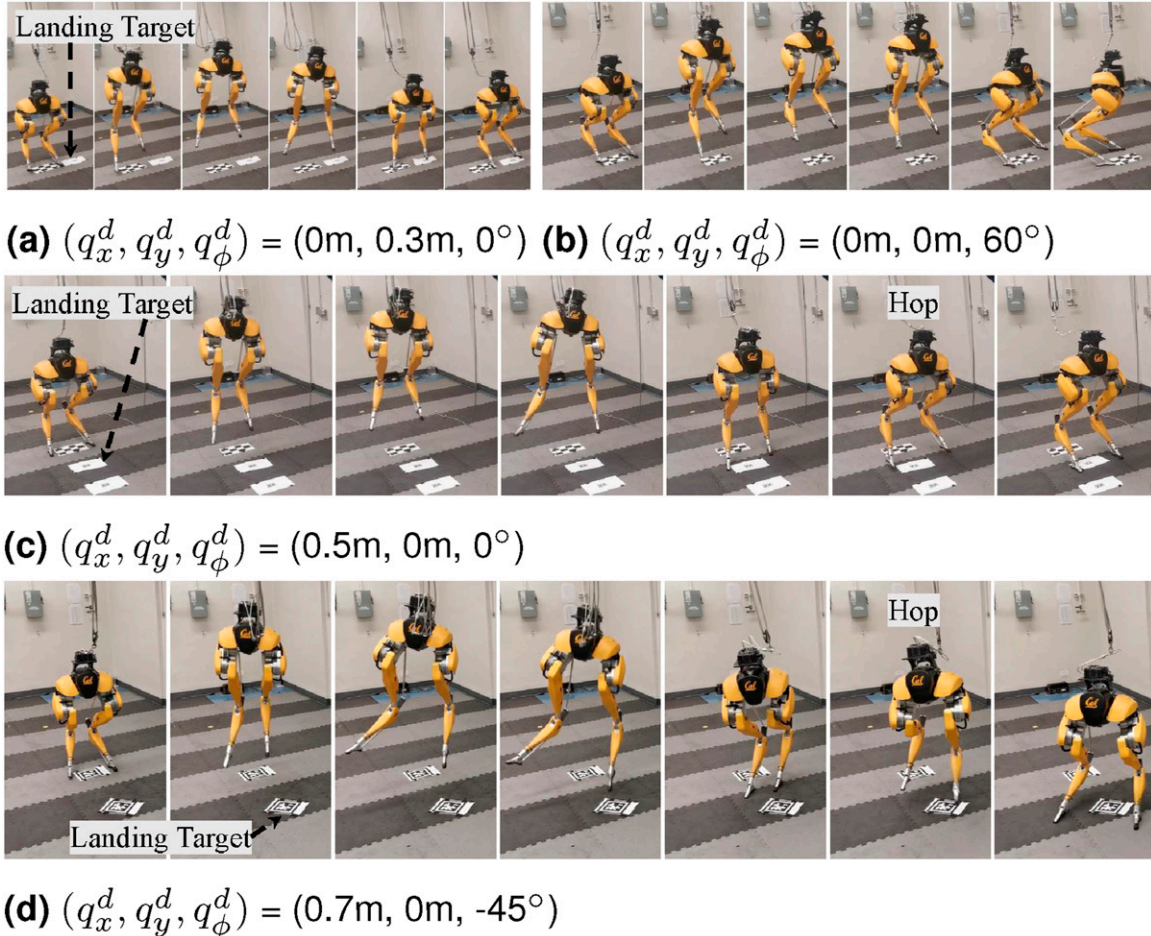


Figure 35. More bipedal jumps using the same flat-ground jumping policy. The paper tag on the ground indicates the jumping target. (a) $(q_x^d, q_y^d, q_\phi^d) = (0\text{ m}, 0.3\text{ m}, 0^\circ)$, (b) $(q_x^d, q_y^d, q_\phi^d) = (0\text{m}, 0\text{ m}, 60^\circ)$, (c) $(q_x^d, q_y^d, q_\phi^d) = (0.5\text{ m}, 0\text{ m}, 0^\circ)$, (d) $(q_x^d, q_y^d, q_\phi^d) = (0.7\text{ m}, 0\text{ m}, -45^\circ)$.